

Jonah Benjamin

CMSC 433: Midterm Exam (Spring 2024)

Question 1 (20 points)

Consider the two standard folds in Haskell, `foldl` and `foldr`, whose definitions are given below:

```
foldr :: (a -> b -> b) -> b -> [a] -> b
foldr f b [] = b
foldr f b (x:xs) = f x (foldr f b xs)

foldl :: (b -> a -> b) -> b -> [a] -> b
foldl f b [] = b
foldl f b (x:xs) = foldl f (f b x) xs
```

Now consider the following two very similar but distinct folds over lists of integers:

```
f1 :: [Int] -> Int
f1 = foldr (-) 0

f2 :: [Int] -> Int
f2 = foldl (-) 0
```

Write two Dafny programs, one corresponding to each of `f1` and `f2`, but operating on arrays rather than lists. We've given you the method signatures to get you started:

```
method f1(a:array<int>) returns (out:int) {
  var i := a.Length - 1;
  out := 0;
  while (i >= 0) {
    out := out - a[i];
    i := i - 1;
  }
  return out;
}
```

```
method f2(a:array<int>) returns (out:int) {
  var i := 0;
  out := 0;
  while (i < a.Length) {
    out := out - a[i];
    i := i + 1;
  }
  return out;
}
```

Question 2 (20 points)

For each of the Haskell expressions below, write their (most general) Haskell type or "ill-typed" if it contains a type error. The type signatures of all functions below are provided in the appendix at the end.

(a) $\backslash x \rightarrow x$

Example answer:

$a \rightarrow a$

(b) $\backslash x y \rightarrow (x,y)$

$a \rightarrow b \rightarrow (a,b)$

(c) $\backslash x y \rightarrow \text{if } x == y \text{ then show } x \text{ else show } (x,y)$

$\exists a. a \rightarrow a \rightarrow \text{String}$

(d) $\backslash x l \rightarrow x : l ++ l ++ [x]$

$a \rightarrow [a] \rightarrow [a]$

(e) $\text{foldr } (\text{const } 1) \ [a] \rightarrow \text{Int}$

(f) $(^2, 42)$

Ill typed

(g) $(.) ^2 42$

$a \rightarrow (\text{String}, a)$

(h) $\text{reverse} . \text{reverse}$

$[a] \rightarrow [a]$

(i) $\backslash l \rightarrow \text{filter } (< 42) \ (1 ++ 1)$

$[\text{Int}] \rightarrow [\text{Int}]$

(j) $\text{let } f \ x = x \text{ in } (f 'a', f \text{ True})$

$(a \rightarrow b) \rightarrow c \rightarrow c$

(k) $\text{fmap } (\text{fmap } (+1)) \ [\text{Nothing}]$

Ill typed

$\text{fmap } (\text{fmap } (+1)) \ [\text{Nothing}]$

Question 3 (20 points)

For each of the types below, write a Haskell expression that has that type. Don't write trivial expressions (such as [], Nothing, or undefined) unless there is no other option. You can use any function from the appendix; do syntax, list comprehensions, or any valid Haskell.

(a) $\text{Int} \rightarrow \text{Int}$

Example answers:

$\backslash x \rightarrow x + 1$
 $(+1)$

(b) $\text{Bool} \rightarrow [\text{Bool}]$

$\backslash x \rightarrow \text{if } x \text{ then } [x] \text{ else } []$

(c) $a \rightarrow \text{Maybe } b$

undefined

(d) $(\text{Int} \rightarrow \text{Char} \rightarrow \text{Bool}) \rightarrow [\text{Int}] \rightarrow [\text{Char}] \rightarrow [\text{Bool}]$

fun f (h:t) (h2:t2) = let x = f (h+1) h2 in if (h2 == 'a') == x then f (Just a) (Just b) else f (Just a) (Nothing)
fun f (a->b->c) -> Maybe a -> Maybe b -> Maybe c
fun f (Just a) (Just b) = Just (f a b)
fun f (Just a) (Nothing) = Nothing
fun f (Nothing) (Just b) = Nothing
fun f (Nothing) (Nothing) = Nothing
let x = a h in let y = b x in ret (h:t) = f x in (x,h):[]

(h) $\text{Eq } a \Rightarrow a \rightarrow [a] \rightarrow [a]$

fun x t = x:t (or [])

(i) Show $a \Rightarrow [a] \rightarrow \text{IO String}$

foldr (\x -> putStrLn x) ()

(j) $(a,b) \rightarrow (a \rightarrow b \rightarrow c) \rightarrow c$

\ (x,y) fun -> (fun x y)

(k) $((a,b) \rightarrow c) \rightarrow c$

undefined

fun x = \ (a,b) -> x (a,b)

e. $(\backslash (Just a) (Just b) \rightarrow Just (f a b))$

d. $\backslash f (h:t) (h2:t2) \rightarrow \text{let } x = h+1 \text{ in if } (f h h2) == (h2 == 'a') \text{ then } [True] \text{ else } [False]$

Question 5 (20 points)

Implement the following Haskell functions:

- (a) Implement a function `weave` that given two lists with elements of the same type, returns a list with elements alternating between the two lists. For example:

`weave [1,2,3] [4,5,6] = [1,4,2,5,3,6]`

You can assume that the lists have the same length.

`weave (h:t) (h2:t2) = h:[h2] ++ weave t t2`
`weave [] [] = []`

- (b) Implement a function `powerSet`, that given a list of elements of some type, computes the list of all of its sublists, including the empty list and the original list itself, in any order. For example:

`powerSet [1,2,3] = [[], [1], [2], [3], [1,2], [1,3], [2,3], [1,2,3]]`

`powerSet (h:t) = [h:] ++ powerSet t`
`powerSet [] = []`

Ceren Meri
117996739

CMSC 433: Midterm Exam (Spring 2024)

Question 1 (20 points)

Consider the two standard folds in Haskell, `foldl` and `foldr`, whose definitions are given below:

```
foldr :: (a -> b -> b) -> b -> [a] -> b      [1, 2, 3]
foldr f b [] = b
foldr f b (x:xs) = f x (foldr f b xs)    f 1 foldr (0 - 3)
```

```
foldl :: (b -> a -> b) -> b -> [a] -> b
foldl f b [] = b
foldl f b (x:xs) = foldl f (f b x) xs
```

Now consider the following two very similar but distinct folds over lists of integers:

```
f1 :: [Int] -> Int
f1 = foldr (-) 0
```

```
f2 :: [Int] -> Int
f2 = foldl (-) 0
```

Write two Dafny programs, one corresponding to each of `f1` and `f2`, but operating on arrays rather than lists. We've given you the method signatures to get you started:

```
method f1(a:array<int>) returns (out:int) {
  out := 0;
  var i := a.Length - 1;
  while (i >= 0)
    invariant i >= -1
  }
  out := out - a[i];
  i := i - 1;
}
```

```
method f2(a:array<int>) returns (out:int) {
  out := 0;
  var i := 0;
  while (i < a.Length)
    invariant i <= a.Length
  }
  out := out - a[i];
  i := i + 1;
}
```

Question 2 (20 points)

For each of the Haskell expressions below, write their (most general) Haskell type or "ill-typed" if it contains a type error. The type signatures of all functions below are provided in the appendix at the end.

(a) $\backslash x \rightarrow x$

Example answer:

$a \rightarrow a$

(b) $\backslash x y \rightarrow (x, y)$

$a \rightarrow b \rightarrow (a, b)$

(c) $\backslash x y \rightarrow$ if $x == y$ then show x else show (x, y)

$a \rightarrow a \rightarrow \text{String}$

(d) $\backslash x l \rightarrow \backslash x : l \mapsto ++ [x]$

$8 [10]$

$[8, 10] ++$

$[10]$

$[8]$

$a \rightarrow [a] \rightarrow [a]$

* (e) $\text{foldr} (\text{const } \#)$

$[a] \rightarrow [Int]$

* (f) $(\cdot "42")$

(g) $(\cdot) "42"$

* (h) $\text{reverse} \cdot \text{reverse}$

$[a] \rightarrow [a]$

(i) $\backslash l \rightarrow \text{filter} (< 42) (l ++ l)$

$[Int] \rightarrow [Int]$

(j) $\text{let } f x = x \text{ in } (f 'a', f \text{True})$

Type error, cannot pass a char and a bool to f.

* (k) $\text{fmap} (\text{fmap} (+1)) [\text{Nothing}]$

fine

$((\text{Maybe} \rightarrow \text{Maybe Int}) \rightarrow \text{Maybe} \rightarrow \text{Maybe Int}) \rightarrow [\text{Maybe}] \rightarrow [\text{Maybe}]$

Question 3 (20 points)

For each of the types below, write a Haskell expression that has that type. Don't write trivial expressions (such as [], Nothing, or undefined) unless there is no other option. You can use any function from the appendix; do syntax, list comprehensions, or any valid Haskell.

(a) $\text{Int} \rightarrow \text{Int}$

Example answers:

$\backslash x \rightarrow x + 1$
 $(+1)$

(b) $\text{Bool} \rightarrow [\text{Bool}]$

$\backslash x \rightarrow \text{if } x \text{ then } [x] \text{ else } [\text{False}]$

* (c) $a \rightarrow \text{Maybe } b$

$\backslash x \rightarrow \text{Nothing}$

(d) $(\text{Int} \rightarrow \text{Char} \rightarrow \text{Bool}) \rightarrow [\text{Int}] \rightarrow [\text{Char}] \rightarrow [\text{Bool}]$

$\backslash f \backslash x \backslash xs \rightarrow (g : ys) \rightarrow \text{if } x == 2 \&\& y == 'a' \&\& f x y \text{ then } [\text{True}] \text{ else } [\text{False}]$

(e) $(a \rightarrow b \rightarrow c) \rightarrow \text{Maybe } a \rightarrow \text{Maybe } b \rightarrow \text{Maybe } c$

$\backslash f \backslash x \backslash y \rightarrow f (\text{maybeToList } x ++ \text{maybeToList } y)$

(f) $(a \rightarrow b) \rightarrow (b \rightarrow \text{Bool}) \rightarrow [a] \rightarrow [b]$

$\backslash f \backslash g \backslash (x : xs) \rightarrow \text{do } res \leftarrow f x$
 $\text{if } g \text{ res} \leftarrow g \text{ res}$
 $\text{if } res2 \leftarrow g \text{ res}$
 $\text{if } res2 \text{ then } [res] \text{ else } [res]$

(g) $\text{Maybe } a \rightarrow (a \rightarrow [b]) \rightarrow [(a,b)]$

$\backslash x \backslash f \rightarrow \text{case } x \text{ of}$
 $\text{Just } a \rightarrow \text{let } (y : ys) = f x \text{ in } [(a, y)]$
 $\text{Nothing} \rightarrow \text{let } (y : ys) = f x \text{ in } [\text{Nothing}, y]$

(h) $\text{Eq } a \Rightarrow a \rightarrow [a] \rightarrow [a]$

$\backslash x \backslash y \rightarrow x : y$

(i) Show $a \Rightarrow [a] \rightarrow \text{IO String}$

$\backslash (x : xs) \rightarrow \text{putStrLn "Hello"}$

(j) $(a,b) \rightarrow (a \rightarrow b \rightarrow c) \rightarrow c$

$\backslash (x,y) \cdot f \rightarrow f x y$

(k) $((a,b) \rightarrow c) \rightarrow c$

undefined?

Ceren Mert

Question 4 (20 points)

Consider the following Dafny program that inefficiently calculates the cube of a number:

```
method Cube(m:int) returns (y:int)
  requires m > 0
  ensures y == m*m*m
{
  y := 0;
  var x := m * m;
  var z := m;
  while (z > 0)
  {
    z := z - 1;
    y := y + x;
  }
}
```

y	x	M	z	
0	9	3	3	init
9	9	3	2	after iter 1
18	9	3	1	after iter 2
27	9	3	0	after 3rd iter

$(M-z) * x == y$
 $0 * 9 == 0$
 $1 * 9 == 9$
 $2 * 9 == 18$
 $3 * 9 == 27$

$(M-z) * x == y$

Fill in the annotations in the following program to show that the Hoare triple given by the outermost pre- and post-conditions is valid. Be completely precise and pedantic in the way you apply Hoare rules write assertions in *exactly* the form given by the rules rather than just logically equivalent ones. The provided blanks have been constructed so that if you work backwards from the end of the program you should only need to use the rule of consequence in the places indicated with $\rightarrow$. These implication steps can (silently) rely on all the usual rules of arithmetic.

```
{ { m > 0 } }  $\rightarrow$ 
{ { 0 == (M-M) * (M * M) } }
  y := 0;
{ { y == (M-M) * (M * M) } }
  var x := m * m;
{ { y == (M-M) * x } }
  var z := m;
{ { y == (M-z) * x } }
  while (z > 0) {
    { { z > 0 } }  $\&\&$  y == (M-z) * x
    { { (M-(z-1)) * x == (y+x) } }  $\&\&$  (z-1) > 0
    z := z - 1;
    { { (M-z) * x == (y+x) } }  $\&\&$  z > 0
    y := y + x;
    { { (M-z) * x == y } }  $\&\&$  z > 0
  }
  { { z > 0 } }  $\&\&$  (z > 0)  $\&\&$  (M-z) * x == y
  add
  z > 0 ?
```

invariants;

$z > 0$

$(M-z) * x == y$

Ceren Mert

Question 5 (20 points)

Implement the following Haskell functions:

- (a) Implement a function `weave` that given two lists with elements of the same type, returns a list with elements alternating between the two lists. For example:

`weave [1,2,3] [4,5,6] = [1,4,2,5,3,6]`

You can assume that the lists have the same length.

```
weave :: [a] -> [a] -> [a]
weave [] [] = []
weave _ [] = []
weave (x:xs) (y:ys) = [x,y] ++ weave xs ys
```

- (b) Implement a function `powerSet`, that given a list of elements of some type, computes the list of all of its sublists, including the empty list and the original list itself, in any order. For example:

`powerSet [1,2,3] = [[], [1,2,3], [1,2], [1,3], [1], [2,3], [2], [3], []]`

element + everything

`powerSet :: [a] -> [[a]]`

`powerSet [] = [[]]`

`powerSet (x:xs) =`

`powerSet xs ++ map (\lst -> x : lst) (powerSet xs)`

note this after line 5
`-- powerSet [1,2] = [[], [1], [2], [1,2]]`

`powerSet 1 : [2] = powerSet [2] ++ (map (\lst -> 1 : lst) powerSet [2])`

`[[], [2]] ++ ([1], [1,2])`

`[[], [2]]`

`[1], [1,2]`

`[[], [2]]`

`[1], [1,2]`

`[[], [2]] ++ [1], [1,2]`

`[[], [2]]`

CMSC 433: Midterm Exam (Spring 2024)

Question 1 (20 points)

Consider the two standard folds in Haskell, `foldl` and `foldr`, whose definitions are given below:

```
foldr :: (a -> b -> b) -> b -> [a] -> b
foldr f b [] = b
foldr f b (x:xs) = f x (foldr f b xs)

foldl :: (b -> a -> b) -> b -> [a] -> b
foldl f b [] = b
foldl f b (x:xs) = foldl f (f b x) xs
```

Now consider the following two very similar but distinct folds over lists of integers:

```
f1 :: [Int] -> Int
f1 = foldr (-) 0

f2 :: [Int] -> Int
f2 = foldl (-) 0
```

Write two Daffy programs, one corresponding to each of `f1` and `f2`, but operating on arrays rather than lists. We've given you the method signatures to get you started:

<pre>method f1(a:array<int>) returns (out:int) { var i := a.length - 1; var out := 0; while i >= 0 { out := out - a[i]; } }</pre>	<pre>method f2(a:array<int>) returns (out:int) { var out := 0; var i := 0; while i < a.length { out := out - a[i]; } }</pre>
--	---

Question 2 (20 points)

For each of the Haskell expressions below, write their (most general) Haskell type or "ill-typed" if it contains a type error. The type signatures of all functions below are provided in the appendix at the end.

(a) $\backslash x \rightarrow x$

Example answer:

$a \rightarrow a$

(b) $\backslash x\ y \rightarrow (x,y)$

$a \rightarrow b \rightarrow (a,b)$

(c) $\backslash x\ y \rightarrow$ if $x == y$ then show x else show (x,y)

ill-typed

(d) $\backslash x\ l \rightarrow x : l ++ l ++ [x]$

$a \rightarrow [a] \rightarrow [a]$

(e) `foldr (const 1)`

ill-typed

(f) `( "42"`

$(a, \text{String } b)$

(g) `(.) "42"`

ill-typed

(h) `reverse . reverse`

$a \rightarrow a$

(i) $\backslash l \rightarrow$ filter $(< 42) (l ++ l)$

$\text{Num } a \Rightarrow [a] \rightarrow [a]$

(j) `let f x = x in (f 'a', f True)`

ill-typed

(k) `fmap (fmap (+1)) [Nothing]`

ill-typed

Question 3 (20 points)

For each of the types below, write a Haskell expression that has that type. Don't write trivial expressions (such as [], Nothing, or undefined) unless there is no other option. You can use any function from the appendix, do syntax, list comprehensions, or any valid Haskell.

(a) `Int -> Int`

Example answers:

`\x -> x + 1`

`(+1)`

(b) `Bool -> [Bool]`

`\x -> [x == False]`

(c) `a -> Maybe b`

`\a -> undefined`

(d) `(Int -> Char -> Bool) -> [Int] -> [Char] -> [Bool]`

`\f a b -> [let h = head a in let h2 = head b in [True || (f h1 h2)]]`

(e) `(a -> b -> c) -> Maybe a -> Maybe b -> Maybe c`

`\f a b -> Just (f a b)`

(f) `(a -> b) -> (b -> Bool) -> [a] -> [b]`

`\f g l -> let res = f (head l) in if g(res) then [res] else [res]`

(g) `Maybe a -> (a -> [b]) -> [(a,b)]`

`\a f -> case a of [Nothing] = [] | (Just x) = [ (x, head (f x)) ]`

(h) `Eq a => a -> [a] -> [a]`

`\a -> if head a == last a then a else []`

(i) `Show a => [a] -> IO String`

(j) `(a,b) -> (a -> b -> c) -> c`

`\(a,b) f -> f a b`

(k) `((a,b) -> c) -> c`

Question 4 (20 points)

Consider the following Dafny program that inefficiently calculates the cube of a number:

```
method Cube(m:int) returns (y:int)
  requires m > 0
  ensures y == m*m*m
{
  y := 0;
  var x := m * m;
  var z := m;
  while (z > 0)
  {
    z := z - 1;
    y := y + x;
  }
}
```

Handwritten annotations for the program:

- Initial state: $m = 5, x = 25, z = 5$
- Loop invariant: $y = m * z$
- Loop body annotations:
 - Before $z := z - 1$: $x = m * z$
 - Before $y := y + x$: $y + x = m * m$

Fill in the annotations in the following program to show that the Hoare triple given by the outermost pre- and post- conditions is valid. Be completely precise and pedantic in the way you apply Hoare rules—write assertions in *exactly* the form given by the rules rather than just logically equivalent ones. The provided blanks have been constructed so that if you work backwards from the end of the program you should only need to use the rule of consequence in the places indicated with $\rightarrow>$. These implication steps can (silently) rely on all the usual rules of arithmetic.

```
{ { m > 0 } }  $\rightarrow>$ 
{ { m > 0 } }  $\wedge$   $0 = 0$ 
  y := 0;
{ { m > 0 } }  $\wedge$   $y = 0$ 
  var x := m * m;
{ { m > 0 } }  $\wedge$   $x = m * m$   $\wedge$   $y = 0$ 
  var z := m;
{ { m > 0 } }  $\wedge$   $y = 0$   $\wedge$   $x = m * m$   $\wedge$   $z = m$   $\wedge$   $y = x - z * z$ 
  while (z > 0) {
    { { m > 0 } }  $\wedge$   $z > 0$ 
    { { y + x == x - (z-1) * (z-1) } }  $\wedge$ 
      z := z - 1;
    { { y + x == x - z * z } }
    y := y + x;
    { { y == x - z * z } }
  }
  { { y == x - z * z } }  $\wedge$   $!(z > 0)$   $\wedge$   $x = m * m$   $\wedge$   $m > 0$ 
  { { y = m * m * m } }  $\rightarrow>$ 
```

$$m(m * m)$$

$$x(z) -$$

Question 5 (20 points)

Implement the following Haskell functions:

- (a) Implement a function `weave` that given two lists with elements of the same type, returns a list with elements alternating between the two lists. For example:

```
weave [1,2,3] [4,5,6] = [1,4,2,5,3,6]
```

You can assume that the lists have the same length.

```
weave [] = []
```

```
weave (h1:t1) (h2:t2) = h1:h2:(weave t1 t2)
```

- (b) Implement a function `powerset`, that given a list of elements of some type, computes the list of all of its sublists, including the empty list and the original list itself, in any order. For example:

```
powerset [1,2,3] = [[1,2,3], [1,2], [1,3], [1], [2,3], [2], [3], []]
```

```
powerset [] = [its <del>[]</del> []]
```


Jacob Mangin
117949124

CMSC 433: Midterm Exam (Spring 2024)

Question 1 (20 points)

Consider the two standard folds in Haskell, `foldl` and `foldr`, whose definitions are given below:

```
foldr :: (a -> b -> b) -> b -> [a] -> b
foldr f b [] = b
foldr f b (x:xs) = f x (foldr f b xs)

foldl :: (b -> a -> b) -> b -> [a] -> b
foldl f b [] = b
foldl f b (x:xs) = foldl f (f b x) xs
```

Now consider the following two very similar but distinct folds over lists of integers:

```
f1 :: [Int] -> Int
f1 = foldr (-) 0

f2 :: [Int] -> Int
f2 = foldl (-) 0
```

Write two Dafny programs, one corresponding to each of `f1` and `f2`, but operating on arrays rather than lists. We've given you the method signatures to get you started:

<pre>method f1(a:array<int>) returns (out:int) { var i:int := a.length-1; out := 0; while (i >= 0) { out := a[i] - out; i := i-1; } }</pre>	<pre>method f2(a:array<int>) returns (out:int) { var i:int := 1; out := a[0]; while (i < a.length) { out := out - a[i]; i := i+1; } }</pre>
--	--

Question 2 (20 points)

For each of the Haskell expressions below, write their (most general) Haskell type or "ill-typed" if it contains a type error. The type signatures of all functions below are provided in the appendix at the end.

(a) $\backslash x \rightarrow x$

Example answer:

$a \rightarrow a$

(b) $\backslash x y \rightarrow (x, y)$

$a \rightarrow b \rightarrow (a, b)$

(c) $\backslash x y \rightarrow \text{if } x == y \text{ then show } x \text{ else show } (x, y)$

$a \rightarrow a \rightarrow \text{String}$

(d) $\backslash x l \rightarrow x : l ++ l ++ [x]$

$b \rightarrow [c] \rightarrow [b]$

(e) foldr (const 1)

$(a \rightarrow b \rightarrow a) \rightarrow (a \rightarrow b \rightarrow b)$

(f) $(\cdot 42)$

$[\text{String}]$

(g) $(\cdot) "42"$

$\text{String} \rightarrow \text{Char} \rightarrow \text{Char} \rightarrow (\text{Char}, \text{Char})$

(h) $\text{reverse} \cdot \text{reverse}$

$[a] \rightarrow [a] \rightarrow (b \rightarrow c) \rightarrow (a \rightarrow b) \rightarrow a \rightarrow c \rightarrow ([a] \rightarrow [a]) \rightarrow [a]$

(i) $\backslash l \rightarrow \text{filter } (< 42) (l ++ l)$

$a \rightarrow (a \rightarrow b) \rightarrow [a] \rightarrow [a]$

(j) $\text{let } f \ x = x \text{ in } (f 'a', f \text{ True})$

$(a \rightarrow b) \rightarrow b \rightarrow \text{Bool}$

(k) $\text{fmap (fmap (+1)) [Nothing]}$

ill-typed

Question 3 (20 points)

For each of the types below, write a Haskell expression that has that type. Don't write trivial expressions (such as [], Nothing, or undefined) unless there is no other option. You can use any function from the appendix, do syntax, list comprehensions, or any valid Haskell.

(a) `Int -> Int`

Example answers:

`\x -> x + 1`
`(+1)`

(b) `Bool -> [Bool]`

`\true -> [true]`

(c) `a -> Maybe b`

`undefined`

(d) `(Int -> Char -> Bool) -> [Int] -> [Char] -> [Bool]`

`\zipWith (==) [1,2,3] ['a','b','c']`

(e) `(a -> b -> c) -> Maybe a -> Maybe b -> Maybe c`

`\f Just a Just b -> f a b`

(f) `(a -> b) -> (b -> Bool) -> [a] -> [b]`

`\f1 f2 [a] -> if f2(f1 a) then [f1 a] else`

(g) `Maybe a -> (a -> [b]) -> [(a,b)]`

`\Just a -> maybeToList a`

(h) `Eq a => a -> [a] -> [a]`

`\delete a [a]`

(i) `Show a => [a] -> IO String`

`\[a] -> putStrLn a`

(j) `(a,b) -> (a -> b -> c) -> c`

`\(a,b) f -> f a b`

(k) `((a,b) -> c) -> c`

`\f a b -> f () a b`

Question 4 (20 points)

Consider the following Dafny program that inefficiently calculates the cube of a number:

```

method Cube(m:int) returns (y:int)
  requires m > 0
  ensures y == m*m*m
  {
    y := 0;
    var x := m * m;
    var z := m;
    while (z > 0)
    {
      z := z - 1;
      y := y + x;
    }
  }

```

$$\begin{array}{r} 2 \\ 3 \overline{) 2} \\ \underline{2} \\ 0 \end{array}$$

Fill in the annotations in the following program to show that the Hoare triple given by the outermost pre- and post-conditions is valid. Be completely precise and pedantic in the way you apply Hoare rules—write assertions in *exactly* the form given by the rules rather than just logically equivalent ones. The provided blanks have been constructed so that if you work backwards from the end of the program you should only need to use the rule of consequence in the places indicated with $\Rightarrow$. These implication steps can (silently) rely on all the usual rules of arithmetic.

```

{ { m > 0 } } -->
{ { 0 = 0 }
  y := 0;
  { { y = 0 }
    var x := m * m;
    { { y = 0 } x = m * m
      var z := m;
      { { y = 0 } while (z > 0) {
        { { z > 0 } y < x * m
          { { z > 0 } z - 1 = m - (m - z)
            z := z - 1;
            { { z > 0 } m - z }
              y := y + x;
              { { z = m - (m - z) }
                { { z > 0 } y + x < x * m
                  }
                { { z > 0 } y = m * m * m }
              }
            }
          }
        }
      }
    }
  }
}
-->

```


Question 5 (20 points)

Implement the following Haskell functions:

- (a) Implement a function `weave` that given two lists with elements of the same type, returns a list with elements alternating between the two lists. For example:

`weave [1,2,3] [4,5,6] = [1,4,2,5,3,6]`

You can assume that the lists have the same length.

`weave [] [] = []`

`weave xs [] = xs`

`weave [] ys = ys`

`weave (x:xs) (y:ys) = x:y:weave xs ys`

- (b) Implement a function `powerset`, that given a list of elements of some type, computes the list of all of its sublists, including the empty list and the original list itself, in any order. For example:

`powerset [1,2,3] = [[], [1], [2], [3], [1,2], [1,3], [2,3], [1,2,3]]`

`powerset [] = [[]]`

`powerset [a] = [[a], []]`

`powerset (x:xs) = auxFunc x : powerset xs`

`auxFunc x [] = [x]`

`auxFunc x (xs:xs') = [x]:[xs]:xs' : [auxFunc x xs']`

Brian Rapelowski
117095005

CMSC 433: Midterm Exam (Spring 2024)

Question 1 (20 points)

Consider the two standard folds in Haskell, `foldl` and `foldr`, whose definitions are given below:

```
foldr :: (a -> b -> b) -> b -> [a] -> b
foldr f b [] = b
foldr f b (x:xs) = f x (foldr f b xs)

foldl :: (b -> a -> b) -> b -> [a] -> b
foldl f b [] = b
foldl f b (x:xs) = foldl f (f b x) xs
```

Now consider the following two very similar but distinct folds over lists of integers:

```
f1 :: [Int] -> Int
f1 = foldr (-) 0

f2 :: [Int] -> Int
f2 = foldl (-) 0
```

Write two Dafny programs, one corresponding to each of `f1` and `f2`, but operating on arrays rather than lists. We've given you the method signatures to get you started:

<pre>method f1(a:array<int>) returns (out:int) { out := 0; var i : int := 0; while (i < a.length) { out := a[a.length-i-1] - out; i := i+1; } }</pre>	<pre>method f2(a:array<int>) returns (out:int) { out := 0; var i : int := 0; while (i < a.length) { out := out - a[i]; i := i+1; } }</pre>
--	---

Question 2 (20 points)

For each of the Haskell expressions below, write their (most general) Haskell type or "ill-typed" if it contains a type error. The type signatures of all functions below are provided in the appendix at the end.

(a) $\backslash x \rightarrow x$

Example answer:

$a \rightarrow a$

(b) $\backslash x y \rightarrow (x, y)$

$a \rightarrow b \rightarrow (a, b)$

(c) $\backslash x y \rightarrow \text{if } x == y \text{ then show } x \text{ else show } (x, y)$

ill-typed

(d) $\backslash x l \rightarrow x : l ++ l ++ [x]$

$a \rightarrow [a] \rightarrow [a]$

(e) $\text{foldr} (\text{const } 1)$

$[a] \rightarrow [a]$

(f) ("42")

~~ill-typed~~ (Null, String)

(g) ("42")

ill-typed

(h) $\text{reverse} . \text{reverse}$

$[a] \rightarrow [a]$

(i) $\backslash l \rightarrow \text{filter } (< 42) (l ++ l)$

$[Int] \rightarrow [Int]$

(j) $\text{let } f x = x \text{ in } (f 'a', f \text{True})$

ill-typed

(k) $\text{fmap } (\text{fmap } (+1)) [\text{Nothing}]$

ill-typed

Question 3 (20 points)

For each of the types below, write a Haskell expression that has that type. Don't write trivial expressions (such as `[]`, `Nothing`, or `undefined`) unless there is no other option. You can use any function from the `appendix`, do syntax, list comprehensions, or any valid Haskell.

(a) `Int -> Int`

Example answers:

`\x -> x + 1`
`(+1)`

(b) `Bool -> [Bool]`

`\x -> if x > 0 then [x] else [x]`

(c) `a -> Maybe b`

`undefined`

(d) `(Int -> Char -> Bool) -> [Int] -> [Char] -> [Bool]`

`\ f (x:x:) (y:y) -> (f x y) : []`

(e) `(a -> b -> c) -> Maybe a -> Maybe b -> Maybe c`

`\ f (Just x) (Just y) -> Just (f x y)`

(f) `(a -> b) -> (b -> Bool) -> [a] -> [b]`

`\ f g lst -> map f lst`

(g) `Maybe a -> (a -> [b]) -> [(a,b)]`

`\ (Just x) f -> [(x, head (f x))]`

(h) `Eq a => a -> [a] -> [a]`

`\ x lst -> map (\y -> if y == x then x else x)`

(i) `Show a => [a] -> IO String`

`\ x -> print "Hello"`

(j) `(a,b) -> (a -> b -> c) -> c`

`\ (x,y) f -> f x y,`

(k) `((a,b) -> c) -> c`

`undefined`

$$P = z >= 0 \ \&\& \ y = x \cdot (m - z)$$

Question 4 (20 points)

Consider the following Dafny program that inefficiently calculates the cube of a number:

```
method Cube(m:int) returns (y:int)
  requires m > 0
  ensures y == m*m*m
{
  y := 0;
  var x := m * m;
  var z := m;
  while (z > 0)
  {
    z := z - 1;
    y := y + x;
  }
}
```

$$\begin{matrix} m = 2 \\ x = 4 \\ z = 2 \ 1 \ 0 \\ y = 0 \ 4 \ 8 \end{matrix}$$

$$y = -$$

$$m = 4$$

$$x = 16$$

$$z = 4 \ 3 \ 2 \ 1 \ 0$$

$$y = 0 \ 16 \ 32 \ 48 \ 64$$

$$y = x \cdot (m - z)$$

$$0 = 16 \cdot (4 - 4) = 0$$

$$32 = 16 \cdot (4 - 2) =$$

Fill in the annotations in the following program to show that the Hoare triple given by the outermost pre- and post-conditions is valid. Be completely precise and pedantic in the way you apply Hoare rules—write assertions in *exactly* the form given by the rules rather than just logically equivalent ones. The provided blanks have been constructed so that if you work backwards from the end of the program you should only need to use the rule of consequence in the places indicated with $\rightarrow>$. These implication steps can (silently) rely on all the usual rules of arithmetic.

```
{ { m > 0 } }  $\rightarrow>$ 
{ { 0 = (m * m) * (z - z) } }  $\&\&$  m >= 0
  y := 0;
  { { y = (m * m) * (z - z) } }  $\&\&$  m >= 0
    var x := m * m;
    { { y = x * (z - z) } }  $\&\&$  m >= 0
      var z := m;
      { { y = x * (m - z) } }  $\&\&$  z >= 0
        while (z > 0) {
          { { y = x * (m - z) } }  $\&\&$  z >= 0  $\&\&$  z > 0
            { { (y + x) * (m - (z - 1)) } }  $\&\&$  z - 1 >= 0
              z := z - 1;
              { { (y + x) * (m - z) } }  $\&\&$  z >= 0
                y := y + x;
                { { y = x * (m - z) } }  $\&\&$  z >= 0
              }
            { { y = x * (m - z) } }  $\&\&$  z >= 0  $\&\&$  z > 0
          }
        }
      { { y = m * m * m } }  $\rightarrow>$ 
```

$$m > 0$$

$$z >= 0$$

Question 5 (20 points)

Implement the following Haskell functions:

- (a) Implement a function `weave` that given two lists with elements of the same type, returns a list with elements alternating between the two lists. For example:

`weave [1,2,3] [4,5,6] = [1,4,2,5,3,6]`

You can assume that the lists have the same length.

`weave :: [a] -> [a] -> [a]`

`weave [] [] = []`

`weave (x:xs) (y:ys) = (x:y):(weave xs ys)`

- (b) Implement a function `powerset`, that given a list of elements of some type, computes the list of all of its sublists, including the empty list and the original list itself, in any order. For example:

`powerset [1,2,3] = [[1,2,3], [1,2], [1,3], [1], [2,3], [2], [3], []]`

`powerset :: [a] -> [[a]]`

`powerset [] = []`

`powerset (x:xs) = [He|per (x:xs), ++ (powerset xs)]`

`He|per :: [a] -> [[a]]`
`He|per [] = []`
`He|per (x:ys) = [He|per [x]]`
`He|per (x:ys) = [He|per (x:ys)]`

Anders
Spear

118002224

CMSC 433: Midterm Exam (Spring 2024)

aspr@umt.edu

Question 1 (20 points)

Consider the two standard folds in Haskell, `foldl` and `foldr`, whose definitions are given below:

```
foldr :: (a -> b -> b) -> b -> [a] -> b      [1,2,3]
foldr f b [] = b                               f 1 (+2 (+3 0))
foldr f b (x:xs) = f x (foldr f b xs)

foldl :: (b -> a -> b) -> b -> [a] -> b
foldl f b [] = b                               f 0 (+1) 2) 3)
foldl f b (x:xs) = foldl f (f b x) xs          f 0 1) 2) 3)
```

Now consider the following two very similar but distinct folds over lists of integers:

```
f1 :: [Int] -> Int
f1 = foldr (-) 0

f2 :: [Int] -> Int
f2 = foldl (-) 0
```

Write two Dafny programs, one corresponding to each of `f1` and `f2`, but operating on arrays rather than lists. We've given you the method signatures to get you started:

<pre>method f1(a:array<int>) returns (out:int) { out = a[0]; var i = 1; while (i < a.length) { out = out - a[i]; } }</pre>	<pre>method f2(a:array<int>) returns (out:int) { out = 0; var i = 0; while (i < a.length) { out = out - a[i]; i = i + 1; } }</pre>
---	---

Question 2 (20 points)

For each of the Haskell expressions below, write their (most general) Haskell type or "ill-typed" if it contains a type error. The type signatures of all functions below are provided in the appendix at the end.

(a) $\backslash x \rightarrow x$

Example answer:

$a \rightarrow a$

(b) $\backslash x y \rightarrow (x,y)$

$a \rightarrow b \rightarrow (a,b)$

(c) $\backslash x y \rightarrow \text{if } x == y \text{ then show } x \text{ else show } (x,y)$

$a \rightarrow a \rightarrow \text{string}$

(d) $\backslash x l \rightarrow x : l ++ l ++ [x]$

$a \rightarrow [a] \rightarrow [a]$

(e) foldr (const 1)

~~$a \rightarrow b \rightarrow a$~~

$a \rightarrow b \rightarrow a$

(f) ("42")

$a \rightarrow (a, \text{string})$

(g) ("42")

$a \rightarrow (\text{string}, a)$

(h) $\text{reverse} . \text{reverse}$

$[a] \rightarrow [a]$

(i) $\backslash l \rightarrow \text{filter } (< 42) (l ++ l)$

$[a] \rightarrow [a]$

(j) $\text{let } f x = x \text{ in } (f 'a', f \text{True})$

$(\text{string}, \text{Bool})$

(k) $\text{fmap (fmap (+1)) [Nothing]}$

$f \text{int} \rightarrow f \text{int}$

$f a \rightarrow f b$

$f a \rightarrow f \text{Num}$

$f \text{Num}$

OC max Yux #

Question 3 (20 points)

For each of the types below, write a Haskell expression that has that type. Don't write trivial expressions (such as `[]`, `Nothing`, or `undefined`) unless there is no other option. You can use any function from the appendix, do syntax, list comprehensions, or any valid Haskell.

(a) `Int -> Int`

Example answers:

`x -> x + 1`
`(+1)`

(b) `Bool -> [Bool]`
`\x -> [x]`

(c) `a -> Maybe b`
`\x -> Nothing`

(d) `(Int -> Char -> Bool) -> [Int] -> [Char] -> [Bool]`
~~`f1 f2 (a:ns) -> f2 (f1 a) true [f1 a] else []`~~
`\(f) (a -> b) -> (b -> Bool) -> [a] -> [b]`
`\f (a -> b) -> c -> Maybe a -> Maybe b -> Maybe c`
`\f (Just a) (Just b) -> Just (f a b)`
`\f -> Nothing`
`\f1 f2 (a:ns) -> f2 (f1 a) true [f1 a] else []`

(e) `(a -> b -> c) -> Maybe a -> Maybe b -> Maybe c`
`\(Just a) f -> [a]`

(f) `Eq a => a -> [a] -> [a]`
`a == b: bs = True if a == b else False`

(g) `Maybe a -> (a -> [b]) -> [[a,b]]`

(h) `Eq a => a -> [a] -> [a]`
`a == b: bs = True if a == b else False`

(i) `Show a => [a] -> IO String`
`\a -> map show a`

(j) `(a,b) -> (a -> b -> c) -> c`
`\(a,b) f -> f a b`

(k) `((a,b) -> c) -> c`
`\f -> f (undefined, undefined)`
`\f -> f (a,b)` ← is it legal to have uninitialized vars?

Question 4 (20 points)

Consider the following Dafny program that inefficiently calculates the cube of a number:

```
method Cube(m:int) returns (y:int)
  requires m > 0
  ensures y == m*m*m
{
  y := 0;
  var x := m * m;
  var z := m;
  while (z > 0)
  {
    z := z - 1;
    y := y + x;
  }
}
```

by $z \leq m$

Fill in the annotations in the following program to show that the Hoare triple given by the outermost pre- and post- conditions is valid. Be completely precise and pedantic in the way you apply Hoare rules- write assertions in *exactly* the form given by the rules rather than just logically equivalent ones. The provided blanks have been constructed so that if you work backwards from the end of the program you should only need to use the rule of consequence in the places indicated with $\rightarrow>$. These implication steps can (silently) rely on all the usual rules of arithmetic.

```
{ { m > 0 } }  $\rightarrow>$ 
{ {  $m \leq m$  } }
  y := 0;
  { {  $m \leq m$  } }
  var x := m * m;
  { {  $m \leq m$  } }
  var z := m;
  { {  $z \leq m$  } }
  while (z > 0) {
    { {  $z \leq m$  } }
    { {  $(z-1) \leq m$  } }
    z := z - 1;
    { {  $z \leq m$  } }
    y := y + x;
  }
  { {  $z \leq m$  } }
  { {  $y = m * m * m$  } }  $\rightarrow>$ 
```

Handwritten annotations:

- Before `var x := m * m;`: $y \leq m$ and $0 \leq m(m-m)$
- Before `var z := m;`: $y \leq m$ and $y \leq m \cdot (m-m)$
- Before `while (z > 0) {`: $x = m(m-2)$
- Inside loop, before `z := z - 1;`: $y \leq m$ and $y \leq m(m-2)$
- Inside loop, before `y := y + x;`: $y \leq m$ and $y \leq m(m-2)$
- After loop: $y \leq m$ and $y = m(m-2)$
- Before final `{ { y = m * m * m } }`: $z \leq m$ and $y \leq m(m-2)$ and $0 \leq (z-1)$

Question 5 (20 points)

Implement the following Haskell functions:

- (a) Implement a function `weave` that given two lists with elements of the same type, returns a list with elements alternating between the two lists. For example:

`weave [1,2,3] [4,5,6] = [1,4,2,5,3,6]`

You can assume that the lists have the same length.

~~def~~ `function weave :: [a] -> [a] -> [a]`

`function weave [] [] = []`

`function`

`weave x:xs b = x:(weave xs b)`

`function weave2 x b:bs = b:(weave x bs)`

- (b) Implement a function `powerSet`, that given a list of elements of some type, computes the list of all of its sublists, including the empty list and the original list itself, in any order. For example:

`powerSet [1,2,3] = [[1,2,3], [1,2], [1,3], [1], [2,3], [2], [3], []]`

`function powerSet :: [a] -> [[a]]`

`function powerSet [] = [[]]`

`function powerSet h:[] = [[]h, []] < not satisfactory`

`< not satisfactory`

`function powerSet h:t = concat([map (\x -> [h,x]) t) (powerSet t)`

Michelle Han

CMSC 433: Midterm Exam (Spring 2024)

Question 1 (20 points)

Consider the two standard folds in Haskell, `foldl` and `foldr`, whose definitions are given below:

```
foldr :: (a -> b -> b) -> b -> [a] -> b
foldr f b [] = b
foldr f b (x:xs) = f x (foldr f b xs)

foldl :: (b -> a -> b) -> b -> [a] -> b
foldl f b [] = b
foldl f b (x:xs) = foldl f (f b x) xs
```

Now consider the following two very similar but distinct folds over lists of integers:

```
f1 :: [Int] -> Int
f1 = foldr (-) 0

f2 :: [Int] -> Int
f2 = foldl (-) 0
```

Write two Dafny programs, one corresponding to each of `f1` and `f2`, but operating on arrays rather than lists. We've given you the method signatures to get you started:

<pre>method f1(a:array<int>) returns (out:int) { var i := a.Length - 1; out := 0; while i >= 0 { out := a[i] - out; i := i - 1; }</pre>	<pre>method f2(a:array<int>) returns (out:int) { var i := 0; out := 0; while i < a.Length { out := out - a[i]; i := i + 1; }</pre>
--	---

Question 2 (20 points)

For each of the Haskell expressions below, write their (most general) Haskell type or "ill-typed" if it contains a type error. The type signatures of all functions below are provided in the appendix at the end.

(a) $\backslash x \rightarrow x$

Example answer:

$a \rightarrow a$

(b) $\backslash x\ y \rightarrow (x,y)$

$a \rightarrow b \rightarrow (a,b)$

(c) $\backslash x\ y \rightarrow \text{if } x == y \text{ then show } x \text{ else show } (x,y)$

$\text{Show } a \Rightarrow a \rightarrow a \rightarrow \text{String}$

(d) $\backslash x\ l \rightarrow x : l ++ l ++ [x]$

$a \rightarrow [a] \rightarrow [a]$

(e) $\text{foldr } (\text{const } 1)$

$(a \rightarrow b \rightarrow b) \rightarrow b \rightarrow b$

(f) $(\cdot "42")$

$a \rightarrow b \rightarrow \text{String} \rightarrow (a, b, \text{String})$

(g) $(\cdot "42")$

$\text{String} \rightarrow a \rightarrow (\text{String}, a)$

(h) $\text{reverse} . \text{reverse}$

$([a] \rightarrow [a]) \rightarrow ([a], [a]) \rightarrow [a] \rightarrow [a]$

(i) $\backslash l \rightarrow \text{filter } (< 42) (l ++ l)$

$(\text{Int} \rightarrow \text{Bool}) \rightarrow [\text{Int}] \rightarrow [\text{Int}]$

(j) $\text{let } f\ x = x \text{ in } (f 'a', f \text{True})$

$(a \rightarrow b) \rightarrow b \rightarrow (\text{Char}, \text{Boolean})$

(k) $\text{fmap } (\text{fmap } (+1)) [\text{Nothing}]$

$((\text{Int} \rightarrow \text{Int}) \rightarrow [\text{Maybe Int}]) \rightarrow [\text{Maybe Int}] \rightarrow [[\text{Maybe Int}]]$

Question 3 (20 points)

For each of the types below, write a Haskell expression that has that type. Don't write trivial expressions (such as [], Nothing, or undefined) unless there is no other option. You can use any function from the appendix; do syntax, list comprehensions, or any valid Haskell.

(a) `Int -> Int`

Example answers:

`\x -> x + 1`
`(+1)`

(b) `Bool -> [Bool]`

`\x -> [not x]`

(c) `a -> Maybe b`

`foo = Nothing`

(d) `(Int -> Char -> Bool) -> [Int] -> [Char] -> [Bool]`

`foo = zipWith (\x y -> x == 1 && y == 'a') [1,2,3] ['a']`

(e) `(a -> b -> c) -> Maybe a -> Maybe b -> Maybe c`

(f) `(a -> b) -> (b -> Bool) -> [a] -> [b]`

`\x y -> const x . not y`

(g) `Maybe a -> (a -> [b]) -> [(a,b)]`

(h) `Eq a => a -> [a] -> [a]`

`delete`

(i) `Show a => [a] -> IO String`

(j) `(a,b) -> (a -> b -> c) -> c`

`\(x,y) -> f x y`

(k) `((a,b) -> c) -> c`

`foo f (x,y) = f (x,y)`

Question 4 (20 points)

Consider the following Dafny program that inefficiently calculates the cube of a number:

```

method Cube(m:int) returns (y:int)
  requires m > 0
  ensures y == m*m*m
  {
    y := 0;
    var x := m * m;
    var z := m;
    while (z > 0)
    {
      z := z - 1;
      y := y + x;
    }
  }

```

Fill in the annotations in the following program to show that the Hoare triple given by the outermost pre- and post-conditions is valid. Be completely precise and pedantic in the way you apply Hoare rules: write assertions in *exactly* the form given by the rules rather than just logically equivalent ones. The provided blanks have been constructed so that if you work backwards from the end of the program you should only need to use the rule of consequence in the places indicated with $\rightarrow$. These implication steps can (silently) rely on all the usual rules of arithmetic.

[illegible]

Question 5 (20 points)

Implement the following Haskell functions:

- (a) Implement a function `weave` that given two lists with elements of the same type, returns a list with elements alternating between the two lists. For example:

`weave [1,2,3] [4,5,6] = [1,4,2,5,3,6]`

You can assume that the lists have the same length.

```
weave [] = []  
weave _ [] = []  
weave = zipWith (:)
```

- (b) Implement a function `powerset`, that given a list of elements of some type, computes the list of all of its sublists, including the empty list and the original list itself, in any order. For example:

`powerset [1,2,3] = [[], [1], [2], [1,3], [2,3], [3], [1,2,3]]`

```
powerset [] = [[]]
```

```
powerset [x] = [[x], []]
```

```
powerset (x:xs) = [x] ++ powerset xs) : []
```


Aravon Abcu
117253801

CMSC 433: Midterm Exam (Spring 2024)

Question 1 (20 points)

Consider the two standard folds in Haskell, `foldl` and `foldr`, whose definitions are given below:

```
foldr :: (a -> b -> b) -> b -> [a] -> b  (-) ((-) 0 1) 2) 3  [1, 2, 3]
foldr f b [] = b
foldr f b (x:xs) = f x (foldr f b xs)

foldl :: (b -> a -> b) -> b -> [a] -> b
foldl f b [] = b
foldl f b (x:xs) = foldl f (f b x) xs
```

Now consider the following two very similar but distinct folds over lists of integers:

```
f1 :: [Int] -> Int  [1, 2, 3]  1 - (2 - (3 - 0)) = 2
f1 = foldr (-) 0

f2 :: [Int] -> Int  [1, 2, 3]  3 - (2 - (1 - 0)) = 2
f2 = foldl (-) 0
```

Write two Dafny programs, one corresponding to each of `f1` and `f2`, but operating on arrays rather than lists. We've given you the method signatures to get you started:

```
method f1(a:array<int>) returns (out:int) {
  var diff := 0;
  var i := a.length-1;
  while i > 0 {
    diff := a[i] - diff;
    i := i-1;
  }
  out := diff;
}

method f2(a:array<int>) returns (out:int) {
  var diff := 0;
  var i := 0;
  while i < a.length {
    diff := diff - a[i];
    i := i+1;
  }
  out := diff;
}
```

Question 2 (20 points)

For each of the Haskell expressions below, write their (most general) Haskell type or "ill-typed" if it contains a type error. The type signatures of all functions below are provided in the appendix at the end.

(a) $\backslash x \rightarrow x$

Example answer:

$a \rightarrow a$

(b) $\backslash x y \rightarrow (x, y)$

$a \rightarrow b \rightarrow (a, b)$

(c) $\backslash x y \rightarrow \text{if } x == y \text{ then show } x \text{ else show } (x, y)$

$\text{Eq } a \Rightarrow a \rightarrow b \rightarrow \text{IO String}$

★ (d) $\backslash x l \rightarrow x : l ++ l ++ l ++ [x]$

$a \rightarrow [a] \rightarrow [a]$

(e) $\text{foldr } (\text{const } 1)$

ill-typed, const takes two arguments

★ (f) ("42")

ill-typed

(g) $(.) \text{"42"}$

$b \rightarrow ([\text{Char}], b)$

(h) $\text{reverse} . \text{reverse}$

$[a] \rightarrow [a] \rightarrow [a]$

(i) $\backslash l \rightarrow \text{filter } (< 42) (l ++ l)$ duplicate list

or $[Int] \rightarrow [Int]$

★ (j) $\text{let } f x = x \text{ in } (f 'a', f \text{True})$

$(\text{Char}, \text{Bool})$

★ (k) $\text{fmap } (\text{fmap } (+1)) [\text{Nothing}]$

$[\text{map } (+1)] \text{ (int} \rightarrow \text{int)}$

Nothing

Question 3 (20 points)

For each of the types below, write a Haskell expression that has that type. Don't write trivial expressions (such as [], Nothing, or undefined) unless there is no other option. You can use any function from the appendix, do syntax, list comprehensions, or any valid Haskell.

(a) `Int -> Int`

Example answers:

`\x -> x + 1`
`(+1)`

(b) `Bool -> [Bool]`

`repeat True`

(c) `a -> Maybe b`

`\x -> Nothing`
(d) `(Int -> Char -> Bool) -> [Int] -> [Char] -> [Bool]`

`zipWith (\i c -> if i < 10 && c == 'a') then True else False)`

(e) `(a -> b -> c) -> Maybe a -> Maybe b -> Maybe c`

(f) `(a -> b) -> (b -> Bool) -> [a] -> [b]`

`\f g xs -> filter g (map f xs)`

~~(g)~~ `Maybe a -> (a -> [b]) -> [(a,b)]`

`\x f ->`

(h) `Eq a => a -> [a] -> [a]`

`delete`

(i) `Show a => [a] -> IO String`

`show (++ [])`

(j) `(a,b) -> (a -> b -> c) -> c`

`\(x,y) f -> f x y`

~~(k)~~ `((a,b) -> c) -> c`

`\h ->`

Question 4 (20 points)

Consider the following Dafny program that inefficiently calculates the cube of a number:

```
method Cube(m:int) returns (y:int)
  requires m > 0
  ensures y == m*m*m
{
```

```
  y := 0;
```

```
  var x := m * m;
```

```
  var z := m;
```

```
  while (z > 0)
  {
```

```
    z := z - 1;
```

```
    y := y + x;
```

```
  }
```

```
    y = (m-z) * x
```

```
    z >= 0    x == m^3
```

```
}
```

z	x	y	m
3	4	0	2
2	4	4	2
1	4	8	2
0	4	12	2

Fill in the annotations in the following program to show that the Hoare triple given by the outermost pre- and post- conditions is valid. Be completely precise and pedantic in the way you apply Hoare rules - write assertions in *exactly* the form given by the rules rather than just logically equivalent ones. The provided blanks have been constructed so that if you work backwards from the end of the program you should only need to use the rule of consequence in the places indicated with $\rightarrow$. These implication steps can (silently) rely on all the usual rules of arithmetic.

```
{ { m > 0 } } ->>
{ { 0 < m+m } == (m-(m-1)) * (m*m) } && m-1 >= 0 && m && m == m * m
  y := 0;
  { { y + (m+m) == (m-(m-1)) * (m*m) } && m-1 >= 0 && m && m == m * m }
    var x := m * m;
    { { y+x == (m-(m-1)) * x && m-1 >= 0 && x == m*m } }
      var z := m;
      { { y+x == (m-(z-1)) * x && z-1 >= 0 && x == m*m } }
        while (z > 0) {
          { { (y+x == (m-(z-1)) * x && z-1 >= 0 && x == m*m) && z > 0 } } ->>
            { { y+x == (m-(z-1)) * x && z-1 >= 0 && x == m*m } }
              z := z - 1;
            { { (y+x) == (m-z) * x && z >= 0 && x == m*m } }
              y := y + x;
            { { y == (m-z) * x && z >= 0 && x == m*m } }
          }
        { { (y == (m-z) * x && z >= 0 && x == m*m) && !(z > 0) } } ->>
          { { y = m * m * m } }
```


Question 5 (20 points)

Implement the following Haskell functions:

- (a) Implement a function `weave` that given two lists with elements of the same type, returns a list with elements alternating between the two lists. For example:

```
weave [1,2,3] [4,5,6] = [1,4,2,5,3,6]
```

You can assume that the lists have the same length.

```
weave :: [a] -> [a] -> [a]
weave [] [] = []
weave [a] [] = a
weave [] b = b
weave (x:xs) (y:ys) = x:y:(weave xs ys)
```

- (b) Implement a function `powerset`, that given a list of elements of some type, computes the list of all of its sublists, including the empty list and the original list itself, in any order. For example:

```
powerset [1,2,3] = [[1,2,3], [1,2], [1,3], [1], [2,3], [2], [3], []]
```

```
powerset :: [a] -> [[a]]
powerset [] = []
powerset (x:xs) = foldr (\z acc ->
```


ARJUN N/AHA
120406607

CMSC 433: Midterm Exam (Spring 2024)

Question 1 (20 points)

Consider the two standard folds in Haskell, `foldl` and `foldr`, whose definitions are given below:

```
foldr :: (a -> b -> b) -> b -> [a] -> b
foldr f b [] = b
foldr f b (x:xs) = f x (foldr f b xs)

foldl :: (b -> a -> b) -> b -> [a] -> b
foldl f b [] = b
foldl f b (x:xs) = foldl f (f b x) xs
```

Now consider the following two very similar but distinct folds over lists of integers:

```
f1 :: [Int] -> Int
f1 = foldr (-) 0

f2 :: [Int] -> Int
f2 = foldl (-) 0
```

Write two Dafny programs, one corresponding to each of `f1` and `f2`, but operating on arrays rather than lists. We've given you the method signatures to get you started:

```
method f1(a:array<int>) returns (out:int) {
  var i: int := a.Length - 1;
  out := 0
  while (i >= 0) {
    out := out - a[i];
    i := i - 1;
  }
}
```

```
method f2(a:array<int>) returns (out:int) {
  var i: int := 0;
  out := 0;
  while (i < a.Length - 1) {
    out := out - a[i];
    i := i + 1;
  }
}
```

}

}

Question 2 (20 points)

For each of the Haskell expressions below, write their (most general) Haskell type or "ill-typed" if it contains a type error. The type signatures of all functions below are provided in the appendix at the end.

(a) $\backslash x \rightarrow x$

Example answer:

$a \rightarrow a$

(b) $\backslash x\ y \rightarrow (x,y)$

$a \rightarrow b \rightarrow (a,b)$

(c) $\backslash x\ y \rightarrow \text{if } x == y \text{ then show } x \text{ else show } (x,y)$

$a \rightarrow b \rightarrow \text{}$

(d) $\backslash x\ l \rightarrow x : l ++ l ++ [x]$

$a \rightarrow b \rightarrow [a]$

(e) $\text{foldr } (\text{const } 1)$

$a \rightarrow (a \rightarrow b \rightarrow a) \rightarrow a$

(f) $(\cdot 42^*)$

(a,b)

(g) $(\cdot) "42"$

$a \rightarrow b \rightarrow (a,b)$

(h) $\text{reverse} . \text{reverse}$

$[a] \rightarrow [a]$

(i) $\backslash l \rightarrow \text{filter } (< 42) (l ++ l)$

$a \rightarrow [a]$

(j) $\text{let } f\ x = x \text{ in } (f\ 'a', f\ \text{True})$

(k) $\text{fmap } (\text{fmap } (+1))\ [\text{Nothing}]$

Question 3 (20 points)

For each of the types below, write a Haskell expression that has that type. Don't write trivial expressions (such as [], Nothing, or undefined) unless there is no other option. You can use any function from the appendix, do syntax, list comprehensions, or any valid Haskell.

(a) $\text{Int} \rightarrow \text{Int}$

Example answers:

$\backslash x \rightarrow x + 1$
 $(+1)$

(b) $\text{Bool} \rightarrow [\text{Bool}]$

$\backslash x \rightarrow [] : x$

(c) $a \rightarrow \text{Maybe } b$

$\backslash \text{undefined}$

(d) $(\text{Int} \rightarrow \text{Char} \rightarrow \text{Bool}) \rightarrow [\text{Int}] \rightarrow [\text{Char}] \rightarrow [\text{Bool}]$

zipWith

(e) $(a \rightarrow b \rightarrow c) \rightarrow \text{Maybe } a \rightarrow \text{Maybe } b \rightarrow \text{Maybe } c$

$\backslash x y z \rightarrow$

(f) $(a \rightarrow b) \rightarrow (b \rightarrow \text{Bool}) \rightarrow [a] \rightarrow [b]$

$\backslash x \rightarrow \text{filter}$

(g) $\text{Maybe } a \rightarrow (a \rightarrow [b]) \rightarrow [(a, b)]$

(h) $\text{Eq } a \Rightarrow a \rightarrow [a] \rightarrow [a]$

(i) $\text{Show } a \Rightarrow [a] \rightarrow \text{IO String}$

print

(j) $(a, b) \rightarrow (a \rightarrow b \rightarrow c) \rightarrow c$

(k) $((a, b) \rightarrow c) \rightarrow c$

Question 4 (20 points)

Consider the following Dafny program that inefficiently calculates the cube of a number:

```
method Cube(m:int) returns (y:int)
  requires m > 0
  ensures y == m*m*m
{
  y := 0;
  var x := m * m;
  var z := m;
  while (z > 0)
  {
    z := z - 1;
    y := y + x;
  }
}
```

Fill in the annotations in the following program to show that the Hoare triple given by the outermost pre- and post- conditions is valid. Be completely precise and pedantic in the way you apply Hoare rules—write assertions in *exactly* the form given by the rules rather than just logically equivalent ones. The provided blanks have been constructed so that if you work backwards from the end of the program you should only need to use the rule of consequence in the places indicated with $\rightarrow>$. These implication steps can (silently) rely on all the usual rules of arithmetic.

```
{ { m > 0 } }  $\rightarrow>$ 
{ {
  y := 0;
  var x := m * m;
  var z := m;
  while (z > 0) {
    { { z > 0 } }  $\rightarrow>$ 
    { { y = m * m + x } }
    { { z := z - 1;
      y := y + x;
    } }
  }
} }
{ { z < 0 } }  $\rightarrow>$ 
{ { y = m * m * m } }
{ { y = m * m * m } }
```

Question 5 (20 points)

Implement the following Haskell functions:

- (a) Implement a function `weave` that given two lists with elements of the same type, returns a list with elements alternating between the two lists. For example:

`weave [1,2,3] [4,5,6] = [1,4,2,5,3,6]`

You can assume that the lists have the same length.

~~`weave [] = []`~~
~~`weave = [] = []`~~

`weave (x:xs) (y:ys) = (x:y) ++ weave xs ys`

- (b) Implement a function `powerset`, that given a list of elements of some type, computes the list of all of its sublists, including the empty list and the original list itself, in any order. For example:

`powerset [1,2,3] = [[1,2,3], [1,2], [1,3], [1], [2,3], [2], [3], []]`

`powerset [] = [[]]`

`powerset (x:xs) = powerset xs ++ map (x:) powerset xs`

Lee Le
UID: 118028941

CMSC 433: Midterm Exam (Spring 2024)

Question 1 (20 points)

Consider the two standard folds in Haskell, `foldl` and `foldr`, whose definitions are given below:

```
foldr :: (a -> b -> b) -> b -> [a] -> b
foldr f b [] = b
foldr f b (x:xs) = f x (foldr f b xs)

foldl :: (b -> a -> b) -> b -> [a] -> b
foldl f b [] = b
foldl f b (x:xs) = foldl f (f b x) xs
```

Now consider the following two very similar but distinct folds over lists of integers:

```
f1 :: [Int] -> Int
f1 = foldr (-) 0

f2 :: [Int] -> Int
f2 = foldl (-) 0
```

Write two Dafny programs, one corresponding to each of `f1` and `f2`, but operating on arrays rather than lists. We've given you the method signatures to get you started:

<pre>method f1(a:array<int>) returns (out:int) { var i := a.length; out := 0; while (i > 0) invariant a.length >= i & i >= 0 { out := out - a[i]; i := i - 1; } return out; }</pre>	<pre>method f2(a:array<int>) returns (out:int) { var i := 0; out := 0; while (i < a.length) invariant 0 <= i <= a.length { out := out - a[i]; i := i + 1; } return out; }</pre>
--	--

Question 2 (20 points)

For each of the Haskell expressions below, write their (most general) Haskell type or "ill-typed" if it contains a type error. The type signatures of all functions below are provided in the appendix at the end.

(a) $\backslash x \rightarrow x$

Example answer:

$a \rightarrow a$

(b) $\backslash x\ y \rightarrow (x,y)$

$a \rightarrow b \rightarrow (a,b)$

(c) $\backslash x\ y \rightarrow \text{if } x == y \text{ then show } x \text{ else show } (x,y)$

$a \rightarrow a \rightarrow \text{string}$

(d) $\backslash x\ l \rightarrow x : l ++ l ++ [x]$

$a \rightarrow [a] \rightarrow [a]$

(e) $\text{foldr} (\text{const } l)$

$a \rightarrow [b] \rightarrow a$

(f) $(\cdot "42")$

$a \rightarrow (a, \text{string})$

(g) $(\cdot "42")$

ill-typed

(h) $\text{reverse} . \text{reverse}$

$[a] \rightarrow [a]$

(i) $\backslash l \rightarrow \text{filter } (< 42) (l ++ l)$

$[a] \rightarrow (a \rightarrow \text{Bool}) \rightarrow [a] \rightarrow [a]$

(j) $\text{let } f\ x = x \text{ in } (f\ 'a', f\ \text{True})$

$(a \rightarrow b) \rightarrow (b, b)$

(k) $\text{fmap} (\text{fmap } (+1)) [\text{Nothing}]$

$(a \rightarrow b) \rightarrow [\text{maybe } a] \rightarrow [\text{maybe } b]$

Question 3 (20 points)

Some functions are left ambiguous so that it isn't int, char, bool, etc, Denoted as "func" or "f"

For each of the types below, write a Haskell expression that has that type. Don't write trivial expressions (such as [], Nothing, or undefined) unless there is no other option. You can use any function from the appendix, do syntax, list comprehensions, or any valid Haskell.

(a) Int -> Int

Example answers:

\x -> x + 1
(+1)

(b) Bool -> [Bool]

func Boolize [] : Bool

(c) a -> Maybe b

~~x~~ -> Just (f x)

(d) (Int -> Char -> Bool) -> [Int] -> [Char] -> [Bool]

zipWith (f . c) fromEnum (>1) [ints] [chars]

(e) (a -> b -> c) -> Maybe a -> Maybe b -> Maybe c

zipWith f (maybeFunc x) (maybeFunc y)

(f) (a -> b) -> (b -> Bool) -> [a] -> [b]

\x f -> filter (f) [x -> 1, 1, 1, 1]

(g) Maybe a -> (a -> [b]) -> [(a, b)]

func mf = maybe [] (\a -> [(a, f a)]) m

(h) Eq a => a -> [a] -> [a]

delete x xs

(i) Show a => [a] -> IO String

func ele lst = putStr \$ show (ele!lst)

(j) (a, b) -> (a -> b -> c) -> c

func (a, b) f = f a b

(k) ((a, b) -> c) -> c

Question 4 (20 points)

Consider the following Dafny program that inefficiently calculates the cube of a number:

```
method Cube(m:int) returns (y:int)
  requires m > 0
  ensures y == m*m*m
{
  y := 0;
  var x := m * m;
  var z := m;
  while (z > 0)
  {
    z := z - 1;
    y := y + x;
  }
}
```

Fill in the annotations in the following program to show that the Hoare triple given by the outermost pre- and post-conditions is valid. Be completely precise and pedantic in the way you apply Hoare rules—write assertions in *exactly* the form given by the rules rather than just logically equivalent ones. The provided blanks have been constructed so that if you work backwards from the end of the program you should only need to use the rule of consequence in the places indicated with $\rightarrow$. These implication steps can (silently) rely on all the usual rules of arithmetic.

```
{ { m > 0 } }  $\rightarrow$ 
{ { y = m * m * m
  y := 0;
  { { y = 0 } }  $\wedge$  x = x  $\wedge$  z = z
    var x := m * m;
    { { y = 0 } }  $\wedge$  x = m * m  $\wedge$  z = m
      var z := m;
      { { m * z = 0 } }  $\wedge$  y = x * m
        while (z > 0) {
          { { m * z = 0 } }  $\wedge$  x = m * m  $\wedge$  y = 0
            { { z * z = z - 1 } }  $\wedge$  x = m * m  $\wedge$  y = (z - 1) * x
              z := z - 1;
              { { z = z - 1 } }  $\wedge$  y = (z - 1) * x
                y := y + x;
                { { z = z - 1 } }  $\wedge$  y = (z - 1) * x
              }
            { { z > 0 } }  $\wedge$  x = m * m  $\wedge$  y = x * m
          }
        }
      }
    }
  }
}
```

Hard to write &&, will use 'and'.

Question 5 (20 points)

Implement the following Haskell functions:

- (a) Implement a function `weave` that given two lists with elements of the same type, returns a list with elements alternating between the two lists. For example:

`weave [1,2,3] [4,5,6] = [1,4,2,5,3,6]`

You can assume that the lists have the same length.

`weave [] [] = []`
`weave [1] [2] = [1,2]`
`weave [x:xs] (y:ys) = x:y:weave xs ys`

- (b) Implement a function `powerset`, that given a list of elements of some type, computes the list of all of its sublists, including the empty list and the original list itself, in any order. For example:

`powerset [1,2,3] = [[], [1], [2], [3], [1,2], [1,3], [2,3], [1,2,3]]`

`powerset = foldl (\acc xs -> acc + map (x:) acc) [] xs`
`[[]]`

Arnav Bhatia UID: 117747201

CMSC 433: Midterm Exam (Spring 2024)

Question 1 (20 points)

Consider the two standard folds in Haskell, `foldl` and `foldr`, whose definitions are given below:

```
foldr :: (a -> b -> b) -> b -> [a] -> b
foldr f b [] = b
foldr f b (x:xs) = f x (foldr f b xs)

foldl :: (b -> a -> b) -> b -> [a] -> b
foldl f b [] = b
foldl f b (x:xs) = foldl f (f b x) xs
```

Now consider the following two very similar but distinct folds over lists of integers:

```
f1 :: [Int] -> Int
f1 = foldr (-) 0

f2 :: [Int] -> Int
f2 = foldl (-) 0
```

Write two Dafny programs, one corresponding to each of `f1` and `f2`, but operating on arrays rather than lists. We've given you the method signatures to get you started:

<pre>method f1(a:array<int>) returns (out:int) { var i :: int := a.length - 1; out := 0; while (i >= 0) invariant i >= 0; invariant i < a.length; { out := out - a[i]; i := i - 1; } out }</pre>	<pre>method f2(a:array<int>) returns (out:int) { var i :: int := 0; out := 0; while (i < a.length) invariant i < a.length; invariant i >= 0; { out := out - a[i]; i := i + 1; } out }</pre>
---	--

Question 2 (20 points)

For each of the Haskell expressions below, write their (most general) Haskell type or "ill-typed" if it contains a type error. The type signatures of all functions below are provided in the appendix at the end.

(a) $\backslash x \rightarrow x$

Example answer:

$a \rightarrow a$

(b) $\backslash x\ y \rightarrow (x,y)$

$a \rightarrow b \rightarrow (a,b)$

(c) $\backslash x\ y \rightarrow$ if $x == y$ then show x else show (x,y)

$a \rightarrow b \rightarrow \text{String}$

(d) $\backslash x\ l \rightarrow x : l ++ l ++ [x]$

$a \rightarrow a \rightarrow [a]$

(e) `foldr (const 1)`

(f) `("42")`

ill-typed

(g) `(.) "42"`

$b \rightarrow (Int, b)$

(h) `reverse . reverse`

$[a] \rightarrow [a]$

(i) $\backslash l \rightarrow$ filter (< 42) $(l ++ l)$

$a \rightarrow [a]$

(j) let $f\ x = x$ in $(f\ 'a', f\ \text{True})$

$(\text{Char} \rightarrow \text{Bool}) \rightarrow \text{Char} \rightarrow \text{Bool}$

(k) `fmap (fmap (+1)) [Nothing]`

Question 3 (20 points)

For each of the types below, write a Haskell expression that has that type. Don't write trivial expressions (such as [], Nothing, or undefined) unless there is no other option. You can use any function from the appendix, do syntax, list comprehensions, or any valid Haskell.

(a) `Int -> Int`

Example answers:

`\x -> x + 1`

`(+1)`

(b) `Bool -> [Bool]`

`\a -> repeat True`

(c) `a -> Maybe b`

`\a -> lookup a [1..1]`

(d) `(Int -> Char -> Bool) -> [Int] -> [Char] -> [Bool]`

(e) `(a -> b -> c) -> Maybe a -> Maybe b -> Maybe c`

`zipWith Maybe a Maybe b`

(f) `(a -> b) -> (b -> Bool) -> [a] -> [b]`

(g) `Maybe a -> (a -> [b]) -> [(a,b)]`

(h) `Eq a => a -> [a] -> [a]`

`f x l = \a -> map (a==a) l`

(i) `Show a => [a] -> IO String`

`\a -> show a`

(j) `(a,b) -> (a -> b -> c) -> c`

`f (a,b) x -> x show a show show b`

(k) `((a,b) -> c) -> c`

Question 4 (20 points)

Consider the following Dafny program that inefficiently calculates the cube of a number:

```
method Cube(m:int) returns (y:int)
  requires m > 0
  ensures y == m*m*m
{
  y := 0;
  var x := m * m;    x = 9
  var z := m;        z = 3
  while (z > 0)
  {
    z := z - 1;
    y := y + x;
  }
}
```

$m = 3$

$m = 3$

z	y	x
3	0	9
2	9	9
1	18	9
0	27	9

$x = 9$

$$y = (m-z) * x$$

Fill in the annotations in the following program to show that the Hoare triple given by the outermost pre- and post-conditions is valid. Be completely precise and pedantic in the way you apply Hoare rules—write assertions in *exactly* the form given by the rules rather than just logically equivalent ones. The provided blanks have been constructed so that if you work backwards from the end of the program you should only need to use the rule of consequence in the places indicated with $\rightarrow>$. These implication steps can (silently) rely on all the usual rules of arithmetic.

```
{ { m > 0 } }  $\rightarrow>$ 
{ {  $0 = (m-m) * (m * m)$  } }
  y := 0;
  { {  $y = (m-m) * (m * m)$  } }
  var x := m * m;
  { {  $y = (m-m) * x$  } }
  var z := m;
  { {  $y = (m-z) * x$  } }
  while (z > 0) {
    { { z > 0 } }  $\wedge$   $y = (m-z) * x$ 
    { {  $y + x = (m-(z-1)) * x$  } }  $\rightarrow>$ 
    z := z - 1;
    { {  $y + x = (m-z) * x$  } }
    y := y + x;
    { {  $y = (m-z) * x$  } }
  }
  { { z == 0 } }  $\wedge$   $y = (m-z) * x$ 
  { { y = m * m * m } }  $\rightarrow>$ 
```


Question 5 (20 points)

Implement the following Haskell functions:

- (a) Implement a function `weave` that given two lists with elements of the same type, returns a list with elements alternating between the two lists. For example:

`weave [1,2,3] [4,5,6] = [1,4,2,5,3,6]`

You can assume that the lists have the same length.

```
weave :: [a] -> [a] -> [a]
weave [] (x:xs) = (x:xs)
weave (x:xs) [] = (x:xs)
weave (x:xs) (y:ys) = x:y:(weave xs ys)
```

- (b) Implement a function `powerset`, that given a list of elements of some type, computes the list of all of its sublists, including the empty list and the original list itself, in any order. For example:

```
powerset [1,2,3] = [[1,2,3],[1,2],[1,3],[1],[2,3],[2],[3],[]]
powerset [] -> [[]]
powerset [a] -> [[a]]
powerset [c] = [c]
```

`powerset l = helper l`

```
[1 2 3]
[1 2]
[1 3]
[2 3]
[2]
[3]
[3 1]
[]
```

`x:xs = 1:(2:3)`

```
[1 2 3]
[1 2 3] [1 3 4] [1 2 4]
[1 2 3] [1 3]
[1 4]
```


Andrew Lee
117924194

CMSC 433: Midterm Exam (Spring 2024)

Question 1 (20 points)

Consider the two standard folds in Haskell, `foldl` and `foldr`, whose definitions are given below:

```
foldr :: (a -> b -> b) -> b -> [a] -> b
foldr f b [] = b
foldr f b (x:xs) = f x (foldr f b xs)
```

```
foldl :: (b -> a -> b) -> b -> [a] -> b
foldl f b [] = b
foldl f b (x:xs) = foldl f (f b x) xs
```

Now consider the following two very similar but distinct folds over lists of integers:

```
f1 :: [Int] -> Int
f1 = foldr (-) 0
```

```
f2 :: [Int] -> Int
f2 = foldl (-) 0
```

Write two Dafny programs, one corresponding to each of `f1` and `f2`, but operating on arrays rather than lists. We've given you the method signatures to get you started:

```
method f1(a:array<int>) returns (out:int) {
  var i: int := a.Length-1;
  out := 0;
  while (i >= 0) {
    out := out - a[i];
    i := i - 1;
  }
}
```

```
method f2(a:array<int>) returns (out:int) {
  var i: int := 0;
  out := 0;
  while (i < a.Length) {
    out := out - a[i];
    i := i + 1;
  }
}
```

}

}

Question 2 (20 points)

For each of the Haskell expressions below, write their (most general) Haskell type or "ill-typed" if it contains a type error. The type signatures of all functions below are provided in the appendix at the end.

(a) $\backslash x \rightarrow x$

Example answer:

$a \rightarrow a$

(b) $\backslash x y \rightarrow (x,y)$

$a \rightarrow b \rightarrow (a,b)$

(c) $\backslash x y \rightarrow$ if $x == y$ then show x else show (x,y)

$a \rightarrow b \rightarrow \text{String}$

(d) $\backslash x l \rightarrow x : l ++ l ++ [x]$

$a \rightarrow b \rightarrow b$

(e) `foldr (const 1)`

$(a \rightarrow b \rightarrow a) \rightarrow [a] \rightarrow a$

(f) `("42")`

ill-typed

(g) `(,)` `"42"`

$\text{String} \rightarrow \text{String} \rightarrow (\text{String}, \text{String})$

(h) `reverse . reverse`

$[a] \rightarrow [a]$

(i) $\backslash l \rightarrow$ filter (< 42) $(l ++ l)$

$(a \rightarrow \text{Bool}) \rightarrow [a] \rightarrow [a]$

(j) let $f\ x = x$ in $(f\ 'a', f\ \text{True})$

$x \rightarrow (\text{Char} \rightarrow \text{Bool}) \rightarrow x \rightarrow \text{Char} \rightarrow x \rightarrow \text{Bool}$

(k) `fmap (fmap (+1)) [Nothing]`

ill-typed

Question 3 (20 points)

For each of the types below, write a Haskell expression that has that type. Don't write trivial expressions (such as [], Nothing, or undefined) unless there is no other option. You can use any function from the appendix, do syntax, list comprehensions, or any valid Haskell.

(a) `Int -> Int`

Example answers:

`\x -> x + 1`
`(+1)`

(b) `Bool -> [Bool]`

`\a -> True`

(c) `a -> Maybe b`

`Nothing`

(d) `(Int -> Char -> Bool) -> [Int] -> [Char] -> [Bool]`

`\ a b -> and (a,b) && a.length == b.length`

(e) `(a -> b -> c) -> Maybe a -> Maybe b -> Maybe c`

`zipWith (Maybe a) (Maybe b)`

(f) `(a -> b) -> (b -> Bool) -> [a] -> [b]`

`\ a b -> map a b`

(g) `Maybe a -> (a -> [b]) -> [(a,b)]`

`\ Maybe a b -> zip a b`

(h) `Eq a => a -> [a] -> [a]`

`delete`

(i) `Show a => [a] -> IO String`

`\ IO a -> show a`

(j) `(a,b) -> (a -> b -> c) -> c`

`\ (a,b) -> delete a [a,b]`

(k) `((a,b) -> c) -> c`

`\ a,b -> delete b [a,b]`

Question 4 (20 points)

Consider the following Dafny program that inefficiently calculates the cube of a number:

```
method Cube(m:int) returns (y:int)
  requires m > 0
  ensures y == m*m*m
{
  y := 0;
  var x := m * m;
  var z := m;
  while (z > 0)
  {
    z := z - 1;
    y := y + x;
  }
}
```

Handwritten annotations for the program:

- Initial state: $y = x \cdot (m-2)$
- Loop invariant: $y + x = x \cdot (m - z + 1)$
- At $z = 1$: $x \quad 3 - 3 + 1$
- At $z = 0$: $18 = 9 \cdot (3 - 2 + 1)$

Fill in the annotations in the following program to show that the Hoare triple given by the outermost pre- and post-conditions is valid. Be completely precise and pedantic in the way you apply Hoare rules - write assertions in *exactly* the form given by the rules rather than just logically equivalent ones. The provided blanks have been constructed so that if you work backwards from the end of the program you should only need to use the rule of consequence in the places indicated with $\vdash \gg$. These implication steps can (silently) rely on all the usual rules of arithmetic.

```
{ { m > 0 } }  $\vdash \gg$ 
{ { 0 = m * m * (m-m) } } m > 0 } } (m <= m)
{ { y := 0;
  y = m * m * (m-m) } } m > 0 } } (m <= m)
{ { y = x * (m-m) } } m > 0 } } (m <= m)
{ { y := m;
  y = x * (m-2) } } z > 0 } } (z <= m)
{ { z > 0 } } { { y = x * (m-2) } } (z <= m)
{ { y + x = x * (m-2) } } (z <= m)
{ { z := z - 1;
  y + x = x * (m-2) } } (z <= m)
{ { y := y + x;
  y = x * (m-2) } } (z <= m)
{ { ! (z > 0) } } y = x * (m-2) } } (z <= m)
{ { y = m * m * m } }
```

Handwritten annotations for the program:

- Initial state: $z = x + y + c$

Question 5 (20 points)

Implement the following Haskell functions:

- (a) Implement a function `weave` that given two lists with elements of the same type, returns a list with elements alternating between the two lists. For example:

```
weave [1,2,3] [4,5,6] = [1,4,2,5,3,6]
```

You can assume that the lists have the same length.

```
weave (x : xs) (\s : ss) = x : s : weave (xs ss)
```

- (b) Implement a function `powerset`, that given a list of elements of some type, computes the list of all of its sublists, including the empty list and the original list itself, in any order. For example:

```
powerset [1,2,3] = [[1,2,3], [1,2], [1,3], [1], [2,3], [2], [3], []]
```

```
powerset [] = foldr (\a b -> powerset a ++ b) [[]]
```


Anshagh Bulkar 118501604

CMSC 433: Midterm Exam (Spring 2024)

Question 1 (20 points)

Consider the two standard folds in Haskell, `foldl` and `foldr`, whose definitions are given below:

```
foldr :: (a -> b -> b) -> b -> [a] -> b
foldr f b [] = b
foldr f b (x:xs) = f x (foldr f b xs)

foldl :: (b -> a -> b) -> b -> [a] -> b
foldl f b [] = b
foldl f b (x:xs) = foldl f (f b x) xs
```

Now consider the following two very similar but distinct folds over lists of integers:

```
f1 :: [Int] -> Int
f1 = foldr (-) 0

f2 :: [Int] -> Int
f2 = foldl (-) 0
```

Write two Dafny programs, one corresponding to each of `f1` and `f2`, but operating on arrays rather than lists. We've given you the method signatures to get you started:

<pre>method f1(a:array<int>) returns (out:int) { var i := a.length; out := 0; while (i > 0) { out := out - a[i]; i := i - 1; } return out; }</pre>	<pre>method f2(a:array<int>) returns (out:int) { var i := 0; out := 0; while (i < a.length) { out := out + a[i]; i := i + 1; } return out; }</pre>
---	---

Question 2 (20 points)

For each of the Haskell expressions below, write their (most general) Haskell type or "ill-typed" if it contains a type error. The type signatures of all functions below are provided in the appendix at the end.

(a) $\backslash x \rightarrow x$

Example answer:

$a \rightarrow a$

(b) $\backslash x\ y \rightarrow (x,y)$

$a \rightarrow b \rightarrow (a,b)$

(c) $\backslash x\ y \rightarrow$ if $x == y$ then show x else show (x,y)

ill-typed

(d) $\backslash x\ l \rightarrow x : l ++ l ++ [x]$

$a \rightarrow [a] \rightarrow [a]$

(e) $\text{foldr} (\text{const } 1)$

$a \rightarrow b \rightarrow b$

(f) $(\cdot, 42)$

$a \rightarrow b \rightarrow (a,b)$

(g) $(\cdot, 42)$

ill-typed

(h) $\text{reverse} . \text{reverse}$

$a \rightarrow [a]$

(i) $\backslash l \rightarrow \text{filter} (< 42) (l ++ l)$

$[Int] \rightarrow [Int]$

(j) $\text{let } f\ x = x \text{ in } (f\ 'a', f\ \text{True})$

$(a \rightarrow b) \rightarrow \text{Bool}$

(k) $\text{fmap} (\text{fmap } (+1)) [\text{Nothing}]$

$[Maybe] \rightarrow [Maybe]$

Question 3 (20 points)

For each of the types below, write a Haskell expression that has that type. Don't write trivial expressions (such as [], Nothing, or undefined) unless there is no other option. You can use any function from the appendix, do syntax, list comprehensions, or any valid Haskell.

(a) `Int -> Int`

Example answers:

`\x -> x + 1`
`(+1)`

(b) `Bool -> [Bool]`

`true -> [true]`

(c) `a -> Maybe b`

`\x -> Nothing`

(d) `(Int -> Char -> Bool) -> [Int] -> [Char] -> [Bool]`

`\f [5] [a] -> if (f [5] [a]) then [true] else [false]`

(e) `(a -> b -> c) -> Maybe a -> Maybe b -> Maybe c`

`\f (Just x) (Just y) -> Just (f x y)`

(f) `(a -> b) -> (b -> Bool) -> [a] -> [b]`

`\f g (x:[_]) -> if (g(f x)) then [(f x)] else []`

(g) `Maybe a -> (a -> [b]) -> [(a,b)]`

`\(Just x) f -> case (f (Just x)) of [] -> [(x,[])]`

(h) `Eq a => a -> [a] -> [a]`

`(y:y) -> [(x,y)]`

`delete`

(i) `Show a => [a] -> IO String`

`! (replicate 1000000 'a)`

(j) `(a,b) -> (a -> b -> c) -> c`

`\(x,y) g -> g x y`

(k) `((a,b) -> c) -> c`

`\f -> undefined`

Question 4 (20 points)

Consider the following Dafny program that inefficiently calculates the cube of a number:

```
method Cube(m:int) returns (y:int)
  requires m > 0
  ensures y == m*m*m
{
  y := 0;
  var x := m * m;
  var z := m;
  while (z > 0)
  {
    z := z - 1;
    y := y + x;
  }
}
```

Fill in the annotations in the following program to show that the Hoare triple given by the outermost pre- and post-conditions is valid. Be completely precise and pedantic in the way you apply Hoare rules—write assertions in *exactly* the form given by the rules rather than just logically equivalent ones. The provided blanks have been constructed so that if you work backwards from the end of the program you should only need to use the rule of consequence in the places indicated with $\rightarrow$. These implication steps can (silently) rely on all the usual rules of arithmetic.

```

{ { m > 0 } } -> >
{ { 0 + m < m = (m * m) * (m - (m - 1)) } }
  y := 0;
  { { y + m < m = (m * m) * (m - (m - 1)) } }
    var x := m * m;
    { { y + x < x = x * (m - 1) } }
      var z := m;
      { { y + x = x * (m - z - 1) } }
        while (z > 0) {
          { { y + x = x * (m - z - 1) } }
            { { y + x = x * (m - z - 1) } }
              { { y + x = x * (m - z - 1) } }
                z := z - 1;
                { { y + x > x * (m - z) } }
                  y := y + x;
                  { { y = x * (m - z) } }
                    }
                }
              }
            }
          }
        }
      }
    }
  }
}

```

$$\begin{aligned} y &= 9(8272) \\ x &= 9999 \\ z &= 2210 \end{aligned}$$

Question 5 (20 points)

Implement the following Haskell functions:

- (a) Implement a function `weave` that given two lists with elements of the same type, returns a list with elements alternating between the two lists. For example:

`weave [1,2,3] [4,5,6] = [1,4,2,5,3,6]`

You can assume that the lists have the same length.

```
weave :: [a] -> [a] -> [a]
weave [] [] = []
weave (x:xs) (y:ys) = x:y:(weave xs ys)
```

- (b) Implement a function `powerset`, that given a list of elements of some type, computes the list of all of its sublists, including the empty list and the original list itself, in any order. For example:

`powerset [1,2,3] = [[], [1], [2], [3], [1,2], [1,3], [2,3], [1,2,3]]`

```
powerset :: [a] -> [[a]]
powerset [] = [[]]
powerset (x:xs) = (x:xs):(powerset xs)
```


Pengfei Yan
11776171

CMSC 433: Midterm Exam (Spring 2024)

Question 1 (20 points)

Consider the two standard folds in Haskell, `foldl` and `foldr`, whose definitions are given below:

```
foldr :: (a -> b -> b) -> b -> [a] -> b
foldr f b [] = b
foldr f b (x:xs) = f x (foldr f b xs)

foldl :: (b -> a -> b) -> b -> [a] -> b
foldl f b [] = b
foldl f b (x:xs) = foldl f (f b x) xs
```

Now consider the following two very similar but distinct folds over lists of integers:

```
f1 :: [Int] -> Int
f1 = foldr (-) 0

f2 :: [Int] -> Int
f2 = foldl (-) 0
```

Write two Dafny programs, one corresponding to each of `f1` and `f2`, but operating on arrays rather than lists. We've given you the method signatures to get you started:

<pre>method f1(a:array<int>) returns (out:int) { var i:int := 0; out := 0; while (i < a.length-1) { out := out - a[i]; i := i+1; } }</pre>	<pre>method f2(a:array<int>) returns (out:int) { var i:int := a.length-1; out := 0; while (i >= 0) { out := out - a[i]; i := i-1; } }</pre>
---	--

Question 2 (20 points)

For each of the Haskell expressions below, write their (most general) Haskell type or "ill-typed" if it contains a type error. The type signatures of all functions below are provided in the appendix at the end.

(a) $\backslash x \rightarrow x$

Example answer:

$a \rightarrow a$

(b) $\backslash x y \rightarrow (x,y)$

$a \rightarrow b \rightarrow (a,b)$

(c) $\backslash x y \rightarrow$ if $x == y$ then show x else show (x,y)

$a \rightarrow b \rightarrow \text{String}$

(d) $\backslash x l \rightarrow x : l ++ l ++ [x]$

$a \rightarrow [a] \rightarrow [a]$

(e) $\text{foldr} (\text{const } 1)$

$c \rightarrow b \rightarrow b \rightarrow (a \rightarrow b \rightarrow b)$

(f) $(^*42^*)$

$[\text{String}]$

(g) $(^*42^*)$

$\text{String} \rightarrow \text{String} \rightarrow (\text{String}, \text{String})$

(h) $\text{reverse} . \text{reverse}$

$[b \rightarrow c] \rightarrow (a \rightarrow b) \rightarrow a \rightarrow [c] \rightarrow [c]$

(i) $\backslash l \rightarrow \text{filter} (< 42) (l ++ l)$

$\text{Num } a \Rightarrow (a \rightarrow \text{Bool}) \rightarrow [a] \rightarrow [a]$

(j) $\text{let } f \ x = x \text{ in } (f 'a', f \text{True})$

$(a \rightarrow b) \rightarrow a$

(k) $\text{fmap} (\text{fmap } (+1)) [\text{Nothing}]$

$((\text{Num } a \Rightarrow \text{Maybe } a \rightarrow a) \rightarrow [\text{Maybe } a]) \rightarrow f (\text{Num } a \Rightarrow \text{Maybe } a \rightarrow a) \rightarrow f ([\text{Maybe } a])$

Question 3 (20 points)

For each of the types below, write a Haskell expression that has that type. Don't write trivial expressions (such as [], Nothing, or undefined) unless there is no other option. You can use any function from the appendix; do syntax, list comprehensions, or any valid Haskell.

(a) $\text{Int} \rightarrow \text{Int}$

Example answers:

$\backslash x \rightarrow x + 1$
 $(+1)$

(b) $\text{Bool} \rightarrow [\text{Bool}]$

repeat True

(c) $a \rightarrow \text{Maybe } b$

$\backslash a \rightarrow \text{Just } b$

(d) $(\text{Int} \rightarrow \text{Char} \rightarrow \text{Bool}) \rightarrow [\text{Int}] \rightarrow [\text{Char}] \rightarrow [\text{Bool}]$

zipWith 4 h False

(e) $(a \rightarrow b \rightarrow c) \rightarrow \text{Maybe } a \rightarrow \text{Maybe } b \rightarrow \text{Maybe } c$

liftToMaybe !

(f) $(a \rightarrow b) \rightarrow (b \rightarrow \text{Bool}) \rightarrow [a] \rightarrow [b]$

(g) $\text{Maybe } a \rightarrow (a \rightarrow [b]) \rightarrow [(a,b)]$

(h) $\text{Eq } a \Rightarrow a \rightarrow [a] \rightarrow [a]$

delete 1 [1,2,3]

(i) $\text{Show } a \Rightarrow [a] \rightarrow \text{IO String}$

show "Halo"

(j) $(a,b) \rightarrow (a \rightarrow b \rightarrow c) \rightarrow c$

$\backslash (a,b) \rightarrow (a \rightarrow b \rightarrow c) \rightarrow c$

(k) $((a,b) \rightarrow c) \rightarrow c$

Question 4 (20 points)

Consider the following Dafny program that inefficiently calculates the cube of a number:

```
method Cube(m:int) returns (y:int)
  requires m > 0
  ensures y == m*m*m
{
  y := 0;
  var x := m * m;
  var z := m;
  while (z > 0)
  {
    z := z - 1;
    y := y + x;
  }
}
```

$m=2$
 $y=0 \quad y=4 \quad y=8$
 $x=4 \quad \rightarrow x=4 \quad \rightarrow x=4$
 $z=m=2 \quad z=1 \quad z=0$
 $m=3$
 $y=0 \quad \rightarrow y=9 \quad y=18$
 $x=9 \quad \rightarrow x=9 \quad \rightarrow x=9$
 $z=3 \quad z=2 \quad z=1 \quad z=0$

Fill in the annotations in the following program to show that the Hoare triple given by the outermost pre- and post-conditions is valid. Be completely precise and pedantic in the way you apply Hoare rules—write assertions in *exactly* the form given by the rules rather than just logically equivalent ones. The provided blanks have been constructed so that if you work backwards from the end of the program you should only need to use the rule of consequence in the places indicated with $\rightarrow$. These implication steps can (silently) rely on all the usual rules of arithmetic.

```
{ { m > 0 } }  $\rightarrow$  >>
{ { m > 0 && 0 == m*m*m - 1 } }
  y := 0;
{ { m > 0 && y == m*m*m - 1 } }
  var x := m * m;
  { { m > 0 && y == x - } }
  var z := m;
  { { z > 0 && y == x - } }
  while (z > 0) {
    { { z > 0 && y == x - } }
    { { z-1 > 0 && y+x == x - } }
    z := z - 1;
    { { z > 0 && y+x == x - } }
    y := y + x;
    { { z > 0 && y == x - } }
  }
  { { z > 0 && y == x - } }
  { { y = m * m * m } }
```

Question 5 (20 points)

Implement the following Haskell functions:

- (a) Implement a function `weave` that given two lists with elements of the same type, returns a list with elements alternating between the two lists. For example:

`weave [1,2,3] [4,5,6] = [1,4,2,5,3,6]`

You can assume that the lists have the same length.

weave [3] = [3]
 weave [4] = [4]
 weave xs ys = xs:ys:weave xs ys

- (b) Implement a function `powerSet`, that given a list of elements of some type, computes the list of all of its sublists, including the empty list and the original list itself, in any order. For example:

`powerSet [1,2,3] = [[1,2,3], [1,2], [1,3], [1], [2,3], [2], [3], []]`
`powerSet [] = []`
`powerSet xs = [xs:xs'] | map \x xs' -> powerSet xs'`

Elizabeth Casey
117967791

CMSC 433: Midterm Exam (Spring 2024)

Question 1 (20 points)

Consider the two standard folds in Haskell, `foldl` and `foldr`, whose definitions are given below:

```
foldr :: (a -> b -> b) -> b -> [a] -> b
foldr f b [] = b
foldr f b (x:xs) = f x (foldr f b xs)
```

```
foldl :: (b -> a -> b) -> b -> [a] -> b
foldl f b [] = b
foldl f b (x:xs) = foldl f (f b x) xs
```

Now consider the following two very similar but distinct folds over lists of integers:

```
f1 :: [Int] -> Int
f1 = foldr (-) 0
```

```
f2 :: [Int] -> Int
f2 = foldl (-) 0
```

Write two Dafny programs, one corresponding to each of `f1` and `f2`, but operating on arrays rather than lists. We've given you the method signatures to get you started:

```
method f1(a:array<int>) returns (out:int) {
  var i, int := a.Length-1
  var out, int := 0
  while (i >= 0) {
    out := a[i] - out
    i := i - 1
  }
```

```
method f2(a:array<int>) returns (out:int) {
  var i, int := 0
  var out, int := 0
  while (i < a.Length) {
    out := out - a[i]
    i := i + 1
  }
```

Question 2 (20 points)

For each of the Haskell expressions below, write their (most general) Haskell type or "ill-typed" if it contains a type error. The type signatures of all functions below are provided in the appendix at the end.

(a) $\backslash x \rightarrow x$

Example answer:

$a \rightarrow a$

(b) $\backslash x\ y \rightarrow (x,y)$

$a \rightarrow b \rightarrow (a,b)$

(c) $\backslash x\ y \rightarrow \text{if } x == y \text{ then show } x \text{ else show } (x,y)$

$a \rightarrow (a \rightarrow b)$

(d) $\backslash x\ l \rightarrow x : l ++ l ++ [x]$

$a \rightarrow [a] \rightarrow [a]$

(e) $\text{foldr } (\text{const } 1)$

ill-typed

(f) $(\cdot "42")$

$(\text{Nothing}, a)$

(g) $(\cdot "42")$

ill-typed

(h) $\text{reverse} . \text{reverse}$

$[a] \rightarrow [a]$

(i) $\backslash l \rightarrow \text{filter } (< 42) (l ++ l)$

$[a] \rightarrow [a]$

(j) $\text{let } f\ x = x \text{ in } (f\ 'a', f\ \text{True})$

ill-typed

(k) $\text{fmap } (\text{fmap } (+1)) [\text{Nothing}]$

$[a] \rightarrow [\text{Nothing}]$

Question 3 (20 points)

For each of the types below, write a Haskell expression that has that type. Don't write trivial expressions (such as [], Nothing, or undefined) unless there is no other option. You can use any function from the appendix, do syntax, list comprehensions, or any valid Haskell.

(a) `Int -> Int`

Example answers:

`\x -> x + 1`

`(+1)`

(b) `Bool -> [Bool]`

`\b -> [b]`

(c) `a -> Maybe b`

`\a -> Nothing`

(d) `(Int -> Char -> Bool) -> [Int] -> [Char] -> [Bool]`

`\f n c -> [f n c]`

(e) `(a -> b -> c) -> Maybe a -> Maybe b -> Maybe c`

`\f a b -> Just (f a b)`

(f) `(a -> b) -> (b -> Bool) -> [a] -> [b]`

`\f g xs -> filter g (map f xs)`

(g) `Maybe a -> (a -> [b]) -> [(a,b)]`

`undefined`

(h) `Eq a => a -> [a] -> [a]`

`undefined`

(i) `Show a => [a] -> IO String`

`undefined`

(j) `(a,b) -> (a -> b -> c) -> c`

`\(a,b) f -> f a b`

(k) `((a,b) -> c) -> c`

`undefined`

Question 4 (20 points)

Consider the following Dafny program that inefficiently calculates the cube of a number:

```
method Cube(m:int) returns (y:int)
  requires m > 0
  ensures y == m*m*m
{
  y := 0;
  var x := m * m;
  var z := m;
  while (z > 0)
  {
    z := z - 1;
    y := y + x;
  }
}
```

Fill in the annotations in the following program to show that the Hoare triple given by the outermost pre- and post-conditions is valid. Be completely precise and pedantic in the way you apply Hoare rules---write assertions in *exactly* the form given by the rules rather than just logically equivalent ones. The provided blanks have been constructed so that if you work backwards from the end of the program you should only need to use the rule of consequence in the places indicated with $\vdash \gg$. These implication steps can (silently) rely on all the usual rules of arithmetic.

[illegible]

$$y + x = x(x-1)$$

[illegible]

$$x(t-m) = h$$

$$f_{(m,m)}(t-m) = h$$

$$m \rightarrow h$$

$$C(m, m) =$$

Question 5 (20 points)

Implement the following Haskell functions:

- (a) Implement a function `weave` that given two lists with elements of the same type, returns a list with elements alternating between the two lists. For example:

`weave [1,2,3] [4,5,6] = [1,4,2,5,3,6]`

You can assume that the lists have the same length.

`weave [] [] = []`

`weave (a:as) (b:bs) = a:b(weave as bs)`

- (b) Implement a function `powerset`, that given a list of elements of some type, computes the list of all of its sublists, including the empty list and the original list itself, in any order. For example:

`powerset [1,2,3] = [[], [1,2,3], [1,2], [1,3], [2,3], [1], [2], [3], []]`

`powerset [] = []`

~~`powerset (x:xs) = (x:xs) ++ (powerset xs)`~~
`powerset (x:xs) = (x:xs) ++ (powerset xs)`

Paul Vitters

CMSC 433: Midterm Exam (Spring 2024)

Question 1 (20 points)

Consider the two standard folds in Haskell, `foldl` and `foldr`, whose definitions are given below:

```
foldr :: (a -> b -> b) -> b -> [a] -> b
foldr f b [] = b
foldr f b (x:xs) = f x (foldr f b xs)

foldl :: (b -> a -> b) -> b -> [a] -> b
foldl f b [] = b
foldl f b (x:xs) = foldl f (f b x) xs
```

Now consider the following two very similar but distinct folds over lists of integers:

```
f1 :: [Int] -> Int
f1 = foldr (-) 0

f2 :: [Int] -> Int
f2 = foldl (-) 0
```

Write two Dafny programs, one corresponding to each of `f1` and `f2`, but operating on arrays rather than lists. We've given you the method signatures to get you started:

<pre>method f1(a:array<int>) returns (out:int) { out := 0; var i := 0; while (i < a.length) { out := out - a[i]; i := i + 1; } }</pre>	<pre>method f2(a:array<int>) returns (out:int) { out := 0; var i := a.length - 1; while (i >= 0) { out := a[i] - out; i := i - 1; } }</pre>
---	--

Question 2 (20 points)

For each of the Haskell expressions below, write their (most general) Haskell type or "ill-typed" if it contains a type error. The type signatures of all functions below are provided in the appendix at the end.

(a) $\backslash x \rightarrow x$

Example answer:

$a \rightarrow a$

(b) $\backslash x\ y \rightarrow (x,y)$

$a \rightarrow b \rightarrow (a,b)$

(c) $\backslash x\ y \rightarrow \text{if } x == y \text{ then show } x \text{ else show } (x,y)$

(d) $\backslash x\ l \rightarrow x : l ++ l ++ [x]$

$a \rightarrow [a] \rightarrow [a]$

(e) $\text{foldr } (\text{const } 1)$

$[a] \rightarrow \text{Integer}$

(f) $(\cdot, "42")$

ill-typed

(g) $(\cdot, "42")$

$a \rightarrow (a, \text{String})$

(h) $\text{reverse} \cdot \text{reverse}$

$[a] \rightarrow [a]$

(i) $\backslash l \rightarrow \text{filter } (< 42) (l ++ l)$

$a \rightarrow [a]$

(j) $\text{let } f\ x = x \text{ in } (f\ 'a', f\ \text{True})$

$(\text{Char}, \text{Bool})$

(k) $\text{fmap } (\text{fmap } (+1)) [\text{Nothing}]$

ill-typed

Question 3 (20 points)

For each of the types below, write a Haskell expression that has that type. Don't write trivial expressions (such as [], Nothing, or undefined) unless there is no other option. You can use any function from the appendix, do syntax, list comprehensions, or any valid Haskell.

(a) $\text{Int} \rightarrow \text{Int}$

Example answers:

$\backslash x \rightarrow x + 1$

$(+1)$

(b) $\text{Bool} \rightarrow [\text{Bool}]$

$\backslash x \rightarrow \text{if } x \text{ then } [x] \text{ else } []$

(c) $a \rightarrow \text{Maybe } b$

(d) $(\text{Int} \rightarrow \text{Char} \rightarrow \text{Bool}) \rightarrow [\text{Int}] \rightarrow [\text{Char}] \rightarrow [\text{Bool}]$

$\backslash f \rightarrow \text{map} (\lambda i \rightarrow \lambda c \rightarrow \text{if } c == '1' \text{ then } \text{True} \text{ else } \text{False})$

(e) $(a \rightarrow b \rightarrow c) \rightarrow \text{Maybe } a \rightarrow \text{Maybe } b \rightarrow \text{Maybe } c$

$\backslash f \ a \ b \rightarrow$

(f) $(a \rightarrow b) \rightarrow (b \rightarrow \text{Bool}) \rightarrow [a] \rightarrow [b]$

$\backslash f1 \ f2 \ as \rightarrow \text{map } f1 \ as \mapsto \text{map } f2 \ (\text{map } f1 \ as)$

(g) $\text{Maybe } a \rightarrow (a \rightarrow [b]) \rightarrow [(a,b)]$

$\backslash m \ f$

(h) $\text{Eq } a \Rightarrow a \rightarrow [a] \rightarrow [a]$

$\backslash x \ (y : fs) \rightarrow \text{if } x == y \text{ then } ys \text{ else } [y]$

(i) $\text{Show } a \Rightarrow [a] \rightarrow \text{IO String}$

$\backslash (x : xs) \rightarrow \text{show } x$

(j) $(a,b) \rightarrow (a \rightarrow b \rightarrow c) \rightarrow c$

$\backslash (x,y) \ f \rightarrow (f \ x \ y)$

(k) $((a,b) \rightarrow c) \rightarrow c$

Question 4 (20 points)

Consider the following Dafny program that inefficiently calculates the cube of a number:

```
method Cube(m:int) returns (y:int)
  requires m > 0
  ensures y == m*m*m
{
  y := 0;
  var x := m * m;
  var z := m;
  while (z > 0)
  {
    z := z - 1;
    y := y + x;
  }
}
```

Fill in the annotations in the following program to show that the Hoare triple given by the outermost pre- and post- conditions is valid. Be completely precise and pedantic in the way you apply Hoare rules — write assertions in *exactly* the form given by the rules rather than just logically equivalent ones. The provided blanks have been constructed so that if you work backwards from the end of the program you should only need to use the rule of consequence so that if you work backwards from the end of the program you should only rely on all the usual rules of arithmetic.

```
{ { m > 0 } } ->>
{ { m > 0 } }
  y := 0;
  { { y = 0 } }
    var x := m * m;
    { { x > 0 } }
      var z := m;
      { { z > 0 } }
        while (z > 0) {
          { { } }
          { { } }
          z := z - 1;
          { { z > 0 } }
          y := y + x;
          { { 0 < y <= m^3 } }
        }
      { { } }
    { { } }
  { { y == m * m * m } }
```


Question 5 (20 points)

Implement the following Haskell functions:

- (a) Implement a function `weave` that given two lists with elements of the same type, returns a list with elements alternating between the two lists. For example:

`weave [1,2,3] [4,5,6] = [1,4,2,5,3,6]`

You can assume that the lists have the same length.

~~`weave :: [a] -> [a] -> [a]`~~

~~`weave [] [] = []`~~

~~`weave (x:xs) (y:ys) = x:y:(weave xs ys)`~~

- (b) Implement a function `powerset`, that given a list of elements of some type, computes the list of all of its sublists, including the empty list and the original list itself, in any order. For example:

`powerset [1,2,3] = [[1,2,3], [1,2], [1,3], [1], [2,3], [2], [3], []]`

~~`powerset :: [a] -> [[a]]`~~

~~`powerset [] = [[]]`~~

~~`powerset (x:xs) = x:(map (\x -> powerset x))`~~



Niloufar Zafari

CMSC 433: Midterm Exam (Spring 2024)

Question 1 (20 points)

Consider the two standard folds in Haskell, `foldl` and `foldr`, whose definitions are given below:

```
foldr :: (a -> b -> b) -> b -> [a] -> b
foldr f b [] = b
foldr f b (x:xs) = f x (foldr f b xs)

foldl :: (b -> a -> b) -> b -> [a] -> b
foldl f b [] = b
foldl f b (x:xs) = foldl f (f b x) xs
```

Now consider the following two very similar but distinct folds over lists of integers:

```
f1 :: [Int] -> Int
f1 = foldl (-) 0 1

f2 :: [Int] -> Int
f2 = foldl (-) 0
```

Write two Dafny programs, one corresponding to each of `f1` and `f2`, but operating on arrays rather than lists. We've given you the method signatures to get you started:

<pre>method f1(a:array<Int>) returns (out:Int) { requires a.length > 0 { out := a[a.length-1]; var i := a.length-2; while (i >= 0) { out := out - a[i]; i := i-1; } } }</pre>	<pre>method f2(a:array<Int>) returns (out:Int) { requires a.length > 0 { out := a[0]; var i: int := 0; while (i < a.length) { out := out - a[i]; i := i+1; } } }</pre>
---	--

Question 2 (20 points)

For each of the Haskell expressions below, write their (most general) Haskell type or "ill-typed" if it contains a type error. The type signatures of all functions below are **provided** in the appendix at the end.

(a) $\backslash x \rightarrow x$

Example answer:

$a \rightarrow a$

(b) $\backslash x y \rightarrow (x,y)$

$a \rightarrow b \rightarrow (a,b)$

(c) $\backslash x y \rightarrow \text{if } x == y \text{ then show } x \text{ else show } (x,y)$

$a \rightarrow a \rightarrow \text{String}$

(d) $\backslash x l \rightarrow x : l ++ l ++ [x]$

$a \rightarrow [a] \rightarrow [a]$

(e) `foldr (const 1)`

$[b] \rightarrow a$

(f) `(1..42)`

$(\text{maybe}, \text{String})$

(g) `(.) "42"`

$(\text{maybe}, \text{String})$

(h) `reverse . reverse`

$[a] \rightarrow [a]$

(i) $\backslash l \rightarrow \text{filter } (< 42) (l ++ l)$

$[Int] \rightarrow [Int]$

(j) `let f x = x in (f 'a', f True)`

type error

(k) `fmap (fmap (+1)) [Nothing]`

$[maybe Int] \rightarrow [maybe Int]$

Question 3 (20 points)

For each of the types below, write a Haskell expression that has that type. Don't write trivial expressions (such as [], Nothing, or undefined) unless there is no other option. You can use any function from the appendix, do syntax, list comprehensions, or any valid Haskell.

(a) $\text{Int} \rightarrow \text{Int}$

Example answers:

$\backslash x \rightarrow x + 1$
 $(+1)$

(b) $\text{Bool} \rightarrow [\text{Bool}]$

$f i$
 $| i == \text{True} = [i]$
 $| \text{otherwise} = []$

(c) $a \rightarrow \text{Maybe } b$

$f a = \text{Nothing}$

(d) $(\text{Int} \rightarrow \text{Char} \rightarrow \text{Bool}) \rightarrow [\text{Int}] \rightarrow [\text{Char}] \rightarrow [\text{Bool}]$

$f a [] [] = []$
 $f a (x:xs) (y:ys) =$

(e) $(a \rightarrow b \rightarrow c) \rightarrow \text{Maybe } a \rightarrow \text{Maybe } b \rightarrow \text{Maybe } c$
 $f a b$
 $| a == b = \text{True} = [a, b]$
 $| \text{otherwise} = [a, x, y]$
 $| \text{cab} == \text{Maybe } c = \text{Just } c$

(f) $(a \rightarrow b) \rightarrow (b \rightarrow \text{Bool}) \rightarrow [a] \rightarrow [b]$
 $f a b (x:xs) = \text{True} = [a, x]$
 $| b(a, x) == \text{True} = [a, x]$

(g) $\text{Maybe } a \rightarrow (a \rightarrow [b]) \rightarrow [(a, b)]$
 $f a b$
 $| \text{Maybe } a = (\text{head } (a, b), \text{Just } a)$
 $| \text{otherwise} = []$

(h) $\text{Eq } a \Rightarrow a \rightarrow [a] \rightarrow [a]$

$f a b = a : b$

(i) $\text{Show } a \Rightarrow [a] \rightarrow \text{IO String}$

$f (x:xs) = \text{Show } x$

(j) $(a, b) \rightarrow (a \rightarrow b \rightarrow c) \rightarrow c$

$f (a, b) c = c a b$

(k) $((a, b) \rightarrow c) \rightarrow c = \text{undefined}$

Question 4 (20 points)

Consider the following Dafny program that inefficiently calculates the cube of a number:

```
method Cube(m:int) returns (y:int)
  requires m > 0
  ensures y == m*m*m
{
  y := 0;
  var x := m * m;
  var z := m;
  while (z > 0)
  {
    z := z - 1;
    y := y + x;
  }
}
```

Fill in the annotations in the following program to show that the Hoare triple given by the outermost pre- and post-conditions is valid. Be completely precise and pedantic in the way you apply Hoare rules—write assertions in *exactly* the form given by the rules rather than just logically equivalent ones. The provided blanks have been constructed so that if you work backwards from the end of the program you should only need to use the rule of consequence so that if you work backwards from the end of the program you should only rely on all the usual rules of arithmetic.

```
{ { m > 0 } } -->
{ { 0 + (m * m * m) == m * m * m } }
  y := 0;
  { { y + (m * m * m) == m * m * m } }
    var x := m * m;
    { { y + (m * m * m) == x * m } }
      var z := m;
      { { y + (z * m * m) == x * m } }
        while (z > 0) {
          { { y + (z * m * m) == x * m && z > 0 } } -->
            { { y + x + (z - 1) * m * m == x * m } }
              z := z - 1;
              { { y + x + (z * m * m) == x * m } }
                y := y + x;
                { { y + (z * m * m) == x * m } }
              }
            { { y + (z * m * m) == x * m && (z <= 0) && (z > 0) } } -->
          { { y == m * m * m } }
```

Question 5 (20 points)

Implement the following Haskell functions:

- (a) Implement a function `weave` that given two lists with elements of the same type, returns a list with elements alternating between the two lists. For example:

```
weave [1,2,3] [4,5,6] = [1,4,2,5,3,6]
                                1 1 1
                                1, 4, 2, 5
```

You can assume that the lists have the same length.

```
weave [] = []
```

```
weave (x:xs) (y:ys) = [x,y] ++ weave xs ys
```

- (b) Implement a function `powerSet`, that given a list of elements of some type, computes the list of all of its sublists, including the empty list and the original list itself, in any order. For example:

```
powerSet [1,2,3] = [ [], [1], [2], [3], [1,2], [1,3], [2,3], [1,2,3] ]
```

```
powerSet [] = [ [] ]
```

```
powerSet (x:xs) = [x:] ++ powerSet xs
```


Diego Lopez

CMSC 433: Midterm Exam (Spring 2024)

Question 1 (20 points)

Consider the two standard folds in Haskell, `foldl` and `foldr`, whose definitions are given below:

```
foldr :: (a -> b -> b) -> b -> [a] -> b
foldr f b [] = b
foldr f b (x:xs) = f x (foldr f b xs)

foldl :: (b -> a -> b) -> b -> [a] -> b
foldl f b [] = b
foldl f b (x:xs) = foldl f (f b x) xs
```

Now consider the following two very similar but distinct folds over lists of integers:

```
f1 :: [Int] -> Int
f1 = foldr (+) 0

f2 :: [Int] -> Int
f2 = foldl (-) 0
```

Write two Dafny programs, one corresponding to each of `f1` and `f2`, but operating on arrays rather than lists. We've given you the method signatures to get you started:

<pre>method f1(a:array<int>) returns (out:int) { var i := 0; out := 0; while i < a.Length { out := out + a[i]; i := i + 1; } return out; }</pre>	<pre>method f2(a:array<int>) returns (out:int) { var i := a.Length - 1; out := 0; while i > 0 { out := out - a[i]; i := i - 1; } }</pre>
---	---

Question 2 (20 points)

For each of the Haskell expressions below, write their (most general) Haskell type or "ill-typed" if it contains a type error. The type signatures of all functions below are provided in the appendix at the end.

(a) $\backslash x \rightarrow x$

Example answer:

$a \rightarrow a$

(b) $\backslash x\ y \rightarrow (x,y)$

$a \rightarrow b \rightarrow (a,b)$

(c) $\backslash x\ y \rightarrow$ if $x == y$ then show x else show (x,y)

$a \rightarrow b \rightarrow \text{String}$

(d) $\backslash x\ l \rightarrow x : l ++ l ++ [x]$

$a \rightarrow [a] \rightarrow [a]$

(e) $\text{foldr} (\text{const } 1)$

$(a \rightarrow b \rightarrow b) \rightarrow$

(f) ("42")

ill-typed

(g) ("42")

$a \rightarrow (a,b)$

(h) $\text{reverse} . \text{reverse}$

$[a] \rightarrow [a]$

(i) $\backslash l \rightarrow \text{filter} (< 42) (l ++ l)$

$[a] \rightarrow \text{Maybe } [a]$

(j) $\text{let } f\ x = x \text{ in } (f\ 'a', f\ \text{True})$

$a \rightarrow a \rightarrow a \rightarrow \text{Bool}$

(k) $\text{fmap} (\text{fmap } (+1)) [\text{Nothing}]$

ill-typed

Question 3 (20 points)

For each of the types below, write a Haskell expression that has that type. Don't write trivial expressions (such as [], Nothing, or undefined) unless there is no other option. You can use any function from the appendix, do syntax, list comprehensions, or any valid Haskell.

(a) `Int -> Int`

Example answers:

`\x -> x + 1`
`(+1)`

(b) `Bool -> [Bool]`

True: []

(c) `a -> Maybe b`

undefined

(d) `(Int -> Char -> Bool) -> [Int] -> [Char] -> [Bool]`

zipWith [0 'a' True

(e) `(a -> b -> c) -> Maybe a -> Maybe b -> Maybe c`

Maybe ->

(f) `(a -> b) -> (b -> Bool) -> [a] -> [b]`

Maybe

(g) `Maybe a -> (a -> [b]) -> [(a,b)]`

*!**

(h) `Eq a => a -> [a] -> [a]`

Maybe

(i) `Show a => [a] -> IO String`

undefined

(j) `(a,b) -> (a -> b -> c) -> c`

undefined

(k) `((a,b) -> c) -> c`

undefined

Question 4 (20 points)

Consider the following Dafny program that inefficiently calculates the cube of a number:

```
method Cube(m:int) returns (y:int)
  requires m > 0
  ensures y == m*m*m
{
  y := 0;
  var x := m * m;
  var z := m;
  while (z > 0)
  {
    z := z - 1;
    y := y + x;
  }
}
```

Fill in the annotations in the following program to show that the Hoare triple given by the outermost pre- and post- conditions is valid. Be completely precise and pedantic in the way you apply Hoare rules—write assertions in *exactly* the form given by the rules rather than just logically equivalent ones. The provided blanks have been constructed so that if you work backwards from the end of the program you should only need to use the rule of consequence so that if you work backwards from the end of the program you should only rely on all the usual rules of arithmetic.

```
{ { m > 0 } } ->>
{ { at m == m q m > 0 } }
  y := 0;
  { { y + m == m 1 m > 0 } }
    var x := m * m;
    { { (y + m) * (y + m) == x 1 y + m == m m m > 0 } }
      var z := m;
      { { (y + z) * (y + z) == x 0 y + z == z 0 z > 0 } }
        while (z > 0) {
          { { (y + z) * (y + z) == x 0 y + z == z 0 z > 0 } } ->>
            { { (y + z) * (y + z) == x 0 (z > 0) } }
              z := z - 1;
              { { (y + z - 1) * (y + z - 1) == x 1 y + z - 1 == m 0 } } (z > 0)
                y := y + x;
                { { x == (y + z - 1) * (y + z - 1) + m 1 y + z - 1 == m 1 (z > 0) } }
                  { { x := (y + z - 1) * (y + z - 1) + m 0 (y + z - 1) * (y + z - 1) } } ->>
                    { { y == m * m * m } }
```

! (z > 0)

! ((y + z - 1) * (y + z - 1) == x)

$$\begin{aligned}
 y &= y + x \\
 3 &= 3 + 3 \\
 3 &= 3 \cdot 3 - 9 \quad 27 \\
 (y + z - 1) * (y + z - 1) + m &= 2 \cdot 3 \cdot 3 + 3 \cdot 3 = 18 \\
 2 \cdot 2 + (2 \cdot 2) &= 4 + 4 = 8
 \end{aligned}$$

Question 5 (20 points)

Implement the following Haskell functions:

- (a) Implement a function `weave` that given two lists with elements of the same type, returns a list with elements alternating between the two lists. For example:

`weave [1,2,3] [4,5,6] = [1,4,2,5,3,6]`

`2: pattern [x:]`

You can assume that the lists have the same length.

`weave :: [a] -> [a] -> [a]`
`weave [] [] = []`
`weave (x:xs) (y:ys) = (x:y: weave xs ys)`

- (b) Implement a function `powerSet`, that given a list of elements of some type, computes the list of all of its sublists, including the empty list and the original list itself, in any order. For example:

`powerSet [1,2,3] = [[1,2,3], [1,2], [1,3], [1], [2,3], [2], [3], []]`

`powerSet :: [a] -> [[a]]`
`powerSet [] = [[]]`
`powerSet (x:xs) = (x:acc -> acc ++ [x] ++ powerSet xs)`

Andrew Lee
117891102

CMSC 433: Midterm Exam (Spring 2024)

Question 1 (20 points)

Consider the two standard folds in Haskell, `foldl` and `foldr`, whose definitions are given below:

```
foldr :: (a -> b -> b) -> b -> [a] -> b
foldr f b [] = b
foldr f b (x:xs) = f x (foldr f b xs)

foldl :: (b -> a -> b) -> b -> [a] -> b
foldl f b [] = b
foldl f b (x:xs) = foldl f (f b x) xs
```

Now consider the following two very similar but distinct folds over lists of integers:

```
f1 :: [Int] -> Int
f1 = foldr (-) 0

f2 :: [Int] -> Int
f2 = foldl (-) 0
```

Write two Dafny programs, one corresponding to each of `f1` and `f2`, but operating on arrays rather than lists. We've given you the method signatures to get you started:

<pre>method f1(a:array<int>) returns (out:int) { var i := a.length - 1; out := 0; while i >= 0 { out := out - a[i]; i := i - 1; } }</pre>	<pre>method f2(a:array<int>) returns (out:int) { var i := 0; out := 0; while i < a.length { out := out - a[i]; i := i + 1; } }</pre>
--	---

Question 2 (20 points)

For each of the Haskell expressions below, write their (most general) Haskell type or "ill-typed" if it contains a type error. The type signatures of all functions below are provided in the appendix at the end.

(a) `\x -> x`

Example answer:

`a -> a`

(b) `\x y -> (x,y)`

`a -> b -> (a,b)`

(c) `\x y -> if x == y then show x else show (x,y)`

`a -> a -> String`

(d) `\x l -> x : l ++ l ++ l ++ [x]`

`a -> [a] -> [a]`

(e) `foldr (const 1)`

`!l-typed`

(f) `(:"42")`

(g) `(. "42"`

(h) `reverse . reverse`

`[a] -> [a]`

(i) `\l -> filter (< 42) (l ++ l)`

`[Int] -> [Int]`

(j) `let f x = x in (f 'a', f True)`

(k) `fmap (fmap (+1)) [Nothing]`

Question 3 (20 points)

For each of the types below, write a Haskell expression that has that type. Don't write trivial expressions (such as [], Nothing, or undefined) unless there is no other option. You can use any function from the appendix, do syntax, list comprehensions, or any valid Haskell.

(a) `Int -> Int`

Example answers:

`\x -> x + 1`
`(+1)`

(b) `Bool -> [Bool]`

`\x -> repeat (x == true)`

(c) `a -> Maybe b`

`undefined`

(d) `(Int -> Char -> Bool) -> [Int] -> [Char] -> [Bool]`

`zipWith`

(e) `(a -> b -> c) -> Maybe a -> Maybe b -> Maybe c`

`\f (Maybe a) (Maybe b) -> f a b`

(f) `(a -> b) -> (b -> Bool) -> [a] -> [b]`

`\f g (x:xs) -> let b = f x in if g b then [b] else []`

(g) `Maybe a -> (a -> [b]) -> [(a,b)]`

`\f (Maybe a) -> [f a]`

(h) `Eq a => a -> [a] -> [a]`

`\x xs -> delete x xs`

(i) `Show a => [a] -> IO String`

(j) `(a,b) -> (a -> b -> c) -> c`

`\(a,b) f -> f a b`

(k) `((a,b) -> c) -> c`

Question 4 (20 points)

Consider the following Dafny program that inefficiently calculates the cube of a number:

```
method Cube(m:int) returns (y:int)
  requires m > 0
  ensures y == m*m*m
{
  y := 0;
  var x := m * m;
  var z := m;
  while (z > 0)
  {
    z := z - 1;
    y := y + x;
  }
}
```

$m = 3$
 $y = 0, x = 9, z = 3$
 $y = 9, x = 9, z = 2$
 $y = 18, x = 9, z = 1$
 $y = 27, x = 9, z = 0$
 $\frac{y}{x} = \frac{m-z}{x}$
 $y = x(m-z)$

Fill in the annotations in the following program to show that the Hoare triple given by the outermost pre- and post- conditions is valid. Be completely precise and pedantic in the way you apply Hoare rules---write assertions in *exactly* the form given by the rules rather than just logically equivalent ones. The provided blanks have been constructed so that if you work backwards from the end of the program you should only need to use the rule of consequence in the places indicated with $\rightarrow\rangle$. These implication steps can (silently) rely on all the usual rules of arithmetic.

```
{ { m > 0 } }  $\rightarrow\rangle\rangle$ 
{ { 0 == 0 } }  $\wedge$  { m > 0 }
  y := 0;
  { { y == m * m * 0 } }  $\wedge$  { m > 0 }
    var x := m * m;
    { { y == x * (m - m) } }  $\wedge$  { m > 0 }
      var z := m;
      { { y == x * (m - z) } }  $\wedge$  { z >= 0 }
        while (z > 0) {
          { { y == x * (m - z) } }  $\wedge$  { z >= 0 }  $\wedge$  { z > 0 }
            { { y + x == x * (m - z + 1) } }  $\wedge$  { z - 1 >= 0 }
              z := z - 1;
              { { y + x == x * (m - z) } }  $\wedge$  { z >= 0 }
                y := y + x;
                { { y == x * (m - z) } }  $\wedge$  { z >= 0 }
                  }
                { { y == x * (m - z) } }  $\wedge$  { z >= 0 }  $\wedge$  { ! (z > 0) }
                  { { y == m * m * m } }
                    }  $\rightarrow\rangle\rangle$ 
```


Question 5 (20 points)

Implement the following Haskell functions:

- (a) Implement a function `weave` that given two lists with elements of the same type, returns a list with elements alternating between the two lists. For example:

`weave [1,2,3] [4,5,6] = [1,4,2,5,3,6]`

You can assume that the lists have the same length.

`weave [] [] = []`

`weave (x:xs) (y:ys) = x:y:(weave xs ys)`

- (b) Implement a function `powerSet`, that given a list of elements of some type, computes the list of all of its sublists, including the empty list and the original list itself, in any order. For example:

`powerSet [1,2,3] = [[1,2,3], [1,2], [1,3], [1], [2,3], [2], [3], []]`

`powerSet xs = foldr (\x acc -> [foldr (\x' acc' -> (x':x') : acc') [] acc] ++ acc) [[]] xs`

Mathew Cimala

117168532

CMSC 433: Midterm Exam (Spring 2024)

Question 1 (20 points)

Consider the two standard folds in Haskell, `foldl` and `foldr`, whose definitions are given below:

```
foldr :: (a -> b -> b) -> b -> [a] -> b
foldr f b [] = b
foldr f b (x:xs) = f x (foldr f b xs)

foldl :: (b -> a -> b) -> b -> [a] -> b
foldl f b [] = b
foldl f b (x:xs) = foldl f (f b x) xs
```

Now consider the following two very similar but distinct folds over lists of integers:

```
f1 :: [Int] -> Int
f1 = foldr (-) 0

f2 :: [Int] -> Int
f2 = foldl (-) 0
```

Write two Dafny programs, one corresponding to each of `f1` and `f2`, but operating on arrays rather than lists. We've given you the method signatures to get you started:

<pre>method f1(a:array<int>) returns (out:int) { out := 0; var i: int := 0; //ensure we start from left while (i < a.Length) { out := out - a[i]; i := i + 1; } return out; }</pre>	<pre>method f2(a:array<int>) returns (out:int) { out := 0; var i: int := a.Length - 1; //ensure we start from right while (i != 0) { out := out - a[i]; i := i - 1; } return out; }</pre>
--	---

* I accidentally switched these two problems around - consider the code switched.

Matthew Címerola
117168532

Question 2 (20 points)

For each of the Haskell expressions below, write their (most general) Haskell type or "ill-typed" if it contains a type error. The type signatures of all functions below are provided in the appendix at the end.

(a) $\backslash x \rightarrow x$

Example answer:

$a \rightarrow a$

(b) $\backslash x y \rightarrow (x, y)$

$a \rightarrow b \rightarrow (a, b)$

(c) $\backslash x y \rightarrow \text{if } x == y \text{ then show } x \text{ else show } (x, y)$

ill-typed

(d) $\backslash x l \rightarrow x : l ++ l ++ [x]$

$a \rightarrow [a] \rightarrow [a]$

(e) $\text{foldr } (\text{const } 1)$

~~unadapable~~ ill-typed

(f) $(\cdot, "42")$

~~$((a \rightarrow b \rightarrow (a, b)), c)$~~ ill-typed

(g) $(\cdot, "42")$

$(a \rightarrow b \rightarrow (a, b)) \rightarrow a \rightarrow (a, b)$

(h) $\text{reverse} \cdot \text{reverse}$

$[a] \rightarrow [a] \rightarrow [a]$

(i) $\backslash l \rightarrow \text{filter } (< 42) (l ++ l)$

$[a] \rightarrow [a]$

(j) $\text{let } f \ x = x \text{ in } (f 'a', f \text{ True})$

ill-typed

(k) $\text{fmap } (\text{fmap } (+1)) [\text{Nothing}]$

ill-typed

Matthew
Cmelo

Question 3 (20 points)

For each of the types below, write a Haskell expression that has that type. Don't write trivial expressions (such as `[]`, `Nothing`, or `undefined`) unless there is no other option. You can use any function from the `appendix`, `do` syntax, list comprehensions, or any valid Haskell.

(a) `Int -> Int`

Example answers:

`\x -> x + 1`

`(+1)`

(b) `Bool -> [Bool]`

$\forall x \rightarrow \text{if } x, \text{ then } [True] \text{ else } [False]$

(c) `a -> Maybe b`

$\sim a \rightarrow Nothing$

(d) `(Int -> Char -> Bool) -> [Int] -> [Char] -> [Bool]`

not ill typed

(e) `(a -> b -> c) -> Maybe a -> Maybe b -> Maybe c`

not ill typed

(f) `(a -> b) -> (b -> Bool) -> [a] -> [b]`

not ill typed

(g) `Maybe a -> (a -> [b]) -> [(a,b)]`

not ill typed

(h) `Eq a => a -> [a] -> [a]`

$\backslash a \text{ (heads:tails)} \rightarrow \text{if } a == \text{heads then } [\text{heads:tails}] \text{ else } [\text{tails}]$

(i) `Show a => [a] -> IO String`

$\backslash x \rightarrow \text{if length (reverse x)} == 1 \text{ then putStr "ugh" else putStr "k"}$

(j) `(a,b) -> (a -> b -> c) -> c`

$\cdot f00 \text{ (a,b) } f =$

(k) `((a,b) -> c) -> c`

~~not ill typed~~

$\backslash c \rightarrow (\backslash a, b) \rightarrow c$

27-21 27-18
 $y = x \cdot m - (x \cdot z)$

(miller)
 Mathieu

Question 4 (20 points)

Consider the following Dafny program that inefficiently calculates the cube of a number:

```
method Cube(m:int) returns (y:int)
  requires m > 0
  ensures y == m*m*m
{
  y := 0;
  var x := m * m;
  var z := m;
  while (z > 0)
  {
    z := z - 1;
    y := y + x;
  }
}
```

$x=9 \quad z=3 \quad y=0$
 $x=9 \quad z=2 \quad y=9$
 $x=9 \quad z=1 \quad y=18$
 $x=9 \quad z=0 \quad y=27 \quad y = z \cdot m + x$
 $y = x$

$0 \rightarrow x = y, z = 2, y = 0$
 $1 \rightarrow x = y, z = 1, y = 4$
 $2 \rightarrow x = y, z = 0, y = 8$
 $x = z^y$
 $x = m \cdot m$
 $x + z$
 $y = x - (m \cdot z)$
 $y = x \cdot m - (x \cdot z)$
 $y = 8 - 2$
 $y = m \cdot m - (2 \cdot 2)$
 $y = x - (m \cdot z)$
 $y = x \cdot m - (x \cdot z)$
 $y = x - (m \cdot z)$

Fill in the annotations in the following program to show that the Hoare triple given by the outermost pre- and post- conditions is valid. Be completely precise and pedantic in the way you apply Hoare rules— write assertions in *exactly* the form given by the rules rather than just logically equivalent ones. The provided blanks have been constructed so that if you work backwards from the end of the program you should only need to use the rule of consequence in the places indicated with $\rightarrow$. These implication steps can (silently) rely on all the usual rules of arithmetic.

```
{ { m > 0 } }  $\rightarrow$ 
{ { m > 0 } }  $\wedge$   $0 = m \cdot m \cdot m - (m \cdot m \cdot m)$ 
  y := 0;
  { { m > 0 } }  $\wedge$   $y = m \cdot m \cdot m - (m \cdot m \cdot m)$ 
    var x := m * m;
    { { m > 0 } }  $\wedge$   $y = x \cdot m - (x \cdot m)$ 
      var z := m;
      { { z > 0 } }  $\wedge$   $y = x \cdot m - (x \cdot z)$ 
        while (z > 0) {
          { { z > 0 } }  $\wedge$   $y = x \cdot m - (x \cdot z) \wedge (z > 0)$ 
            { { z-1 > 0 } }  $\wedge$   $y + x = x \cdot m - (x \cdot (z-1))$ 
              z := z - 1;
              { { z > 0 } }  $\wedge$   $y + x = x \cdot m - (x \cdot z)$ 
                y := y + x;
                { { z > 0 } }  $\wedge$   $y = x \cdot m - (x \cdot z)$ 
              }
            { { z > 0 } }  $\wedge$   $y = x \cdot m - (x \cdot z) \wedge (z > 0)$ 
          }
        }
      { { y = m * m * m } }
```

$inv = z > 0$ AND $y = x \cdot m - (x \cdot z)$

Mathias Cimeola

Question 5 (20 points)

Implement the following Haskell functions:

- (a) Implement a function `weave` that given two lists with elements of the same type, returns a list with elements alternating between the two lists. For example:

`weave [1,2,3] [4,5,6] = [1,4,2,5,3,6]`

1,4,2,5,3,6

You can assume that the lists have the same length.

```
weave :: [a] -> [a] -> [a]
weave [] [] = [] // base case
weave (h1:t1) (h2:t2) = h1:h2:(weave t1 t2)
// b/c they are the same length, don't have to
// worry about other cases
```

- (b) Implement a function `powerSet`, that given a list of elements of some type, computes the list of all of its sublists, including the empty list and the original list itself, in any order. For example:

`powerSet [1,2,3] = [[], [1], [2], [3], [1,2], [1,3], [2,3], [1,2,3], []]`

```
powerSet :: [a] -> [[a]]
powerSet (head:tail) = foldr (\x acc -> x:acc) [] head:
powerSet [] = [] // powerSet tail

// the idea is to fold on every sublist,
// and recursively cons this sublist with a
// recursive call
```


Gordon Moss, 117224238

CMSC 433: Midterm Exam (Spring 2024)

Question 1 (20 points)

Consider the two standard folds in Haskell, `foldl` and `foldr`, whose definitions are given below:

```
foldr :: (a -> b -> b) -> b -> [a] -> b
foldr f b [] = b
foldr f b (x:xs) = f x (foldr f b xs)

foldl :: (b -> a -> b) -> b -> [a] -> b
foldl f b [] = b
foldl f b (x:xs) = foldl f (f b x) xs
```

Now consider the following two very similar but distinct folds over lists of integers:

```
f1 :: [Int] -> Int
f1 = foldr (-) 0

f2 :: [Int] -> Int
f2 = foldl (-) 0
```

Write two Dafny programs, one corresponding to each of `f1` and `f2`, but operating on arrays rather than lists. We've given you the method signatures to get you started:

<pre>method f1(a:array<int>) returns (out:int) { var i := a.Length - 1; out := 0; while (i >= 0) { out := out - a[i]; i := i - 1; } }</pre>	<pre>method f2(a:array<int>) returns (out:int) { var i := 0; out := 0; while (i < a.Length) { out := out - a[i]; i := i + 1; } }</pre>
--	---

Question 2 (20 points)

For each of the Haskell expressions below, write their (most general) Haskell type or "ill-typed" if it contains a type error. The type signatures of all functions below are provided in the appendix at the end.

(a) $\backslash x \rightarrow x$

Example answer:

$a \rightarrow a$

(b) $\backslash x\ y \rightarrow (x,y)$

$a \rightarrow b \rightarrow (a,b)$

(c) $\backslash x\ y \rightarrow \text{if } x == y \text{ then show } x \text{ else show } (x,y)$

$a \rightarrow a \rightarrow \text{String}$

(d) $\backslash x\ l \rightarrow x : l ++ l ++ [x]$

$a \rightarrow [a] \rightarrow [a]$

→

(e) foldr (const 1)

$b \rightarrow [a] \rightarrow b$

(f) $(\cdot "42")$

$b \rightarrow (\text{String}, b)$

(g) $(\cdot "42")$

$b \rightarrow (\text{String}, b)$

(h) $\text{reverse} . \text{reverse}$

ill-typed

(i) $\backslash l \rightarrow \text{filter } (< 42) (l ++ l)$

$[a] \rightarrow [a]$

→

(j) $\text{let } f\ x = x \text{ in } (f\ 'a', f\ \text{True})$

$a \rightarrow (b, b)$

→

(k) $\text{fmap (fmap (+1)) [Nothing]}$

$[\text{Maybe } a]$

Question 3 (20 points)

For each of the types below, write a Haskell expression that has that type. Don't write trivial expressions (such as [], Nothing, or undefined) unless there is no other option. You can use any function from the appendix, do syntax, list comprehensions, or any valid Haskell.

(a) $\text{Int} \rightarrow \text{Int}$

Example answers:

$\backslash x \rightarrow x + 1$
 $(+1)$

(b) $\text{Bool} \rightarrow [\text{Bool}]$

$\backslash b \rightarrow \text{if } b \text{ then } [b] \text{ else } [b]$

(c) $a \rightarrow \text{Maybe } b$

(d) $(\text{Int} \rightarrow \text{Char} \rightarrow \text{Bool}) \rightarrow [\text{Int}] \rightarrow [\text{Char}] \rightarrow [\text{Bool}]$

(e) $(a \rightarrow b \rightarrow c) \rightarrow \text{Maybe } a \rightarrow \text{Maybe } b \rightarrow \text{Maybe } c$

$\backslash a \ b \ c$

(f) $(a \rightarrow b) \rightarrow (b \rightarrow \text{Bool}) \rightarrow [a] \rightarrow [b]$

$\backslash f \ g \ x \rightarrow \text{if } (g \ x) \text{ then } [v] \text{ else } [v] \text{ where}$

(g) $\text{Maybe } a \rightarrow (a \rightarrow [b]) \rightarrow [(a, b)]$
 $\backslash v \ f \rightarrow \text{if } (isJust \ v) \text{ then } [f \ (head \ x)]$
 $v = f \ (head \ x)$

(h) $\text{Eq } a \Rightarrow a \rightarrow [a] \rightarrow [a]$

$\backslash a \ b \rightarrow \text{if } a == (head \ b) \text{ then } b \text{ else } b$

(i) Show $a \Rightarrow [a] \rightarrow \text{IO String}$

$\backslash v \rightarrow \text{putStrLn } (\text{show } (head \ v))$

(j) $(a, b) \rightarrow (a \rightarrow b \rightarrow c) \rightarrow c$

$\backslash (x, y) \ f \rightarrow f \ x \ y$

(k) $((a, b) \rightarrow c) \rightarrow c$

Question 4 (20 points)

Consider the following Dafny program that inefficiently calculates the cube of a number:

```
method Cube(m:int) returns (y:int)
  requires m > 0
  ensures y == m*m*m
{
  y := 0;
  var x := m * m;
  var z := m;
  while (z > 0)
  {
    z := z - 1;
    y := y + x;
  }
}
```

Fill in the annotations in the following program to show that the Hoare triple given by the outermost pre- and post-conditions is valid. Be completely precise and pedantic in the way you apply Hoare rules—write assertions in *exactly* the form given by the rules rather than just logically equivalent ones. The provided blanks have been constructed so that if you work backwards from the end of the program you should only need to use the rule of consequence in the places indicated with $\rightarrow>$. These implication steps can (silently) rely on all the usual rules of arithmetic.

```
{ { m > 0 } }  $\rightarrow>$ 
{ { 0 = m.m.(m-m)  $\wedge$  m.m = m.m  $\wedge$  m  $\geq$  0 } }
  y := 0;
{ { y = m.m.(m-m)  $\wedge$  m.m = m.m  $\wedge$  m  $\geq$  0 } }
  var x := m * m;
{ { y = m.m.(m-m)  $\wedge$  x = m.m  $\wedge$  m  $\neq$  0 } }
  var z := m;
{ { y = m.m.(m-z)  $\wedge$  z  $\geq$  0  $\wedge$  x = m.m } }
  while (z > 0) {
    { { y = m.m.(m-z)  $\wedge$  z  $\geq$  0  $\wedge$  z > 0  $\wedge$  x = m.m } }  $\rightarrow>$ 
    { { y + x = m.m.(m-z+1)  $\wedge$  z  $\geq$  0 } }
    z := z - 1;
    { { y + x = m.m.(m-z)  $\wedge$  z  $\geq$  0 } }
    y := y + x;
    { { y = m.m.(m-z)  $\wedge$  z  $\geq$  0 } }
  }
{ { y = m.m.(m-z)  $\wedge$  !(z > 0)  $\wedge$  z  $\geq$  0 } }  $\rightarrow>$ 
{ { y = m * m * m } }
```

Question 5 (20 points)

Implement the following Haskell functions:

- (a) Implement a function `weave` that given two lists with elements of the same type, returns a list with elements alternating between the two lists. For example:

`weave [1,2,3] [4,5,6] = [1,4,2,5,3,6]`

You can assume that the lists have the same length.

`weave a b = foldr (\e1 e2 acc → e1:e2:acc) ,`

`[], (zip a b)`

- (b) Implement a function `powerSet`, that given a list of elements of some type, computes the list of all of its sublists, including the empty list and the original list itself, in any order. For example:

`powerSet [1,2,3] = [[1,2,3], [1,2], [1,3], [1], [2,3], [2], [3], []]`

`powerSetAux a =`

`case a of`

`[] → []`

`| x:xs → a: (powerSetAux xs)`

`powerSet a =`

`case a of`

`[] → [[]]`

`| x:xs → (powerSetAux (reverse a)) ++ (powerSet xs)`

Thomas Wolcott

CMSC 433: Midterm Exam (Spring 2024)

Question 1 (20 points)

Consider the two standard folds in Haskell, `foldl` and `foldr`, whose definitions are given below:

```
foldr :: (a -> b -> b) -> b -> [a] -> b
foldr f b [] = b
foldr f b (x:xs) = f x (foldr f b xs)

foldl :: (b -> a -> b) -> b -> [a] -> b
foldl f b [] = b
foldl f b (x:xs) = foldl f (f b x) xs
```

Now consider the following two very similar but distinct folds over lists of integers:

```
f1 :: [Int] -> Int
f1 = foldr (-) 0

f2 :: [Int] -> Int
f2 = foldl (-) 0

f1 list = foldr (\n acc->n-acc) 0 list
f2 list = foldl (\acc n->acc-n) 0 list
```

Write two Dafny programs, one corresponding to each of `f1` and `f2`, but operating on arrays rather than lists. We've given you the method signatures to get you started:

```
method f1(a:array<int>) returns (out:int) {
  out := 0
  var i := a.Length-1
  while i >= 0 {
    out := a[i]-out;
    i := i-1;
  }
}

method f2(a:array<int>) returns (out:int) {
  out := 0
  var i := 0
  while i < a.Length {
    out := out-a[i];
    i := i+1;
  }
}
```

}

}

Question 2 (20 points)

For each of the Haskell expressions below, write their (most general) Haskell type or "ill-typed" if it contains a type error. The type signatures of all functions below are provided in the appendix at the end.

(a) $\backslash x \rightarrow x$

Example answer:

$a \rightarrow a$

(b) $\backslash x y \rightarrow (x, y)$

$a \rightarrow b \rightarrow (a, b)$

(c) $\backslash x y \rightarrow \text{if } x == y \text{ then show } x \text{ else show } (x, y)$

$a \rightarrow a \rightarrow \text{String}$

(d) $\backslash x l \rightarrow x : l ++ l ++ l ++ [x]$

ill-typed

(e) $\text{foldr} (\text{const } 1)$

ill-typed

(f) $(\cdot, "42")$

$a \rightarrow (a, \text{String})$

(g) $(\cdot, "42")$

ill-typed

(h) $\text{reverse} . \text{reverse}$

$[a] \rightarrow [a]$

(i) $\backslash l \rightarrow \text{filter } (< 42) (l ++ l)$ $\leftarrow "l"$ not "1"

$[int] \Rightarrow [int]$

(j) $\text{let } f x = x \text{ in } (f 'a', f \text{True})$

$(\text{Char}, \text{bool})$

(k) $\text{fmap} (\text{fmap } (+1)) [\text{Nothing}]$

$[\text{Maybe } int]$

const!
 $b \rightarrow int$

$\text{fmap } (+1)$
 $f : int \rightarrow f : int$
 $\text{fmap } (\text{fmap } (+1))$
 $f : int \rightarrow f : int$

Question 3 (20 points)

For each of the types below, write a Haskell expression that has that type. Don't write trivial expressions (such as [], Nothing, or undefined) unless there is no other option. You can use any function from the appendix, do syntax, list comprehensions, or any valid Haskell.

- (a) `Int -> Int`
Example answers:
`\x -> x + 1`
`(+1)`
- (b) `Bool -> [Bool]`
`\x -> [x && True]`
- (c) `a -> Maybe b`
`\x -> Nothing`
- (d) `(Int -> Char -> Bool) -> [Int] -> [Char] -> [Bool]`
`\f i list c list -> zipWith (\i c -> (f (i+1) c) && (c /= '0')) i_list c_list`
- (e) `(a -> b -> c) -> Maybe a -> Maybe b -> Maybe c`
`\f a m b m -> (a m) >>= (\a -> (b m) >>= (\b -> f a b)))`
- (f) `(a -> b) -> (b -> Bool) -> [a] -> [b]`
`\f1 f2 list -> filter f2 $ map f1 list`
- (g) `Maybe a -> (a -> [b]) -> [(a,b)]`
`\a m f -> foldr (\a abs -> map (\b -> (a,b)) (f a)) [] (maybeToList a m)`
- (h) `Eq a => a -> [a] -> [a]`
`\x -> filter (== x)`
- (i) `Show a => [a] -> IO String`
`xs -> do (show xs)`
`putStrLn (show xs)`
- (j) `(a,b) -> (a -> b -> c) -> c`
`\(x,y) f -> f x y`
- (k) `((a,b) -> c) -> c`
`\f -> undefined`

`[a] -> (a -> [b]) -> [(a,b)]`
`map (\a -> map (\b -> (a,b)) (f a)) as`

Question 4 (20 points)

Consider the following Dafny program that inefficiently calculates the cube of a number:

```
method Cube(m:int) returns (y:int)
  requires m > 0
  ensures y == m*m*m
{
  y := 0;
  var x := m * m;
  var z := m;
  while (z > 0)
  {
    z := z - 1;
    y := y + x;
  }
}
```

Fill in the annotations in the following program to show that the Hoare triple given by the outermost pre- and post- conditions is valid. Be completely precise and pedantic in the way you apply Hoare rules---write assertions in *exactly* the form given by the rules rather than just logically equivalent ones. The provided blanks have been constructed so that if you work backwards from the end of the program you should only need to use the rule of consequence in the places indicated with $\rightarrow>$. These implication steps can (silently) rely on all the usual rules of arithmetic.

```
{ { m > 0 } }  $\rightarrow>$ 
{ {  $m \geq 0 \wedge (m-m) * m * m = 0$  } }
  y := 0;
{ {  $m \geq 0 \wedge (m-m) * m * m = y$  } }
  var x := m * m;
{ {  $m \geq 0 \wedge (m-m) * x = y$  } }
  var z := m;
{ {  $z \geq 0 \wedge (m-z) * x = y$  } }
  while (z > 0) {
    { {  $z \geq 0 \wedge (m-z) * x = y$  } }  $\wedge$   $x = y$  }  $\wedge$   $(z > 0)$ 
    { {  $z \geq 0 \wedge (m-(z-1)) * x = y + x$  } }
      z := z - 1;
    { {  $z \geq 0 \wedge (m-z) * x = y + x$  } }
      y := y + x;
    { {  $z \geq 0 \wedge (m-z) * x = y$  } }
  }
{ {  $z \geq 0 \wedge (m-z) * x = y$  } }  $\wedge$   $(z > 0)$ 
{ { y = m * m * m } }  $\rightarrow>$ 
```

Question 5 (20 points)

Implement the following Haskell functions:

- (a) Implement a function `weave` that given two lists with elements of the same type, returns a list with elements alternating between the two lists. For example:

`weave [1,2,3] [4,5,6] = [1,4,2,5,3,6]`

You can assume that the lists have the same length.

```
weave [] [] = []
weave [x] [] = [x]
weave [] [y] = [y]
weave (x:xs) (y:ys) = if (length xs) >= (length ys) then
  x : (weave xs (y:ys)) else y : (weave (x:xs) ys)
```

- (b) Implement a function `powerset`, that given a list of elements of some type, computes the list of all of its sublists, including the empty list and the original list itself, in any order. For example:

`powerset [1,2,3] = [[], [1,2,3], [1,2], [1,3], [1], [2,3], [2], [3], []]`

```
powerset [] = [[]]
powerset (x:xs) = (map (x:) (powerset xs)) ++ (powerset xs)
```

you can use recursion

```
[1] x=1
xs=[]
p xs = [[]]
      [x] ++ [1]

[1,2] x=1
      xs=[2]
      p xs = [[2], [1]]
            [1,2], [1] ++ [1,2], [2]
```


evlyn zhao UID 117901054

CMSC 433: Midterm Exam (Spring 2024)

Question 1 (20 points)

Consider the two standard folds in Haskell, `foldl` and `foldr`, whose definitions are given below:

```
foldr :: (a -> b -> b) -> b -> [a] -> b
foldr f b [] = b
foldr f b (x:xs) = f x (foldr f b xs)

foldl :: (b -> a -> b) -> b -> [a] -> b
foldl f b [] = b
foldl f b (x:xs) = foldl f (f b x) xs
```

Now consider the following two very similar but distinct folds over lists of integers:

```
f1 :: [Int] -> Int
f1 = foldr (-) 0

f2 :: [Int] -> Int
f2 = foldl (-) 0
```

Write two Dafny programs, one corresponding to each of `f1` and `f2`, but operating on arrays rather than lists. We've given you the method signatures to get you started:

<pre>method f1(a:array<int>) returns (out:int) { out := 0; var i: int := 0; while (i < a.Length) invariant 0 <= i <= a.Length { out := out - a[i]; i := i + 1; } }</pre>	<pre>method f2(a:array<int>) returns (out:int) { out := 0; var i: int := a.Length - 1; while (i >= 0) invariant 0 <= i <= a.Length - 1 { out := out - a[i]; i := i - 1; } }</pre>
---	--

Question 2 (20 points)

For each of the Haskell expressions below, write their (most general) Haskell type or "ill-typed" if it contains a type error. The type signatures of all functions below are provided in the appendix at the end.

(a) $\backslash x \rightarrow x$

Example answer:

$a \rightarrow a$

(b) $\backslash x y \rightarrow (x,y)$

$a \rightarrow b \rightarrow (a,b)$

(c) $\backslash x y \rightarrow$ if $x == y$ then show x else show (x,y)

$a \rightarrow a \rightarrow \text{String}$

(d) $\backslash x l \rightarrow x : l ++ l ++ [x]$

$a \rightarrow [a] \rightarrow [a]$

(e) $\text{foldr } (\text{const } 1)$

$a \rightarrow [a] \rightarrow a$

(f) $(\cdot, 42)$

ill typed

(g) $(\cdot, 42)$

$b \rightarrow ([\text{Char}], b)$

(h) $\text{reverse} . \text{reverse}$

$[a] \rightarrow [a]$

(i) $\backslash l \rightarrow \text{filter } (< 42) (l ++ l)$

$[a] \rightarrow [a]$

(j) $\text{let } f x = x \text{ in } (f 'a', f \text{True})$

ill typed

(k) $\text{fmap } (\text{fmap } (+1)) [\text{Nothing}]$

$\text{Num } a \rightarrow \text{Num } a \rightarrow \text{Maybe } a$

ill typed

Question 3 (20 points)

For each of the types below, write a Haskell expression that has that type. Don't write trivial expressions (such as `[]`, `Nothing`, or `undefined`) unless there is no other option. You can use any function from the `appendix`, `do` syntax, `list comprehensions`, or any valid Haskell.

(a) `Int -> Int`

Example answers:

`\x -> x + 1`
`(+1)`

(b) `Bool -> [Bool]`

`\x -> [not x]`

(c) `a -> Maybe b`

`\x -> Nothing`

(d) `(Int -> Char -> Bool) -> [Int] -> [Char] -> [Bool]`

`\f -> zipWith f`

(e) `(a -> b -> c) -> Maybe a -> Maybe b -> Maybe c`

`\f -> zipWith (maybe f)`

(f) `(a -> b) -> (b -> Bool) -> [a] -> [b]`

`\f g l -> map f l`

(g) `Maybe a -> (a -> [b]) -> [(a,b)]`

`map [ ]`

(h) `Eq a => a -> [a] -> [a]`

`\a (x:xs) -> case of a == x | true -> xs | false -> a : xs`

(i) `Show a => [a] -> IO String`

`undefined`

(j) `(a,b) -> (a -> b -> c) -> c`

`\(x,y) f -> f x y`

(k) `((a,b) -> c) -> c`

`undefined`

Question 4 (20 points)

Consider the following Dafny program that inefficiently calculates the cube of a number:

```

method Cube(m:int) returns (y:int)
  requires m > 0
  ensures y == m*m*m
{
  y := 0;
  var x := m * m;
  var z := m;
  while (z > 0)
  {
    z := z - 1;
    y := y + x;
  }
}

```

$27 - 3 * 9$

$y = (m * m * m) - z * x$

$9 = 27 - 2 * (9)$

$inv: y = m * m * m - z * x \quad \&\& \quad z >= 0$

Fill in the annotations in the following program to show that the Hoare triple given by the outermost pre- and post-conditions is valid. Be completely precise and pedantic in the way you apply Hoare rules--write assertions in *exactly* the form given by the rules rather than just logically equivalent ones. The provided blanks have been constructed so that if you work backwards from the end of the program you should only need to use the rule of consequence in the places indicated with $\rightarrow>$. These implication steps can (silently) rely on all the usual rules of arithmetic.

```

{ { m > 0 } }  $\rightarrow>$ 
{ { 0 == m*m*m - m*m*m && m >= 0 } }
  y := 0;
  { { y == m*m*m - m*m*m && m >= 0 } }
    var x := m * m;
    { { y == m*m*m - m*m*x && m >= 0 } }
      var z := m;
      { { y == m*m*m - z*x && z >= 0 } }
        while (z > 0) {
          { { y == m*m*m - z*x && z >= 0 && z > 0 } }  $\rightarrow>$ 
            { { y + x == m*m*m - (z-1)*x && z-1 >= 0 } }
              z := z - 1;
              { { y + x == m*m*m - z*x && z >= 0 } }
                y := y + x;
                { { y == m*m*m - z*x && z >= 0 } }
              }
            { { y == m*m*m - z*x && z >= 0 && !(z > 0) } }  $\rightarrow>$ 
          }
        }
      }
    }
  }

```


Question 5 (20 points)

Implement the following Haskell functions:

- (a) Implement a function `weave` that given two lists with elements of the same type, returns a list with elements alternating between the two lists. For example:

`weave [1,2,3] [4,5,6] = [1,4,2,5,3,6]`

You can assume that the lists have the same length.

`weave :: [a] → [a] → [a]`

`weave a [] = a`

`weave [] a = a`

`weave (x:xs) (y:ys) = x:y : weave xs ys`

- (b) Implement a function `powerset`, that given a list of elements of some type, computes the list of all of its sublists, including the empty list and the original list itself, in any order. For example:

`powerset [1,2,3] = [[1,2,3], [1,2], [1,3], [1], [2,3], [2], [3], []]`

`powerset :: [a] → [[a]]`

`powerset [] = []`

`powerset (x:[]) = [x]`

`powerset (x:xs) = [x] : powerset xs : powerset (x:xs)`

Jose Cruz
UID: 117992670

CMSC 433: Midterm Exam (Spring 2024)

Question 1 (20 points)

Consider the two standard folds in Haskell, `foldl` and `foldr`, whose definitions are given below:

```
foldr :: (a -> b -> b) -> b -> [a] -> b
foldr f b [] = b
foldr f b (x:xs) = f x (foldr f b xs)
```

```
foldl :: (b -> a -> b) -> b -> [a] -> b
foldl f b [] = b
foldl f b (x:xs) = foldl f (f b x) xs
```

Now consider the following two very similar but distinct folds over lists of integers:

```
f1 :: [Int] -> Int      (1,2,3) (1-(2-(3-0)))
f1 = foldr (-) 0
```

```
f2 :: [Int] -> Int      (1,2,3) 0-1-2-3
f2 = foldl (-) 0
```

Write two Dafny programs, one corresponding to each of `f1` and `f2`, but operating on arrays rather than lists. We've given you the method signatures to get you started:

```
method f1(a:array<int>) returns (out:int) {

```

```
    var i:int := a.Length-1
    out := 0
    while (i >= 0) {
        out := a[i] - out
        i := i-1
    }
}
```

```
method f2(a:array<int>) returns (out:int) {

```

```
    var i:int := 0
    out := 0
    while (i < a.Length) {
        out := out - a[i]
        i := i+1
    }
}
```

Question 2 (20 points)

For each of the Haskell expressions below, write their (most general) Haskell type or "ill-typed" if it contains a type error. The type signatures of all functions below are provided in the appendix at the end.

(a) $\backslash x \rightarrow x$

Example answer:

$a \rightarrow a$

(b) $\backslash x\ y \rightarrow (x,y)$

$a \rightarrow b \rightarrow (a,b)$

(c) $\backslash x\ y \rightarrow \text{if } x == y \text{ then show } x \text{ else show } (x,y)$

$a \rightarrow a \rightarrow \text{String}$

(d) $\backslash x\ l \rightarrow x : l ++ l ++ [x]$

$a \rightarrow [a] \rightarrow [a]$

(e) $\text{foldr} (\text{const } 1)$

$(a \rightarrow b \rightarrow a)$

$:: \text{Int} \rightarrow \text{Int}$

(f) $(\cdot, 42)$

$:: \text{Int}$

(g) $(\cdot) "42"$

$a \rightarrow (\text{String}, a)$

(h) $\text{reverse} . \text{reverse}$

$[a] \rightarrow [a]$

(i) $\backslash l \rightarrow \text{filter} (< 42) (l ++ l)$

$[Int] \rightarrow [Int]$

(j) $\text{let } f\ x = x \text{ in } (f\ 'a', f\ \text{True})$

$:: \text{Int}$

(k) $\text{fmap} (\text{fmap } (+1)) [\text{Nothing}]$

$:: \text{Int}$

$(a \rightarrow b) \rightarrow f\ a \rightarrow f\ b$

Question 3 (20 points)

For each of the types below, write a Haskell expression that has that type. Don't write trivial expressions (such as [], Nothing, or undefined) unless there is no other option. You can use any function from the appendix, do syntax, list comprehensions, or any valid Haskell.

(a) `Int -> Int`

Example answers:

`\x -> x + 1`
`(+1)`

(b) `Bool -> [Bool]`

`\x -> x : [True]`

(c) `a -> Maybe b`

`Nothing`

(d) `(Int -> Char -> Bool) -> [Int] -> [Char] -> [Bool]`

`\g [i:is] [chars] -> g i c`

(e) `(a -> b -> c) -> Maybe a -> Maybe b -> Maybe c`

`\g (Just a) (Just b) -> Just (g a b)`

(f) `(a -> b) -> (b -> Bool) -> [a] -> [b]`

`\g1 g2 [a:as] -> if g2 (g1 a) then [g1 a] else []`

(g) `Maybe a -> (a -> [b]) -> [(a,b)]`

`\(Just a) f -> let g a = [x:xs] in [(a,x)]`

(h) `Eq a => a -> [a] -> [a]`

`\a list -> a : list`

(i) `Show a => [a] -> IO String`

`\(x:xs) -> putStrLn (show x)`

(j) `(a,b) -> (a -> b -> c) -> c`

`\(a,b) f -> f a b`

(k) `((a,b) -> c) -> c`

`undefined`

Question 4 (20 points)

Consider the following Dafny program that inefficiently calculates the cube of a number:

```

method Cube(m:int) returns (y:int)
  requires m > 0
  ensures y == m*m*m
{
  y := 0;
  var x := m * m;
  var z := m;
  while (z > 0)
  {
    z := z - 1;
    y := y + x;
  }
}

```

Handwritten annotations for the program:

- Initial state: $m=3, z=3, y=0$
- Loop iteration 1: $z=2, y=9$
- Loop iteration 2: $z=1, y=18$
- Loop iteration 3: $z=0, y=27$
- Final state: $x = (m-z) = 3$

Fill in the annotations in the following program to show that the Hoare triple given by the outermost pre- and post- conditions is valid. Be completely precise and pedantic in the way you apply Hoare rules--write assertions in *exactly* the form given by the rules rather than just logically equivalent ones. The provided blanks have been constructed so that if you work backwards from the end of the program you should only need to use the rule of consequence in the places indicated with $\rightarrow>$. These implication steps can (silently) rely on all the usual rules of arithmetic.

```

{{ m > 0 }}  $\rightarrow>$ 
{{ m > 0  $\wedge$  z == (m-m) * (m-m) == 0 }}
  y := 0;
{{ m > 0  $\wedge$  z == (m-m) * (m-m) == y }}
  var x := m * m;
{{ m > 0  $\wedge$  z == x * (m-m) == y }}
  var z := m;
{{ m > 0  $\wedge$  x * (m-z) == y }}
  while (z > 0) {
    {{ z > 0  $\wedge$  m > 0  $\wedge$  z * (m-z) == y }}  $\rightarrow>$ 
    {{ m > 0  $\wedge$  z * (m-(z-1)) == y + x }}
    z := z - 1;
    {{ m > 0  $\wedge$  z * (m-z) == y + x }}
    y := y + x;
    {{ m > 0  $\wedge$  z * (m-z) == y }}
  }
  {{ ! (z > 0)  $\wedge$  m > 0  $\wedge$  z * (m-z) == y }}  $\rightarrow>$ 
  {{ y == m * m * m }}

```


Question 5 (20 points)

Implement the following Haskell functions:

- (a) Implement a function `weave` that given two lists with elements of the same type, returns a list with elements alternating between the two lists. For example:

`weave [1,2,3] [4,5,6] = [1,4,2,5,3,6]`

You can assume that the lists have the same length.

`weave [] = []`
`weave [x] = [x]`
`weave (x:xs) (y:ys) = (x:[y]) ++ weave xs ys`

- (b) Implement a function `powerset`, that given a list of elements of some type, computes the list of all of its sublists, including the empty list and the original list itself, in any order. For example:

`powerset [1,2,3] = [[1,2,3], [1,2], [1,3], [1], [2,3], [2], [3], []]`

`powerset [] = [[]]`

`powerset (x:xs) = nub [x:vs] ++ [ powerset xs ]` where

`nub [] = []`

`nub (x:ys) = [x:ys] ++ nub (ys)`

Sai Praveer Theella

CMSC 433: Midterm Exam (Spring 2024)

Question 1 (20 points)

Consider the two standard folds in Haskell, `foldl` and `foldr`, whose definitions are given below:

```
foldr :: (a -> b -> b) -> b -> [a] -> b
foldr f b [] = b
foldr f b (x:xs) = f x (foldr f b xs)

foldl :: (b -> a -> b) -> b -> [a] -> b
foldl f b [] = b
foldl f b (x:xs) = foldl f (f b x) xs
```

Now consider the following two very similar but distinct folds over lists of integers:

```
f1 :: [Int] -> Int
f1 = foldr (-) 0

f2 :: [Int] -> Int
f2 = foldl (-) 0
```

Write two Dafny programs, one corresponding to each of `f1` and `f2`, but operating on arrays rather than lists. We've given you the method signatures to get you started:

<pre>method f1(a:array<int>) returns (out:int) { var acc:int := a.length-1; out := 0; while (acc >= 0) { out := out - a[acc]; acc := acc - 1; } }</pre>	<pre>method f2(a:array<int>) returns (out:int) { var acc:int := 0; out := 0; while (acc < a.length) { out := out - a[acc]; acc := acc + 1; } }</pre>
--	---

Question 2 (20 points)

For each of the Haskell expressions below, write their (most general) Haskell type or "ill-typed" if it contains a type error. The type signatures of all functions below are provided in the appendix at the end.

(a) $\backslash x \rightarrow x$

Example answer:

$a \rightarrow a$

(b) $\backslash x\ y \rightarrow (x,y)$

$a \rightarrow b \rightarrow (a,b)$

(c) $\backslash x\ y \rightarrow \text{if } x == y \text{ then show } x \text{ else show } (x,y)$

$(a,b) \rightarrow (a,b) \rightarrow (a,b)$

(d) $\backslash x\ l \rightarrow x : l ++ l ++ [x]$

$a \rightarrow [a] \rightarrow [a]$

(e) $\text{foldr } (\text{const } 1)$

$[a] \rightarrow [Int]$

(f) $(^, 42^)$

$b \rightarrow (String, b)$

(g) $(.) ^42^$

(h) $\text{reverse} . \text{reverse}$

$[a] \rightarrow [a]$

(i) $\backslash l \rightarrow \text{filter } (< 42) (l ++ l)$

$[a] \rightarrow [a]$

(j) $\text{let } f\ x = x \text{ in } (f\ 'a', f\ \text{True})$

$(\text{Bool} \rightarrow b) \rightarrow (b,b)$

(k) $\text{fmap } (\text{fmap } (+1)) [\text{Nothing}]$

Question 3 (20 points)

For each of the types below, write a Haskell expression that has that type. Don't write trivial expressions (such as [], Nothing, or undefined) unless there is no other option. You can use any function from the appendix, do syntax, list comprehensions, or any valid Haskell.

(a) Int -> Int

Example answers:

\x -> x + 1

(+1)

(b) Bool -> [Bool]

let foo val = if val then [val] else [val]

(c) a -> Maybe b

\a -> Nothing

(d) (Int -> Char -> Bool) -> [Int] -> [Char] -> [Bool]

zipWith (\x y -> if x == 1 and y == 'a' then True else False) vals chars

(e) (a -> b -> c) -> Maybe a -> Maybe b -> Maybe c

undefined

(f) (a -> b) -> (b -> Bool) -> [a] -> [b]

elem vals val

(g) Maybe a -> (a -> [b]) -> [(a,b)]

(h) Eq a => a -> [a] -> [a]

~~delete~~ val

(i) Show a => [a] -> IO String

(j) (a,b) -> (a -> b -> c) -> c

(k) ((a,b) -> c) -> c

let foo = if (val1 < val2, val2) -> if val == True and val2 == 2 then 'a' else 'b') == 'a' then 'a' else 'c'

Question 4 (20 points)

Consider the following Dafny program that inefficiently calculates the cube of a number:

```
method Cube(m:int) returns (y:int)
  requires m > 0
  ensures y == m*m*m
{
  y := 0;
  var x := m * m;
  var z := m;
  while (z > 0)
  {
    z := z - 1;
    y := y + x;
  }
}
```

iter 1 2

x	x	z
$m \neq n$	$m \neq n$	m
x	$m * n$	$m - 1$
$2x$	$m + n$	$m - 2$
$3x$		$m - 3$
	$(m - z)x$	

Fill in the annotations in the following program to show that the Hoare triple given by the outermost pre- and post- conditions is valid. Be completely precise and pedantic in the way you apply Hoare rules - write assertions in *exactly* the form given by the rules rather than just logically equivalent ones. The provided blanks have been constructed so that if you work backwards from the end of the program you should only need to use the rule of consequence in the places indicated with $\rightarrow\gg$. These implication steps can (silently) rely on all the usual rules of arithmetic.

```
{ { m > 0 } }  $\rightarrow\gg$ 
{ { 0 = (z - z) * (m * n) } } m * n = m * n && m * z = 0 } }
  y := 0;
  { { y = (z - z) * (m * n) } } m * n = m * n && m * z = 0 } }
    var x := m * m;
    { { y = (z - z) * (x) } } x = m * m && m * z = 0 } }
      var z := m;
      { { y = (m - z) * (x) } } x = m * m && m * z = 0 } }
        while (z > 0) {
          { { y = (m - z) * (x) } } x = m * m && m * z = 0 && z < 0 } }  $\rightarrow\gg$ 
          { { y = (m - z + 1) * (x) } } x = m * m && m * z = 0 } }
            z := z - 1;
            { { y = (m - z) * (x) } } x = m * m && m * z = 0 } }
              y := y + x;
              { { y = (m - z) * (x) } } x = m * m && m * z = 0 } }
            }
          { { y = (m - z) * x } } x = m * m && m * z = 0 && z < 0 } }  $\rightarrow\gg$ 
          { { y = m * m * m } }
```

Question 5 (20 points)

Implement the following Haskell functions:

- (a) Implement a function `weave` that given two lists with elements of the same type, returns a list with elements alternating between the two lists. For example:

`weave [1,2,3] [4,5,6] = [1,4,2,5,3,6]`

You can assume that the lists have the same length.

`weave :: [a] -> [a] -> [a]`
`weave [] [] = []`
`weave (x:xs) (y:ys) = (x:y) ++ (weave xs ys)`

- (b) Implement a function `powerset`, that given a list of elements of some type, computes the list of all of its sublists, including the empty list and the original list itself, in any order. For example:

`powerset [1,2,3] = [[], [1,2,3], [1,2], [1,3], [2,3], [1], [2], [3], []]`

`powerset :: [a] -> [[a]]`
`powerset [] = []`
`powerset lst = foldr (:) (head lst) (powerset tail lst)`



Andrew Xia

CMSC 433: Midterm Exam (Spring 2024)

Question 1 (20 points)

Consider the two standard folds in Haskell, `foldl` and `foldr`, whose definitions are given below:

```
foldr :: (a -> b -> b) -> b -> [a] -> b
foldr f b [] = b
foldr f b (x:xs) = f x (foldr f b xs)

foldl :: (b -> a -> b) -> b -> [a] -> b
foldl f b [] = b
foldl f b (x:xs) = foldl f (f b x) xs
```

Now consider the following two very similar but distinct folds over lists of integers:

```
f1 :: [Int] -> Int
f1 = foldr (-) 0

f2 :: [Int] -> Int
f2 = foldl (-) 0
```

Write two Dafny programs, one corresponding to each of `f1` and `f2`, but operating on arrays rather than lists. We've given you the method signatures to get you started:

	method <code>f1(a:array<int>)</code> returns (out:int) {	method <code>f2(a:array<int>)</code> returns (out:int) {
<pre>out := 0; var i := a.Length - 1; while (i >= 0) { out := out - a[i]; i := i - 1; }</pre>	<pre>out := 0; var i := 0; while (i < a.Length) { out := out - a[i]; i := i + 1; }</pre>	<pre>out := 0; var i := 0; while (i < a.Length) { out := out - a[i]; i := i + 1; }</pre>

Question 2 (20 points)

For each of the Haskell expressions below, write their (most general) Haskell type or "ill-typed" if it contains a type error. The type signatures of all functions below are provided in the appendix at the end.

(a) $\backslash x \rightarrow x$

Example answer:

$a \rightarrow a$

(b) $\backslash x y \rightarrow (x,y)$

$a \rightarrow b \rightarrow (a,b)$

(c) $\backslash x y \rightarrow \text{if } x == y \text{ then show } x \text{ else show } (x,y)$

$\text{Bool} \rightarrow \text{Bool} \rightarrow \text{String}$

(d) $\backslash x l \rightarrow x : l ++ l ++ [x]$

$a \rightarrow [a] \rightarrow [a]$

(e) $\text{foldr} (\text{const } 1)$

$\mathbb{L}$

(f) ("42")

$a \rightarrow \text{String} \rightarrow (a, \text{String})$

(g) ("42")

$a \rightarrow \text{String}$

(h) $\text{reverse} . \text{reverse}$

$[a] \rightarrow [a]$

(i) $\backslash l \rightarrow \text{filter } (< 42) (l ++ l)$

$[\text{Int}] \rightarrow [\text{Int}]$

(j) $\text{let } f \ x = x \text{ in } (f 'a', f \text{ True})$

(k) $\text{fmap } (\text{fmap } (+1)) [\text{Nothing}]$

$[[a]] \rightarrow [[a]]$

Question 3 (20 points)

For each of the types below, write a Haskell expression that has that type. Don't write trivial expressions (such as [], Nothing, or undefined) unless there is no other option. You can use any function from the appendix, do syntax, list comprehensions, or any valid Haskell.

(a) $\text{Int} \rightarrow \text{Int}$

Example answers:

$\backslash x \rightarrow x + 1$

$(+1)$

(b) $\text{Bool} \rightarrow [\text{Bool}]$

$\backslash b \rightarrow \text{repeat } b$

(c) $a \rightarrow \text{Maybe } b$

listToMaybe

(d) $(\text{Int} \rightarrow \text{Char} \rightarrow \text{Bool}) \rightarrow [\text{Int}] \rightarrow [\text{Char}] \rightarrow [\text{Bool}]$

$\text{zipWith } (+)$

(e) $(a \rightarrow b \rightarrow c) \rightarrow \text{Maybe } a \rightarrow \text{Maybe } b \rightarrow \text{Maybe } c$

zipWith

(f) $(a \rightarrow b) \rightarrow (b \rightarrow \text{Bool}) \rightarrow [a] \rightarrow [b]$

(g) $\text{Maybe } a \rightarrow (a \rightarrow [b]) \rightarrow [(a, b)]$

MaybeToList

(h) $\text{Eq } a \Rightarrow a \rightarrow [a] \rightarrow [a]$

$\backslash \text{delete } a []$

(i) $\text{Show } a \Rightarrow [a] \rightarrow \text{IO String}$

(j) $(a, b) \rightarrow (a \rightarrow b \rightarrow c) \rightarrow c$

(k) $((a, b) \rightarrow c) \rightarrow c$

Question 4 (20 points)

Consider the following Dafny program that inefficiently calculates the cube of a number:

```
method Cube(m:int) returns (y:int)
  requires m > 0
  ensures y == m*m*m
{
  y := 0;
  var x := m * m;
  var z := m;
  while (z > 0)
  {
    z := z - 1;
    y := y + x;
  }
}
```

$$\begin{array}{rcl}
 m & = & z \\
 y & = & 0 \\
 x & = & 4 \\
 z & = & 2 \\
 \hline
 y & = & 4 \quad , \quad y = 8
 \end{array}$$

Fill in the annotations in the following program to show that the Hoare triple given by the outermost pre- and post- conditions is valid. Be completely precise and pedantic in the way you apply Hoare rules -- write assertions in *exactly* the form given by the rules rather than just logically equivalent ones. The provided blanks have been constructed so that if you work backwards from the end of the program you should only need to use the rule of consequence in the places indicated with $\rightarrow$. These implication steps can (silently) rely on all the usual rules of arithmetic.

```
{ { m > 0 } }  $\rightarrow$ 
{ { 0 ≤ m · m & m ≥ 0 & m · m = m · m } }
  y := 0;
  { { y ≤ m · m & m ≥ 0 & m · m = m · m } }
    var x := m * m;
    { { y ≤ m · m & m ≥ 0 & x = m · m } }
      var z := m;
      { { y ≤ m · m & z ≥ 0 & x = m · m } }
        while (z > 0) {
          { { y ≤ m · m & z ≥ 0 & x = m · m } }  $\rightarrow$ 
            { { z - 1 ≥ 0 & x = m · m & y + x ≤ m · m & m } }
              z := z - 1;
              { { z ≥ 0 & x = m · m & y + x ≤ m · m & m } }
                y := y + x;
                { { z ≥ 0 & x = m · m & y ≤ m · m & m } }
              }
            { { y = m · m & m ≥ 0 & z = 0 & x = m · m } }  $\rightarrow$ 
          }
        }
```

8

Question 5 (20 points)

Implement the following Haskell functions:

- (a) Implement a function `weave` that given two lists with elements of the same type, returns a list with elements alternating between the two lists. For example:
`weave [1,2,3] [4,5,6] = [1,4,2,5,3,6]`

You can assume that the lists have the same length.

case a b = foldl f (+) (zipWith a b) []

- (b) Implement a function `powerSet`, that given a list of elements of some type, computes the list of all of its sublists, including the empty list and the original list itself, in any order. For example:

`powerSet [1,2,3] = [[1,2,3], [1,2], [1,3], [1], [2,3], [2], [3], []]`

Nikesh
Chakraborty

CMSC 433: Midterm Exam (Spring 2024)

Question 1 (20 points)

$$3 - 1 = 2$$

Consider the two standard folds in Haskell, foldl and foldr, whose definitions are given below:

```
foldr :: (a -> b -> b) -> b -> [a] -> b
foldr f b [] = b
foldr f b (x:xs) = f x (foldr f b xs)

foldl :: (b -> a -> b) -> b -> [a] -> b
foldl f b [] = b
foldl f b (x:xs) = foldl f (f b x) xs
```

$$\begin{aligned} \text{Process } (-) \text{ right to left} \\ \text{foldr } (-) 0 [1, 2, 3] &= 1 - (2 - (3 - 0)) \\ &= 1 - (2 - 3) = 2 \end{aligned}$$

$$\begin{aligned} \text{Process } (-) \text{ left to right} \\ \text{foldl } (-) 0 [1, 2, 3] &= ((3 - 0) - 2) - 1 \\ &= (3 - 2) - 1 = 0 \end{aligned}$$

Now consider the following two very similar but distinct folds over lists of integers:

$$\begin{aligned} f1 :: [Int] \rightarrow Int \\ f1 = \text{foldr } (-) 0 \end{aligned} \quad \begin{aligned} f2 :: [Int] \rightarrow Int \\ f2 = \text{foldl } (-) 0 \end{aligned}$$

$$\begin{aligned} [1, 2, 3, 4] &= ((4 - 3) - 2) - 1 \\ &= 0 \end{aligned}$$

```
f2 :: [Int] -> Int
f2 = foldl (-) 0
```

Write two Dafny programs, one corresponding to each of f1 and f2, but operating on arrays rather than lists. We've given you the method signatures to get you started:

```
method f1(a:array<int>) returns (out:int) {
  var i: int := 0;
  out := 0;
  while (i < a.Length) {
    out := a[a.Length - i - 1] - out;
    i := i + 1;
  }
```

```
method f2(a:array<int>) returns (out:int) {
  var i: int := a.Length - 1;
  out := 0;
  while (i >= 0) {
    out := a[i] - out;
    i := i - 1;
  }
```

```
  return out;
}
```

```
  }
  return out;
}
```

Question 2 (20 points)

For each of the Haskell expressions below, write their (most general) Haskell type or “ill-typed” if it contains a type error. The type signatures of all functions below are provided in the appendix at the end.

(a) $\backslash x \rightarrow x$

Example answer:

$a \rightarrow a$

(b) $\backslash x\ y \rightarrow (x,y)$

$a \rightarrow b \rightarrow (a,b)$

(c) $\backslash x\ y \rightarrow \text{if } x == y \text{ then show } x \text{ else show } (x,y)$

ill-typed

(d) $\backslash x\ l \rightarrow x : l ++ l ++ [x]$

$a \rightarrow [a] \rightarrow [a]$

(e) $\text{foldr } (\text{const } 1)$

$a \rightarrow [a] \rightarrow \text{int}$

(f) ("42")

$((), \text{string})$

(g) ("42")

ill-typed

(h) $\text{reverse} . \text{reverse}$

$[a] \rightarrow [a]$

(i) $\backslash l \rightarrow \text{filter } (< 42) (l ++ l)$

$[\text{Int}] \rightarrow [\text{Int}]$

(j) $\text{let } f\ x = x \text{ in } (f\ 'a', f\ \text{True})$

ill-typed

(k) $\text{fmap } (\text{fmap } (+1)) [\text{Nothing}]$

ill-typed

Question 3 (20 points)

For each of the types below, write a Haskell expression that has that type. Don't write trivial expressions (such as `[]`, `Nothing`, or `undefined`) unless there is no other option. You can use any function from the appendix, do syntax, list comprehensions, or any valid Haskell.

(a) `Int -> Int`

Example answers:

`\x -> x + 1`

`(+1)`

(b) `Bool -> [Bool]`

repeat True

(c) `a -> Maybe b`

\a -> Just (f a)

(d) `(Int -> Char -> Bool) -> [Int] -> [Char] -> [Bool]`

zipWith (\x y -> x && y) [1,2,3] ['a','b','c']

(e) `(a -> b -> c) -> Maybe a -> Maybe b -> Maybe c`

(f) `(a -> b) -> (b -> Bool) -> [a] -> [b]`

(g) `Maybe a -> (a -> [b]) -> [(a,b)]`

(h) `Eq a => a -> [a] -> [a]`

nub

(i) `Show a => [a] -> IO String`

(j) `(a,b) -> (a -> b -> c) -> c`

/(a,b) -> f a b

(k) `((a,b) -> c) -> c`

/f -> snd

Question 4 (20 points)

Consider the following Dafny program that inefficiently calculates the cube of a number: $w \mid x = 2 * 2$

method Cube(m:int) returns (y:int)

requires m > 0

ensures y == m*m*m

{

 y := 0;

 var x := m * m;

 var z := m;

 while (z > 0)

 {

 z := z - 1;

 y := y + x;

 }

}

Loop	x	y	z
1	2 * 2	2 * 2	2
2	2 * 2	2 * 2 * 2	0

$$\Rightarrow x = m * m \quad \&\& \quad x + y = m * m * m$$

Fill in the annotations in the following program to show that the Hoare triple given by the outermost pre- and post-conditions is valid. Be completely precise and pedantic in the way you apply Hoare rules—write assertions in *exactly* the form given by the rules rather than just logically equivalent ones. The provided blanks have been constructed so that if you work backwards from the end of the program you should only need to use the rule of consequence in the places indicated with $\rightarrow$. These implication steps can (silently) rely on all the usual rules of arithmetic.

```

{{ m > 0 }}  $\rightarrow$ 
{{ 0 == 0 && m == m }}
  y := 0;
  {{ y == 0 && m == m }}
  var x := m * m;
  {{ y == 0 && x == m * m && m == m }}
  var z := m;
  {{ x == m * m && x + y == m * m * m }}
  while (z > 0) {
    {{ x == m * m && x + (y + x) == m * m * m && z == 0 }}
    {{ x == m * m && x + (y + x) == m * m * m }}  $\rightarrow$ 
    z := z - 1;
    {{ x == m * m && x + (y + x) == m * m * m }}
    y := y + x;
    {{ x == m * m && x + y == m * m * m }}
  }
  {{ x == m * m && x + y == m * m * m && (z > 0) }}
  {{ y = m * m * m }}  $\rightarrow$ 

```


Question 5 (20 points)

Implement the following Haskell functions:

- (a) Implement a function `weave` that given two lists with elements of the same type, returns a list with elements alternating between the two lists. For example:

```
weave [1,2,3] [4,5,6] = [1,4,2,5,3,6]
```

You can assume that the lists have the same length.

```
weave [] = []
```

```
weave [] = []
```

```
weave (x:xs) (y:ys) = x:y ++ weave xs ys
```

- (b) Implement a function `powerset`, that given a list of elements of some type, computes the list of all of its sublists, including the empty list and the original list itself, in any order. For example:

```
powerset [1,2,3] = [[1,2,3], [1,2], [1,3], [1], [2,3], [2], [3], []]
```

```
powerset [] = []
```


Anna Assefa

CMSC 433: Midterm Exam (Spring 2024)

Question 1 (20 points)

Consider the two standard folds in Haskell, foldl and foldr, whose definitions are given below:

```
foldr :: (a -> b -> b) -> b -> [a] -> b
foldr f b [] = b
foldr f b (x:xs) = f x (foldr f b xs)

foldl :: (b -> a -> b) -> b -> [a] -> b
foldl f b [] = b
foldl f b (x:xs) = foldl f (f b x) xs
```

Now consider the following two very similar but distinct folds over lists of integers:

```
f1 :: [Int] -> Int
f1 = foldr \a n -> 0

f2 :: [Int] -> Int
f2 = foldl (-) 0
```

$$0 - 1 = -1, -1 - 2 = -3, -3 - 2 = -5, -5 - 3 = -8$$

Write two Dafny programs, one corresponding to each of f1 and f2, but operating on arrays rather than lists. We've given you the method signatures to get you started:

```
method f1(a:array<int>) returns (out:int) {
  var out:int := 0;
  var count:int := a.Length - 1;
```

```
  while (count >= 0) {
    out := out - a[count];
    count := count - 1;
  }
```

```
  return out;
}
```

```
}
```

```
method f2(a:array<int>) returns (out:int) {
```

```
  var out:int := 0;
  var count:int := 0;
```

```
  while (count < a.Length) {
    out := a.Length - out;
    count := count + 1;
  }
```

```
  return out;
}
```

```
}
```

```
}
```

```
}
```

Question 2 (20 points)

For each of the Haskell expressions below, write their (most general) Haskell type or "ill-typed" if it contains a type error. The type signatures of all functions below are provided in the appendix at the end.

(a) $\backslash x \rightarrow x$

Example answer:

$a \rightarrow a$

(b) $\backslash x\ y \rightarrow (x,y)$

$a \rightarrow b \rightarrow (a,b)$

(c) $\backslash x\ y \rightarrow$ if $x == y$ then show x else show (x,y)

$a \rightarrow a \rightarrow \text{string}$

(d) $\backslash x\ l \rightarrow x : l ++ l ++ [x]$

$a \rightarrow [a] \rightarrow [a]$

(e) `foldr (const 1)`

ill-typed

(f) `("42")`

$\text{Int} \rightarrow (\text{Int}, \text{Int})$

(g) `() "42"`

$\text{Int} \rightarrow (\text{Int}, \text{Int})$

(h) `reverse . reverse`

ill-typed

(i) $\backslash l \rightarrow \text{filter} (< 42) (l ++ l)$

$[a] \rightarrow [a]$

(j) `let f x = x in (f 'a', f True)`

$() \rightarrow \text{Bool}$

(k) `fmap (fmap (+1)) [Nothing]`

$(\text{Int} \rightarrow \text{Int}) \rightarrow (\text{Int} \rightarrow$

Question 3 (20 points)

For each of the types below, write a Haskell expression that has that type. Don't write trivial expressions (such as [], Nothing, or undefined) unless there is no other option. You can use any function from the appendix, do syntax, list comprehensions, or any valid Haskell.

(a) $\text{Int} \rightarrow \text{Int}$

Example answers:

$\backslash x \rightarrow x + 1$

$(+1)$

(b) $\text{Bool} \rightarrow [\text{Bool}]$

$\backslash x \rightarrow \text{if } x \text{ then } [x] \text{ else } x : [\text{True}]$

(c) $a \rightarrow \text{Maybe } b$

undefined

(d) $(\text{Int} \rightarrow \text{Char} \rightarrow \text{Bool}) \rightarrow [\text{Int}] \rightarrow [\text{Char}] \rightarrow [\text{Bool}]$

$\backslash f : c \rightarrow : [\text{True}]$

(e) $(a \rightarrow b \rightarrow c) \rightarrow \text{Maybe } a \rightarrow \text{Maybe } b \rightarrow \text{Maybe } c$

$\backslash f \ x \ y \rightarrow \text{Just } (f$

(f) $(a \rightarrow b) \rightarrow (b \rightarrow \text{Bool}) \rightarrow [a] \rightarrow [b]$

$\backslash f \ j \ (x : xs) \rightarrow \text{if } j \ (f \ x) \text{ then } [f \ x] \text{ else } f \ x : [f \ x]$

(g) $\text{Maybe } a \rightarrow (a \rightarrow [b]) \rightarrow [(a, b)]$

$\backslash x \ f \rightarrow f$

(h) $\text{Eq } a \Rightarrow a \rightarrow [a] \rightarrow [a]$

(i) Show $a \Rightarrow [a] \rightarrow \text{IO String}$

(j) $(a, b) \rightarrow (a \rightarrow b \rightarrow c) \rightarrow c$

$\backslash (a, b) \ f \rightarrow f \ a \ b$

(k) $((a, b) \rightarrow c) \rightarrow c$

Question 4 (20 points)

Consider the following Dafny program that inefficiently calculates the cube of a number:

```
method Cube(m:int) returns (y:int)
  requires m > 0
  ensures y == m*m*m
{
  y := 0;
  var x := m * m;
  var z := m;
  while (z > 0)
  {
    z := z - 1;
    y := y + x;
  }
}
```

Fill in the annotations in the following program to show that the Hoare triple given by the outermost pre- and post- conditions is valid. Be completely precise and pedantic in the way you apply Hoare rules—write assertions in *exactly* the form given by the rules rather than just logically equivalent ones. The provided blanks have been constructed so that if you work backwards from the end of the program you should only need to use the rule of consequence so that if you work backwards from the end of the program you should only rely on all the usual rules of arithmetic.

```
{ { m > 0 } } ->>
{ { m * m == m * m, ll 0 == 0, ll m > 0 } }
  y := 0;
  { { m * m == m * m, ll y == 0, ll m > 0 } }
    var x := m * m;
    { { x == m * m, ll y == 0, ll m > 0 } }
      var z := m;
      { { x == m * m, ll y == 0, ll z > 0 } }
        while (z > 0) {
          { { y == m * m, ll y == z * x, ll z > 0 } } ->>
            { { x == m * m, ll y + x == z * x, ll z - 1 > 0 } }
              z := z - 1;
              { { x == m * m, ll y + x == z * x, ll z > 0 } }
                { { x == m * m, ll y := y + x; } }
                  { { x == m * m, ll y == z * x, ll z > 0 } }
                    { { x == m * m, ll y == m * m * m, ll ! (z > 0) } }
                      { { y = m * m * m } } ->>
```


Question 5 (20 points)

Implement the following Haskell functions:

- (a) Implement a function `weave` that given two lists with elements of the same type, returns a list with elements alternating between the two lists. For example:

`weave [1,2,3] [4,5,6] = [1,4,2,5,3,6]`

You can assume that the lists have the same length.

`weave :: [a] -> [a] -> [a]`

`weave lst1 lst2 = case (lst1, lst2) of`

`([], []) -> []`

`(x:xs, y:ys) -> x:y:weave xs ys`

- (b) Implement a function `powerSet`, that given a list of elements of some type, computes the list of all of its sublists, including the empty list and the original list itself, in any order. For example:

`powerSet [1,2,3] = [[], [1], [2], [3], [1,2], [1,3], [2,3], [1,2,3], []]`

`powerSet :: [a] -> [[a]]`

`powerSet lst = lst : case lst of`

`[] -> [[]]`

`(x:xs) -> powerSetAux x xs : powerSet xs`

1, 2, 3

`powerSetAux x xs = case (x:xs) of`

`[] -> []`

`(x:xs) -> x:xs:powerSetAux xs`

Riley Gray

CMSC 433: Midterm Exam (Spring 2024)

Question 1 (20 points)

Consider the two standard folds in Haskell, `foldl` and `foldr`, whose definitions are given below:

```
foldr :: (a -> b -> b) -> b -> [a] -> b
foldr f b [] = b
foldr f b (x:xs) = f x (foldr f b xs)
```

```
foldl :: (b -> a -> b) -> b -> [a] -> b
foldl f b [] = b
foldl f b (x:xs) = foldl f (f b x) xs
```

Now consider the following two very similar but distinct folds over lists of integers:

```
f1 :: [Int] -> Int
f1 = foldr (-) 0
```

```
f2 :: [Int] -> Int
f2 = foldl (-) 0
```

Write two Dafny programs, one corresponding to each of `f1` and `f2`, but operating on arrays rather than lists. We've given you the method signatures to get you started:

```
method f1(a:array<int>) returns (out:int) {
```

```
  var out := 0
```

```
  for i := 0..length-1 // var,
```

```
  method f2(a:array<int>) returns (out:int) {
```

```
    var out := 0
```

```
    for i := 0
```

```
      while (i < a.length-1) {
```

```
        out := out - a[i]
```

```
        i := i + 1
```

```
      }
```

```
    while (i >= 0) {
      out := out - a[i]
      i := i - 1
    }
  }
```

```
  return out
```

```
}
```

```
  return out
```

```
}
```

Question 2 (20 points)

For each of the Haskell expressions below, write their (most general) Haskell type or "ill-typed" if it contains a type error. The type signatures of all functions below are provided in the appendix at the end.

(a) $\backslash x \rightarrow x$

Example answer:

$a \rightarrow a$

(b) $\backslash x y \rightarrow (x,y)$

$a \rightarrow b \rightarrow (a,b)$

(c) $\backslash x y \rightarrow \text{if } x == y \text{ then show } x \text{ else show } \underline{\quad}$

$a \text{ ill-typed}$

(d) $\backslash x l \rightarrow x : l ++ l ++ [x]$

$a \rightarrow [a] \rightarrow [a]$

(e) $\text{foldr} (\text{const } 1)$

$[a] \rightarrow [a] \rightarrow [a]$

(f) $(^42)$

ill-typed

(g) $() ^{42}$

$a \rightarrow (\text{String}, a)$

(h) $\text{reverse} . \text{reverse}$

$[a] \rightarrow [a]$

(i) $\backslash l \rightarrow \text{filter } (< 42) (l ++ l)$

$\text{int} \rightarrow [\text{int}]$

(j) $\text{let } f x = x \text{ in } (f 'a', f \underline{\quad})$

ill-typed

(k) $\text{fmap} (\text{fmap } (+1)) [\text{Nothing}]$

Question 3 (20 points)

For each of the types below, write a Haskell expression that has that type. Don't write trivial expressions (such as `[]`, `Nothing`, or `undefined`) unless there is no other option. You can use any function from the appendix, do syntax, list comprehensions, or any valid Haskell.

(a) `Int -> Int`

Example answers:

`\x -> x + 1`
`(+1)`

(b) `Bool -> [Bool]`
`\x -> if x then [x] else [x]`

(c) `a -> Maybe b`

`\x -> Nothing`

(d) `(Int -> Char -> Bool) -> [Int] -> [Char] -> [Bool]`
`zipWith (\x y -> (x,y) == (x,y))`

(e) `(a -> b -> c) -> Maybe a -> Maybe b -> Maybe c`

`zipWith`

(f) `(a -> b) -> (b -> Bool) -> [a] -> [b]`

(g) `Maybe a -> (a -> [b]) -> [(a,b)]`

(h) `Eq a => a -> [a] -> [a]`

(i) `Show a => [a] -> IO String`

(j) `(a,b) -> (a -> b -> c) -> c`

`\(x,y) ->`

(k) `((a,b) -> c) -> c`

Question 4 (20 points)

Consider the following Dafny program that inefficiently calculates the cube of a number:

```
method Cube(m:int) returns (y:int)
  requires m > 0
  ensures y == m*m*m
{
  y := 0;
  var x := m * m;
  var z := m;
  while (z > 0)
  {
    z := z - 1;
    y := y + x;
  }
}
```

Fill in the annotations in the following program to show that the Hoare triple given by the outermost pre- and post- conditions is valid. Be completely precise and pedantic in the way you apply Hoare rules—write assertions in *exactly* the form given by the rules rather than just logically equivalent ones. The provided blanks have been constructed so that if you work backwards from the end of the program you should only need to use the rule of consequence in the places indicated with $\rightarrow$. These implication steps can (silently) rely on all the usual rules of arithmetic.

```
{ { m > 0 } }  $\rightarrow$ 
{ {  $m * m == m * m \wedge 0 == m * m \wedge m > 0$  } }
{ {  $y := 0;$  } }
{ {  $m * m == m * m \wedge y == m * m \wedge m > 0$  } }
{ {  $\text{var } x := m * m;$  } }
{ {  $x == m * m \wedge y == m * m \wedge m > 0$  } }
{ {  $\text{var } z := m;$  } }
{ {  $x == m * m \wedge y == m * m \wedge z > 0$  } }
  while (z > 0)
  { {  $x == m * m \wedge y == m * m \wedge z > 0$  } }
  { {  $x == m * m \wedge y + x == m * m \wedge z > 0$  } }
  { {  $x := m * m \wedge y + x == m * m \wedge z > 0$  } }
  { {  $z := z - 1;$  } }
  { {  $x == m * m \wedge y + x == m * m \wedge z > 0$  } }
  { {  $y := y + x$  } }
  { {  $x == m * m \wedge y == m * m \wedge z > 0$  } }
  }
{ {  $x == m * m \wedge y == m * m \wedge z == 0$  } }
{ { y = m * m * m } }  $\rightarrow$ 
```

Question 5 (20 points)

Implement the following Haskell functions:

zipper?

- (a) Implement a function `weave` that given two lists with elements of the same type, returns a list with elements alternating between the two lists. For example:

`weave [1,2,3] [4,5,6] = [1,4,2,5,3,6]`

zip $\Rightarrow [(1,4), (2,5), (3,6)]$

You can assume that the lists have the same length.

`weave [1,1] [1,2] =`

`foldl (\(x,y),acc) -> x,y:acc) [] (zip [1,1] [1,2])`

- (b) Implement a function `powerset`, that given a list of elements of some type, computes the list of all of its sublists, including the empty list and the original list itself, in any order. For example:

maybe
`powerset [1,2,3] = [[], [1,2,3], [1,2], [1,3], [1], [2,3], [2], [3], []]`

zip $[1,2]$

`powerset [] = [[]]`

`powerset [x] = [[]] ++ powerset [x]`

`powerset [x,y] = [x,y] ++ powerset [x] ++ powerset [y]`

`powerset [x,y,z] = [x,y,z] ++ powerset [x,y] ++ powerset [x,z] ++ powerset [y,z]`

Leah Orozco

UID

117874877

CMSC 433: Midterm Exam (Spring 2024)

Question 1 (20 points)

Consider the two standard folds in Haskell, `foldl` and `foldr`, whose definitions are given below:

```
foldr :: (a -> b -> b) -> b -> [a] -> b
foldr f b [] = b
foldr f b (x:xs) = f x (foldr f b xs)

foldl :: (b -> a -> b) -> b -> [a] -> b
foldl f b [] = b
foldl f b (x:xs) = foldl f (f b x) xs
```

Now consider the following two very similar but distinct folds over lists of integers:

```
f1 :: [Int] -> Int
f1 = foldr (-) 0

f2 :: [Int] -> Int
f2 = foldl (-) 0
```

Write two Dafny programs, one corresponding to each of `f1` and `f2`, but operating on arrays rather than lists. We've given you the method signatures to get you started:

method f1(a:array<Int>) returns (out:int) {	method f2(a:array<int>) returns (out:int) {
<pre>var i: int := a.length - 1; out := 0; while (i >= 0) { out := out - a[i]; i := i - 1; }</pre>	<pre>var i: int := 0; out := 0; while (i < a.length) { out := out - a[i]; i := i + 1; }</pre>

Question 2 (20 points)

For each of the Haskell expressions below, write their (most general) Haskell type or "ill-typed" if it contains a type error. The type signatures of all functions below are provided in the appendix at the end.

(a) $\backslash x \rightarrow x$

Example answer:

$a \rightarrow a$

(b) $\backslash x y \rightarrow (x,y)$

$a \rightarrow b \rightarrow (a,b)$

(c) $\backslash x y \rightarrow \text{if } x == y \text{ then show } x \text{ else show } (x,y)$

$a \rightarrow a \rightarrow \text{String}$

(d) $\backslash x l \rightarrow x : l ++ l ++ x$

$a \rightarrow [a] \rightarrow [a]$

(e) $\text{foldr} (\text{const } 1)$

$a \rightarrow a \rightarrow a$

(f) ("42")

$a \rightarrow (\text{String}, a)$

(g) ("42")

$a \rightarrow (a, \text{String})$

(h) $\text{reverse} . \text{reverse}$

$[a] \rightarrow [a]$

(i) $\backslash l \rightarrow \text{filter } (< 42) (l ++ l)$

$[Int] \rightarrow [Int]$

(j) $\text{let } f x = x \text{ in } (f 'a', f \text{True})$

$a \rightarrow \text{Bool}$

(k) $\text{fmap} (\text{fmap } (+1)) [\text{Nothing}]$

$([a])$

$a \rightarrow b$

Question 3 (20 points)

For each of the types below, write a Haskell expression that has that type. Don't write trivial expressions (such as [], Nothing, or undefined) unless there is no other option. You can use any function from the appendix, do syntax, list comprehensions, or any valid Haskell.

(a) $\text{Int} \rightarrow \text{Int}$

Example answers:

$\backslash x \rightarrow x + 1$

$(+1)$

(b) $\text{Bool} \rightarrow [\text{Bool}]$

$\backslash b \rightarrow \text{if } b \text{ then } [\text{true}] \text{ else } [\text{false}]$

(c) $a \rightarrow \text{Maybe } b$

undefined

(d) $(\text{Int} \rightarrow \text{Char} \rightarrow \text{Bool}) \rightarrow [\text{Int}] \rightarrow [\text{Char}] \rightarrow [\text{Bool}]$

$\backslash f \ (i:ei) \ (c:ec) \rightarrow (f \ i \ c) : []$

(e) $(a \rightarrow b \rightarrow c) \rightarrow \text{Maybe } a \rightarrow \text{Maybe } b \rightarrow \text{Maybe } c$

$\backslash f \ a \ b \rightarrow \text{if } a \neq \text{Nothing} \text{ then if } b \neq \text{Nothing} \text{ then } f \ a \ b \text{ else Nothing}$

(f) $(a \rightarrow b) \rightarrow (b \rightarrow \text{Bool}) \rightarrow [a] \rightarrow [b]$

$\backslash f1 \ f2 \ (a:as) \rightarrow \text{if } f2(f1 \ a) \text{ then } f1 \ a \text{ else } f1 \ a$

(g) $\text{Maybe } a \rightarrow (a \rightarrow [b]) \rightarrow [(a,b)]$

$\backslash a \ f \rightarrow \text{if } a \neq \text{Nothing} \text{ then } (a, \text{head } f \ a) : [] \text{ else } []$

(h) $\text{Eq } a \Rightarrow a \rightarrow [a] \rightarrow [a]$

$\backslash x \ \ell \rightarrow x : \ell$

(i) $\text{Show } a \Rightarrow [a] \rightarrow \text{IO String}$

$\backslash \ell s \rightarrow \text{putStrLn}(\text{head } \ell s)$

(j) $(a,b) \rightarrow (a \rightarrow b \rightarrow c) \rightarrow c$

$\backslash (a,b) \ f \rightarrow f \ a \ b$

(k) $((a,b) \rightarrow c) \rightarrow c$

undefined

Question 4 (20 points)

Consider the following Dafny program that inefficiently calculates the cube of a number:

```
method Cube(m:int) returns (y:int)
  requires m > 0
  ensures y == m*m*m
{
  y := 0;
  var x := m * m;
  var z := m;
  while (z > 0)
  {
    z := z - 1;
    y := y + x;
  }
}
```

Fill in the annotations in the following program to show that the Hoare triple given by the outermost pre- and post- conditions is valid. Be completely precise and pedantic in the way you apply Hoare rules—write assertions in *exactly* the form given by the rules rather than just logically equivalent ones. The provided blanks have been constructed so that if you work backwards from the end of the program you should only need to use the rule of consequence in the places indicated with $\rightarrow>$. These implication steps can (silently) rely on all the usual rules of arithmetic.

```
{f m > 0 } }  $\rightarrow>$ 
{f 0 == (m * m) (m - m) } }
  y := 0;
  {f y == (m * m) (m - m) } }
    var x := m * m;
    {f y == x * (m - m) } }
    var z := m;
    {f y == x * (m - z) } }
    while (z > 0) {
      {f y == x * (m - z) } }
      {f y + x == x * (m - z - 1) } }
      {f y + x == x * (m - z) } }
      {f y + x == x * (m - z) } }
      {f y == x * (m - z) } }
    }
  {f y == x * (m - z) } }
  {f y == m * m * m } }
```

$$4 \cdot 3 = 12$$

$$y = 0 \quad m = 4$$

$$x = 16$$

$$z = 4$$

$$z = 0$$

$$y = 64$$

$$16 \cdot 4$$

$$z = 1$$

$$y = 48$$

$$16 \cdot 3$$

$$z = 2$$

$$y = 32$$

$$16 \cdot 2$$

$$z = 3$$

$$y = 16$$

$$16 \cdot 1$$

$$y = x * (m - z)$$

Question 5 (20 points)

Implement the following Haskell functions:

- (a) Implement a function `weave` that given two lists with elements of the same type, returns a list with elements alternating between the two lists. For example:

`weave [1,2,3] [4,5,6] = [1,4,2,5,3,6]`

You can assume that the lists have the same length.

`weave :: [a] -> [a] -> [a]`

`weave [] [] = []`

`weave xs ys = weaveAux xs ys 0` where

`weaveAux (a:as) [] _ = as`

`weaveAux [] (b:bs) _ = bs`

`weaveAux (a:as) bs 0 = a:weaveAux as bs 1`

`weaveAux as (b:bs) 1 = b:weaveAux as bs 0`

- (b) Implement a function `powerSet`, that given a list of elements of some type, computes the list of all of its sublists, including the empty list and the original list itself, in any order. For example:

`powerSet [1,2,3] = [[], [1,2,3], [1,2], [1,3], [1], [2,3], [2], [3], []]`

`powerSet [] = [[]]`

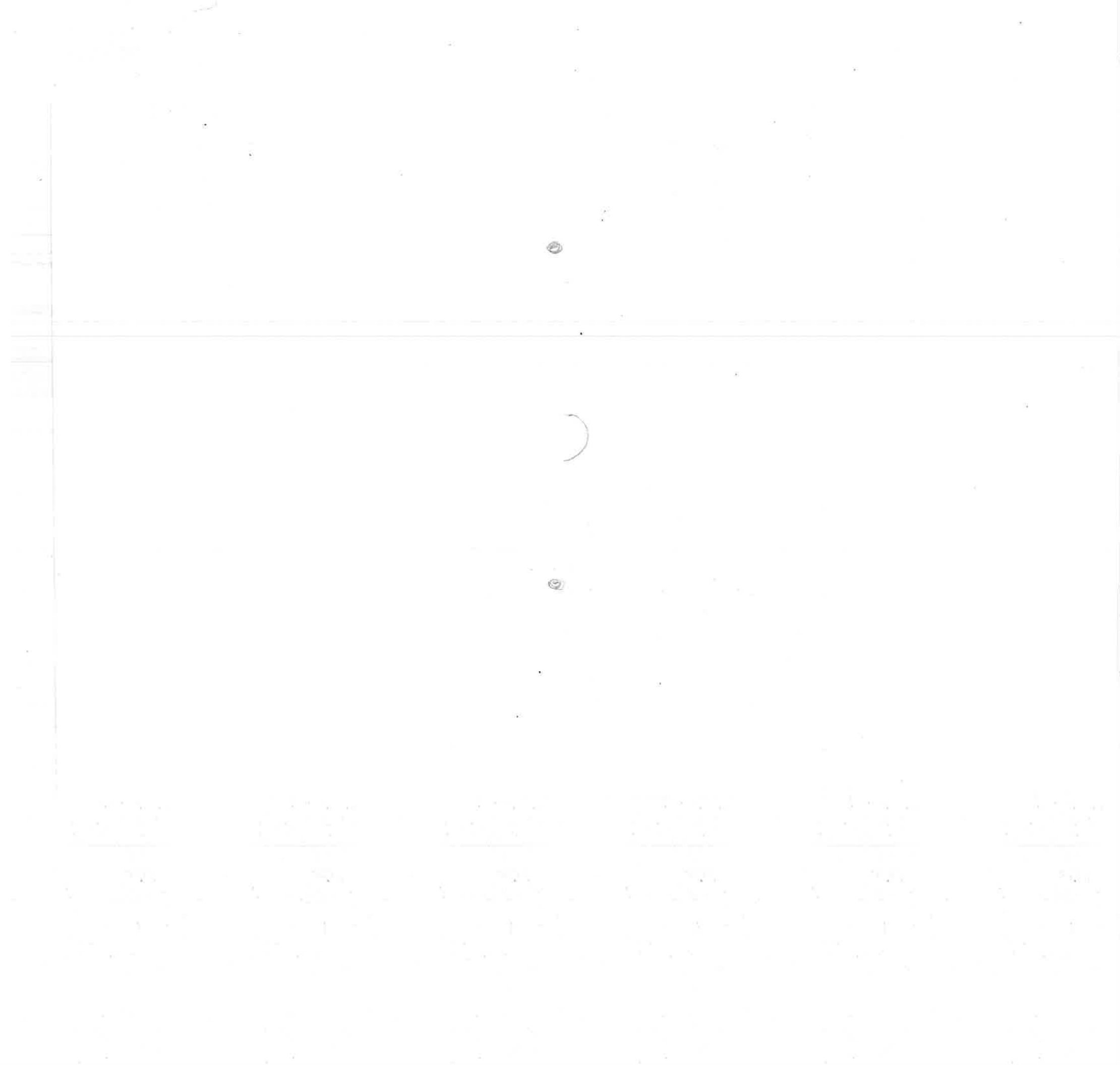
`powerSet [1] = [[]]`

`powerSet lst = foldr (\h acc ->`

`foldr (\x acc -> [x] ++ acc)`

`[] (tail lst)`

`) [[]] lst`



Jaeheang Kim

CMSC 433: Midterm Exam (Spring 2024)

Question 1 (20 points)

Consider the two standard folds in Haskell, `foldl` and `foldr`, whose definitions are given below:

```
foldr :: (a -> b -> b) -> b -> [a] -> b
foldr f b [] = b
foldr f b (x:xs) = f x (foldr f b xs)

foldl :: (b -> a -> b) -> b -> [a] -> b
foldl f b [] = b
foldl f b (x:xs) = foldl f (f b x) xs
```

Now consider the following two very similar but distinct folds over lists of integers:

```
f1 :: [Int] -> Int
f1 = foldr (-) 0
```

```
f2 :: [Int] -> Int
f2 = foldl (-) 0
```

Write two Dafny programs, one corresponding to each of `f1` and `f2`, but operating on arrays rather than lists. We've given you the method signatures to get you started:

```
method f1(a:array<int>) returns (out:int) {
```

```
  var out := 0;
  int i := a.length - 1;
```

```
  while (i >= 0) {
```

```
    out := out - a[i];
    i := i - 1;
  }
```

```
}
```

```
method f2(a:array<int>) returns (out:int) {
```

```
  var out := 0;
  int i := 0;
```

```
  while (i < a.length) {
```

```
    out := out - a[i];
    i := i + 1;
  }
```

```
}
```

```
}
```

```
}
```

Question 2 (20 points)

For each of the Haskell expressions below, write their (most general) Haskell type or "ill-typed" if it contains a type error. The type signatures of all functions below are provided in the appendix at the end.

(a) $\backslash x \rightarrow x$

Example answer:

$a \rightarrow a$

(b) $\backslash x y \rightarrow (x, y)$

$a \rightarrow b \rightarrow (a, b)$

(c) $\backslash x y \rightarrow \text{if } x == y \text{ then show } x \text{ else show } (x, y)$

$a \rightarrow a \rightarrow \text{string}$

(d) $\backslash x l \rightarrow x : 1 ++ 1 ++ [x]$

$a \rightarrow [a] \rightarrow [a]$

(e) `foldr (const 1)`

ill-typed

(f) `(, 42)`

(a, Int)

(g) `(,) "42"`

ill-typed

(h) `reverse . reverse`

$([a] \rightarrow [a]) \rightarrow [a] \rightarrow [a]$

(i) $\backslash l \rightarrow \text{filter } (< 42) (1 ++ 1)$

$[\text{Int}] \rightarrow [\text{Int}]$

(j) `let f x = x in (f 'a', f True)`

ill-type

(k) `fmap (fmap (+1)) [Nothing]`

*fold (a) :
Int -> [Int]*

Question 3 (20 points)

For each of the types below, write a Haskell expression that has that type. Don't write trivial expressions (such as [], Nothing, or undefined) unless there is no other option. You can use any function from the appendix: do syntax, list comprehensions, or any valid Haskell.

(a) `Int -> Int`

Example answers:

`\x -> x + 1`

`(+1)`

(b) `Bool -> [Bool]`

`\x -> if x then [x] else [!x]`

(c) `a -> Maybe b`

`\x -> Just x`

(d) `(Int -> Char -> Bool) -> [Int] -> [Char] -> [Bool]`

`zipWith (\xy -> x == y) [1..3] [a..c]`

(e) `(a -> b -> c) -> Maybe a -> Maybe b -> Maybe c`

`zipWith`

(f) `(a -> b) -> (b -> Bool) -> [a] -> [b]`

(g) `Maybe a -> (a -> [b]) -> [(a,b)]`

(h) `Eq a => a -> [a] -> [a]`

`\x 1 -> if [Eq] == x then x : 1 else 1`

(i) `Show a => [a] -> IO String`

`\1 -> print 1`

(j) `(a,b) -> (a -> b -> c) -> c`

(k) `((a,b) -> c) -> c`

Question 4 (20 points)

Consider the following Dafny program that inefficiently calculates the cube of a number:

```
method Cube(m:int) returns (y:int)
  requires m > 0
  ensures y == m*m*m
{
  y := 0;
  var x := m * m;
  var z := m;
  while (z > 0)
  {
    z := z - 1;
    y := y + x;
  }
}
```

Handwritten annotations for the program above:

- Below `y := 0;`: `y = 0`
- Below `var x := m * m;`: `9`
- Below `var z := m;`: `18`
- Below `while (z > 0)`: `21`
- Below the first `z := z - 1;`: `2`
- Below the first `y := y + x;`: `3`
- Below the second `z := z - 1;`: `2`
- Below the second `y := y + x;`: `1`
- Below the third `z := z - 1;`: `0`

Fill in the annotations in the following program to show that the Hoare triple given by the outermost pre- and post- conditions is valid. Be completely precise and pedantic in the way you apply Hoare rules--write assertions in *exactly* the form given by the rules rather than just logically equivalent ones. The provided blanks have been constructed so that if you work backwards from the end of the program you should only need to use the rule of consequence in the places indicated with $\rightarrow>$. These implication steps can (silently) rely on all the usual rules of arithmetic.

```
{ { m > 0 } }  $\rightarrow>$ 
{ {
  { {
    var x := m * m;
    var z := m;
    { { z >= m } }
    { { z >= 0 } }  $\wedge$   $y = 0$  }  $\wedge$   $x = m^2$ 
    while (z > 0) {
      { { z > 0 } }  $\wedge$   $y = m * m * m$ 
      { { z - 1 >= 0 } }  $\wedge$   $y + x = m * m * m$ 
      { { z >= 0 } }  $\wedge$   $y + x = m * m * m$ 
      y := y + x;
    }
    { { z >= 0 } }  $\wedge$   $y = m * m * m$ 
    { { !(z > 0) } }  $\wedge$   $y = m * m * m$ 
  } }  $\rightarrow>$ 
}
```

Handwritten annotations for the program above:

- Below `var x := m * m;`: `9`
- Below `var z := m;`: `18`
- Below `{ { z >= m } }`: `3`
- Below `{ { z >= 0 } }`: `2`
- Below `{ { z >= 0 } }`: `1`
- Below `{ { z >= 0 } }`: `0`

$$z = 0 \wedge y = 4$$

Question 5 (20 points)

Implement the following Haskell functions:

- (a) Implement a function `weave` that given two lists with elements of the same type, returns a list with elements alternating between the two lists. For example:

`weave [1,2,3] [4,5,6] = [1,4,2,5,3,6]`

You can assume that the lists have the same length.

`weave :: [a] -> [a] -> [a]`

`weave [] [] = []`

`weave (x:xs) (y:ys) = x:y : weave xs ys`

- (b) Implement a function `powerset`, that given a list of elements of some type, computes the list of all of its sublists, including the empty list and the original list itself, in any order. For example:

`powerset [1,2,3] = [[1,2,3], [1,2], [1,3], [1], [2,3], [2], [3], []]`

`powerset :: [a] -> [[a]]`

`powerset [] = []`

`x:xs`

`powerset (x:xs) = x : powerset (xs) : powerset (xs)`

1 2 3

1 2 3 : 2 3 : 3

1 2 3 : 2 3

1 3

2 3

Alex Csontos

118126364

CMSC 433: Midterm Exam (Spring 2024)

Question 1 (20 points)

Consider the two standard folds in Haskell, `foldl` and `foldr`, whose definitions are given below:

```
foldr :: (a -> b -> b) -> b -> [a] -> b
foldr f b [] = b
foldr f b (x:xs) = f x (foldr f b xs)
```

```
foldl :: (b -> a -> b) -> b -> [a] -> b
foldl f b [] = b
foldl f b (x:xs) = foldl f (f b x) xs
```

Now consider the following two very similar but distinct folds over lists of integers:

```
f1 :: [Int] -> Int
f1 = foldr (-) 0
```

```
f2 :: [Int] -> Int
f2 = foldl (-) 0
```

Write two Dafny programs, one corresponding to each of `f1` and `f2`, but operating on arrays rather than lists. We've given you the method signatures to get you started:

<pre>method f1(a:array<int>) returns (out:int) { var i:int := a.length-1; while (i >= 0) { out := out - a[i]; i := i - 1; } }</pre>	<pre>method f2(a:array<int>) returns (out:int) { var i:int := 0; while (i < a.length) out := out - a[i]; i := i + 1; } }</pre>
--	---

Question 2 (20 points)

For each of the Haskell expressions below, write their (most general) Haskell type or "ill-typed" if it contains a type error. The type signatures of all functions below are provided in the appendix at the end.

(a) $\backslash x \rightarrow x$

Example answer:

$a \rightarrow a$

(b) $\backslash x y \rightarrow (x,y)$

$a \rightarrow b \rightarrow (a,b)$

(c) $\backslash x y \rightarrow \text{if } x == y \text{ then show } x \text{ else show } (x,y)$

$a \rightarrow a \rightarrow a$

(d) $\backslash x l \rightarrow x : l ++ l ++ [x]$

$a \rightarrow [a] \rightarrow [a]$

(e) foldr (const 1)

ill-typed

(f) ("42")

$(\text{Nothing}, \text{String})$

(g) $(.) \text{"42"}$

$a \rightarrow (a, \text{String})$

(h) reverse . reverse

$[a] \rightarrow [a]$

(i) $\backslash l \rightarrow \text{filter } (< 42) (l ++ l)$

$(a \rightarrow \text{Bool}) \rightarrow [a] \rightarrow [a]$

(j) $\text{let } f \ x = x \text{ in } (f \ 'a', f \ \text{True})$

$a \rightarrow (\text{Char}, \text{Bool})$

(k) $\text{fmap (fmap (+1)) [Nothing]}$

$[[\text{Int} \rightarrow \text{Int}] \rightarrow [\text{Maybe } a]] \rightarrow [\text{Int}]$

Question 3 (20 points)

For each of the types below, write a Haskell expression that has that type. Don't write trivial expressions (such as []). Nothing, or undefined) unless there is no other option. You can use any function from the appendix; do syntax, list comprehensions, or any valid Haskell.

(a) $\text{Int} \rightarrow \text{Int}$

Example answers:

$\backslash x \rightarrow x + 1$

$(+1)$

(b) $\text{Bool} \rightarrow [\text{Bool}]$

$\text{if } x \text{ then } [x] \text{ else } [\text{False}]$

(c) $a \rightarrow \text{Maybe } b$

$\text{Just } x$

(d) $(\text{Int} \rightarrow \text{Char} \rightarrow \text{Bool}) \rightarrow [\text{Int}] \rightarrow [\text{Char}] \rightarrow [\text{Bool}]$

(e) $(a \rightarrow b \rightarrow c) \rightarrow \text{Maybe } a \rightarrow \text{Maybe } b \rightarrow \text{Maybe } c$

$\text{let } f \ x \ y =$

(f) $(a \rightarrow b) \rightarrow (b \rightarrow \text{Bool}) \rightarrow [a] \rightarrow [b]$

(g) $\text{Maybe } a \rightarrow (a \rightarrow [b]) \rightarrow [(a.b)]$

(h) $\text{Eq } a \Rightarrow a \rightarrow [a] \rightarrow [a]$

$\backslash x \rightarrow \text{intersperse } x \ /$

(i) $\text{Show } a \Rightarrow [a] \rightarrow \text{IO String}$

$\text{if } (\text{length } x) > 0 \text{ then } "yes" \text{ else } "no"$

(j) $(a.b) \rightarrow (a \rightarrow b \rightarrow c) \rightarrow c$

(k) $((a.b) \rightarrow c) \rightarrow c$

Question 4 (20 points)

Consider the following Dafny program that inefficiently calculates the cube of a number:

```
method Cube(m:int) returns (y:int)
  requires m > 0
  ensures y == m*m*m
{
  y := 0;
  var x := m * m;
  var z := m;
  while (z > 0)
  {
    z := z - 1;
    y := y + x;
  }
}
```

Fill in the annotations in the following program to show that the Hoare triple given by the outermost pre- and post-conditions is valid. Be completely precise and pedantic in the way you apply Hoare rules—write assertions in *exactly* the form given by the rules rather than just logically equivalent ones. The provided blanks have been constructed so that if you work backwards from the end of the program you should only need to use the rule of consequence in the places indicated with $\rightarrow$. These implication steps can (silently) rely on all the usual rules of arithmetic.

```
{ { m > 0 } }  $\rightarrow$ 
{ { 0 = m * m * m && z >= 0 } }
{ { y := 0; } }
{ { y = m * m * m && z >= 0 } }
{ { var x := m * m; } }
{ { y = m * x && z >= 0 } }
{ { var z := m; } }
{ { y = z * x && z >= 0 } }
while (z > 0) {
  { { y = z * x && z >= 0 } }
  { { y + x = (z - 1) * x && z >= 0 } }
  { { z := z - 1; } }
  { { y + x = z * x && z >= 0 } }
  { { y := y + x; } }
  { { y = z * x && z >= 0 } }
}
{ { y = z * x && z >= 0 } }
{ { y := z * x && z >= 0 && (z > 0) } }
{ { y = m * m * m } }  $\rightarrow$ 
```


Question 5 (20 points)

Implement the following Haskell functions:

- (a) Implement a function `weave` that given two lists with elements of the same type, returns a list with elements alternating between the two lists. For example:

`weave [1,2,3] [4,5,6] = [1,4,2,5,3,6]`

You can assume that the lists have the same length.

`weave :: [Int] → [Int] → [Int]`

`weave [] _ = []`

`weave _ [] = []`

`weave (x1:xs) (y:ys) = [x,y] ++ (weave xs ys)`

- (b) Implement a function `powerset`, that given a list of elements of some type, computes the list of all of its sublists, including the empty list and the original list itself, in any order. For example:

`powerset [1,2,3] = [[1,2,3], [1,2], [1,3], [1], [2,3], [2], [3], []]`

`powerset [] = [[]]`

`powerset [ ] = [ ]`

`powerset (x:xs) = (x:xs) : (powerset xs) ++ (powerset x)`

Priyal Garg

CMSC 433: Midterm Exam (Spring 2024)

Question 1 (20 points)

Consider the two standard folds in Haskell, `foldl` and `foldr`, whose definitions are given below:

```
foldr :: (a -> b -> b) -> b -> [a] -> b
foldr f b [] = b
foldr f b (x:xs) = f x (foldr f b xs)

foldl :: (b -> a -> b) -> b -> [a] -> b
foldl f b [] = b
foldl f b (x:xs) = foldl f (f b x) xs
```

$f \ 0 \ [2,3] \quad f \ 1 \ (f \ 2 \ (f \ 3 \ (0)))$
 $(1 - (2 + (3 - 0)))$
 $1 - (2 - 3) = 1 - 1 = 2$

Now consider the following two very similar but distinct folds over lists of integers:

```
f1 :: [Int] -> Int
f1 = foldr (-) 0
```

$[1, 2, 3]$
 $f : (f \ 0 \ 3) \ 2 \ 1$
 -3

```
f2 :: [Int] -> Int
f2 = foldl (-) 0
```

-5
 -6

Write two Dafny programs, one corresponding to each of `f1` and `f2`, but operating on arrays rather than lists. We've given you the method signatures to get you started:

```
method f1(a:array<int>) returns (out:int) {
  out := 0;
  var i := a.length - 1;
  while (i >= 0) {
    out := a[i] - out;
    i := i - 1;
  }

  method f2(a:array<int>) returns (out:int) {
    out := 0
    var i := 0
    while (i < a.length) {
      out := out - a[i];
      i := i + 1;
    }
  }
}
```

$out = 3$
 $out = 2 - 3 = -1$
 $out = 1 - (-1) = 2$

Question 2 (20 points)

For each of the Haskell expressions below, write their (most general) Haskell type or "ill-typed" if it contains a type error. The type signatures of all functions below are provided in the appendix at the end.

(a) $\backslash x \rightarrow x$

Example answer:

$a \rightarrow a$

(b) $\backslash x y \rightarrow (x, y)$

$a \rightarrow b \rightarrow (a, b)$

(c) $\backslash x y \rightarrow \text{if } x == y \text{ then show } x \text{ else show } (x, y)$

$a \rightarrow a \rightarrow \text{string}$

(d) $\backslash x l \rightarrow x : l ++ l ++ [x]$

$a \rightarrow [a] \rightarrow [a]$

(e) $\text{foldr} (\text{const } 1)$

$b \rightarrow t \ a \rightarrow b$

(f) ("42")

(string)

(g) ("42")

ill-typed

(h) $\text{reverse} . \text{reverse}$

$[a] \rightarrow [a] \rightarrow [a]$

(i) $\backslash l \rightarrow \text{filter } (< 42) (l ++ l)$

$a \rightarrow [a]$

(j) $\text{let } f \ x = x \text{ in } (f \ 'a', f \ \text{True})$

$(\text{char}, [\text{bool}])$

(k) $\text{fmap} (\text{fmap } (+1)) [\text{Nothing}]$

ill-typed

$\text{fmap } (r1)$

$f \ \text{Int} \rightarrow f \ \text{Int}$

Question 3 (20 points)

For each of the types below, write a Haskell expression that has that type. Don't write trivial expressions (such as `[]`, `Nothing`, or `undefined`) unless there is no other option. You can use any function from the appendix, do syntax: list comprehensions, or any valid Haskell.

- (a) `Int -> Int`
Example answers:
`\x -> x + 1`
`(+1)`
- (b) `Bool -> [Bool]`
`\x -> if x == True then [x] else [x]`
- (c) `a -> Maybe b`
`undefined`
- (d) `(Int -> Char -> Bool) -> [Int] -> [Char] -> [Bool]`
`\ f i c -> zipWith f [5:i] [a':c]`
- (e) `(a -> b -> c) -> Maybe a -> Maybe b -> Maybe c`
`\ f a b -> first (maybeToList f (first (maybeToList a) (first (maybeToList b))))`
- (f) `(a -> b) -> (b -> Bool) -> [a] -> [b]`
`\ f1 f2 lst = takeWhile f2 (map f1 lst)`
- (g) `Maybe a -> (a -> [b]) -> [(a,b)]`
`\ m f -> case m like Just x -> [(x, first (f x))]`
- (h) `Eq a => a -> [a] -> [a]`
`\ m lst -> first (group x lst)`
- (i) `Show a => [a] -> IO String`
`\ lst -> map show lst`
- (j) `(a,b) -> (a -> b -> c) -> c`
`\ t f -> case t of (a,s) -> f a b`
- (k) `((a,b) -> c) -> c`
`undefined`

Question 4 (20 points)

Consider the following Dafny program that inefficiently calculates the cube of a number:

```
method Cube(m:int) returns (y:int)
  requires m > 0
  ensures y == m*m*m
{
  y := 0;
  var x := m * m;
  var z := m;
  while (z > 0)
  {
    z := z - 1;
    y := y + x;
  }
}
```

Fill in the annotations in the following program to show that the Hoare triple given by the outermost pre- and post- conditions is valid. Be completely precise and pedantic in the way you apply Hoare rules—write assertions in *exactly* the form given by the rules rather than just logically equivalent ones. The provided blanks have been constructed so that if you work backwards from the end of the program you should only need to use the rule of consequence in the places indicated with $\rightarrow>$. These implication steps can (silently) rely on all the usual rules of arithmetic.

```
{ { m > 0 } }  $\rightarrow>$ 
{ { m > 0 } }  $\wedge$   $m \cdot m = m \cdot m$   $\wedge$   $0 = (m \cdot m) \cdot (m - m)$  } }
  y := 0;
  { { m > 0 } }  $\wedge$   $m \cdot m = m \cdot m$   $\wedge$   $y = (m \cdot m) \cdot (m - m)$  } }
    var x := m * m;
    { { m > 0 } }  $\wedge$   $x = m \cdot m$   $\wedge$   $y = x \cdot (m - m)$  } }
      var z := m;
      { { z > 0 } }  $\wedge$   $x = m \cdot m$   $\wedge$   $y = x \cdot (m - z)$  } }
        while (z > 0) {
          { { z > 0 } }  $\wedge$   $x = m \cdot m$   $\wedge$   $y = x \cdot (m - z)$  } }  $\rightarrow>$ 
          { { z - 1 > 0 } }  $\wedge$   $x = m \cdot m$   $\wedge$   $(y + x) = x \cdot (m - (z - 1))$  } }
            z := z - 1;
            { { z > 0 } }  $\wedge$   $x = m \cdot m$   $\wedge$   $(y + x) = x \cdot (m - z)$  } }
              y := y + x;
              { { z > 0 } }  $\wedge$   $x = m \cdot m$   $\wedge$   $y = x \cdot (m - z)$  } }
            }
          { { ! (z > 0) } }  $\wedge$   $z = 0$   $\wedge$   $x = m \cdot m$   $\wedge$ 
            { { y = m * m * m } }  $\wedge$   $y = x \cdot (m - z)$  } }  $\rightarrow>$ 
```

Handwritten annotations for the Hoare triple:

x	y	z
$m \cdot m$	0	m
$m \cdot m$	$m \cdot m$	$m - 1$
	$2m \cdot m$	$m - 2$
	$(m - 1)m \cdot m$	1
	$m \cdot m \cdot m$	0
$y = m \cdot m \cdot m, (m - z)$		

Question 5 (20 points)

Implement the following Haskell functions:

- (a) Implement a function `weave` that given two lists with elements of the same type, returns a list with elements alternating between the two lists. For example:

`weave [1,2,3] [4,5,6] = [1,4,2,5,3,6]`

You can assume that the lists have the same length.

`weave [] = []`

`weave [] = []`

`weave (x:xs) (l:ls) = x:[l] ++ weave xs ls`

- (b) Implement a function `powerset`, that given a list of elements of some type, computes the list of all of its sublists, including the empty list and the original list itself, in any order. For example:

`powerset [1,2,3] = [[1,2,3], [1,2], [1,3], [1], [2,3], [2], [3], []]`

```
foldAux acc x = acc ++ (map (\a -> x:a) acc)
powerset lst = foldl foldAux [[]] lst

[] []
[] [1]
[] [1] [2]
[] [1] [2] [3]
```

```
f acc x
f acc x = acc ++ (map (\a -> x:a) acc)
```


Darya Hejbl

CMSC 433: Midterm Exam (Spring 2024)

Question 1 (20 points)

Consider the two standard folds in Haskell, `foldl` and `foldr`, whose definitions are given below:

```
foldr :: (a -> b -> b) -> b -> [a] -> b
foldr f b [] = b
foldr f b (x:xs) = f x (foldr f b xs)

foldl :: (b -> a -> b) -> b -> [a] -> b
foldl f b [] = b
foldl f b (x:xs) = foldl f (f b x) xs
```

Now consider the following two very similar but distinct folds over lists of integers:

```
f1 :: [Int] -> Int
f1 = foldr (-) 0 from back

f2 :: [Int] -> Int
f2 = foldl (-) 0 from front
```

Write two Dafny programs, one corresponding to each of `f1` and `f2`, but operating on arrays rather than lists. We've given you the method signatures to get you started:

<pre>method f1(a:array<int>) returns (out:int) { var i := 0; while (i < a.length) { out = out - a[i]; i = i + 1; } return out; }</pre>	<pre>method f2(a:array<int>) returns (out:int) { var i := 0; while (i < a.length) { out = out - a[i]; i = i + 1; } return out; }</pre>
---	---

Question 2 (20 points)

For each of the Haskell expressions below, write their (most general) Haskell type or "ill-typed" if it contains a type error. The type signatures of all functions below are provided in the appendix at the end.

(a) $\lambda x \rightarrow x$

Example answer:

$a \rightarrow a$

(b) $\lambda x y \rightarrow (x, y)$

$a \rightarrow b \rightarrow (a, b)$

(c) $\lambda x y \rightarrow \text{if } x == y \text{ then show } x \text{ else show } (x, y)$

ill typed

(d) $\lambda x l \rightarrow x : l ++ l ++ [x]$

$a \rightarrow [a] \rightarrow [a]$

(e) $\text{foldr } (\text{const } 1)$

$a \rightarrow [a] \rightarrow a$

~~$\lambda (f) \rightarrow 42$~~

$a \rightarrow (a, \text{String})$

~~$\lambda (g) \rightarrow 42$~~

$b \rightarrow (\text{String}, b)$

(h) $\text{reverse} . \text{reverse} \quad [a] \rightarrow [a]$

$[a] \rightarrow [a]$

(i) $\lambda l \rightarrow \text{filter } (< 42) (l ++ l)$

$[Int] \rightarrow [Int]$

(j) $\text{let } f \ x = x \text{ in } (f 'a', f \text{ True})$

ill typed

(k) $\text{fmap } (\text{fmap } (+1)) [\text{Nothing}]$

$a \rightarrow b$

$a \rightarrow b \rightarrow$

Question 3 (20 points)

For each of the types below, write a Haskell expression that has that type. Don't write trivial expressions (such as `[]`, `Nothing`, or `undefined`) unless there is no other option. You can use any function from the appendix, do syntax, list comprehensions, or any valid Haskell.

(a) `Int -> Int`

Example answers:

`\x -> x + 1`
`(+1)`

(b) `Bool -> [Bool]`

repeat True

— (c) `a -> Maybe b`

undefined

(d) `(Int -> Char -> Bool) -> [Int] -> [Char] -> [Bool]`

(e) `(a -> b -> c) -> Maybe a -> Maybe b -> Maybe c`

(f) `(a -> b) -> (b -> Bool) -> [a] -> [b]`

(g) `Maybe a -> (a -> [b]) -> [(a,b)]`

Just a

(h) `Eq a => a -> [a] -> [a]`

\x

(i) `Show a => [a] -> IO String`

\a -> putStrLn read a

(j) `(a,b) -> (a -> b -> c) -> c`

\(a,b)

->

(k) `((a,b) -> c) -> c`

Question 5 (20 points)

Implement the following Haskell functions:

- (a) Implement a function `weave` that given two lists with elements of the same type, returns a list with elements alternating between the two lists. For example:

```
weave [1,2,3] [4,5,6] = [1,4,2,5,3,6]
```

You can assume that the lists have the same length.

```
weave [] = []
```

```
weave [] = []
```

```
weave (x:xs) (ys:ys) = x:ys:weave xs ys
```

- (b) Implement a function `powerset`, that given a list of elements of some type, computes the list of all of its sublists, including the empty list and the original list itself, in any order. For example:

```
powerset [1,2,3] = [[], [1,2,3], [1,2], [1,3], [1], [2,3], [2], [3], []]
```

```
powerset [] = []
```

```
powerset (x:xs) = x: powerset xs
```


CMSC 433: Midterm Exam (Spring 2024)

Question 1 (20 points)

Consider the two standard folds in Haskell, `foldl` and `foldr`, whose definitions are given below:

```
foldr :: (a -> b -> b) -> b -> [a] -> b
foldr f b [] = b
foldr f b (x:xs) = f x (foldr f b xs)

foldl :: (b -> a -> b) -> b -> [a] -> b
foldl f b [] = b
foldl f b (x:xs) = foldl f (f b x) xs
```

Now consider the following two very similar but distinct folds over lists of integers:

```
f1 :: [Int] -> Int
f1 = foldr (-) 0

f2 :: [Int] -> Int
f2 = foldl (-) 0
```

Write two Dafny programs, one corresponding to each of `f1` and `f2`, but operating on arrays rather than lists. We've given you the method signatures to get you started:

<pre>method f1(a:array<int>) returns (out:int) { var i := 0; for j := a.length to 0 { i := i - a[j]; } out := i; }</pre>	<pre>method f2(a:array<int>) returns (out:int) { var i := 0; for j := 0 to a.length-1 { i := i - a[j]; } out := i; }</pre>
--	--

Question 2 (20 points)

For each of the Haskell expressions below, write their (most general) Haskell type or "ill-typed" if it contains a type error. The type signatures of all functions below are provided in the appendix at the end.

(a) $\backslash x \rightarrow x$

Example answer:

$a \rightarrow a$

(b) $\backslash x y \rightarrow (x,y)$

$a \rightarrow b \rightarrow (a,b)$

(c) $\backslash x y \rightarrow$ if $x == y$ then show x else show (x,y)

ill typed

(d) $\backslash x l \rightarrow x : l ++ l ++ l ++ x$

$a \rightarrow [a] \rightarrow [a]$

(e) $\text{foldr} (\text{const } 1) \leftarrow$

$\text{Int} \rightarrow [\text{Int}] \rightarrow \{\text{Int}\}$

I am assuming that this is the number 1

(f) ("42")

$a \rightarrow (a, \text{string})$

(g) ("42")

$a \rightarrow (\text{string}, a)$

(h) $\text{reverse} . \text{reverse}$

$[a] \rightarrow [a]$

(i) $\backslash l \rightarrow \text{filter} (< 42) (l ++ l)$

$\text{Num } a \Rightarrow [a] \rightarrow [a]$

(j) $\text{let } f x = x \text{ in } (f 'a', f \text{True})$

ill typed

(k) $\text{fmap} (\text{fmap } (+1)) [\text{Nothing}]$

$\text{Num } a \Rightarrow (f a \rightarrow f a) \rightarrow \text{Maybe } f a \rightarrow \text{Maybe } f a$

Question 3 (20 points)

For each of the types below, write a Haskell expression that has that type. Don't write trivial expressions (such as `[]`, `Nothing`, or `undefined`) unless there is no other option. You can use any function from the `appendix`, `do` syntax, `list comprehensions`, or any valid Haskell.

(a) `Int -> Int`

Example answers:

`\x -> x + 1`
`(+1)`

(b) `Bool -> [Bool]`

`\x -> [x || True]`

(c) `a -> Maybe b`

`foo a = Nothing`

(d) `(Int -> Char -> Bool) -> [Int] -> [Char] -> [Bool]`

`foo = zipWith`

(e) `(a -> b -> c) -> Maybe a -> Maybe b -> Maybe c`

`foo f ma mb = ma >>= \a -> mb >>= \b -> (return f a b)`

(f) `(a -> b) -> (b -> Bool) -> [a] -> [b]` `foo f g lst = let xs = map f lst in filter g xs`

(g) `Maybe a -> (a -> [b]) -> [(a,b)]` `foo ma f = let lst = ma >>= \a -> f a`

`in ma >>= \a' -> map (\x -> (a', x)) lst`

(h) `Eq a => a -> [a] -> [a]` `foo a lst = map (\x -> x == a) lst`

(i) `Show a => [a] -> IO String` `foo (x : xs) = putStrLn x`

(j) `(a,b) -> (a -> b -> c) -> c` `foo (a,b) f = f a b`

(k) `((a,b) -> c) -> c` `foo f = undefined`

Question 4 (20 points)

Consider the following Dafny program that inefficiently calculates the cube of a number:

```
method Cube(m:int) returns (y:int)
  requires m > 0
  ensures y == m*m*m
{
  y := 0;
  var x := m * m;
  var z := m;
  while (z > 0)
  {
    z := z - 1;
    y := y + x;
  }
}
```

$m = 3$
 $y = 0$
 $x = 9$
 $z = 3$

 $z = 2$
 $y = 9$
 $z = 1$
 $y = 18$

Fill in the annotations in the following program to show that the Hoare triple given by the outermost pre- and post-conditions is valid. Be completely precise and pedantic in the way you apply Hoare rules—write assertions in *exactly* the form given by the rules rather than just logically equivalent ones. The provided blanks have been constructed so that if you work backwards from the end of the program you should only need to use the rule of consequence in the places indicated with $\rightarrow$. These implication steps can (silently) rely on all the usual rules of arithmetic.

```
{ { m > 0 } }  $\rightarrow$ 
{ { 0 == (m-m) * (m+m)  $\wedge$  m >= 0  $\wedge$  y == m * m } }
  y := 0;
{ { y == (m-m) * (m+m)  $\wedge$  m >= 0  $\wedge$  y == m * m } }
  var x := m * m;
{ { y == (m-m) * x  $\wedge$  m >= 0  $\wedge$  x == m * m } }
  var z := m;
{ { y == (m-z) * x  $\wedge$  z >= 0  $\wedge$  x == m * m } }
  while (z > 0) {
    { { y == (m-z) * x  $\wedge$  z >= 0  $\wedge$  x == m * m  $\wedge$  z > 0 } }  $\rightarrow$ 
    { { y + x == (m-(z-1)) * x  $\wedge$  z-1 >= 0  $\wedge$  x == m * m } }
      z := z - 1;
    { { y + x == (m-z) * x  $\wedge$  z >= 0  $\wedge$  x == m * m } }
      y := y + x;
    { { y == (m-z) * x  $\wedge$  z >= 0  $\wedge$  x == m * m } }
  }
  { { y == (m-z) * x  $\wedge$  z >= 0  $\wedge$  x == m * m } }  $\rightarrow$ 
  { { y == m * m } }
```


Question 5 (20 points)

Implement the following Haskell functions:

- (a) Implement a function `weave` that given two lists with elements of the same type, returns a list with elements alternating between the two lists. For example:

```
weave [1,2,3] [4,5,6] = [1,4,2,5,3,6]
```

You can assume that the lists have the same length.

```
weave [1st1 1st2] = aux 1st1 1st2 True
```

where

```
aux (x:xs) 1st True = x:aux xs 1st False
```

```
aux 1st (y:ys) False = y:aux 1st ys True
```

```
aux _ _ _ = []
```

- (b) Implement a function `powerset`, that given a list of elements of some type, computes the list of all of its sublists, including the empty list and the original list itself, in any order. For example:

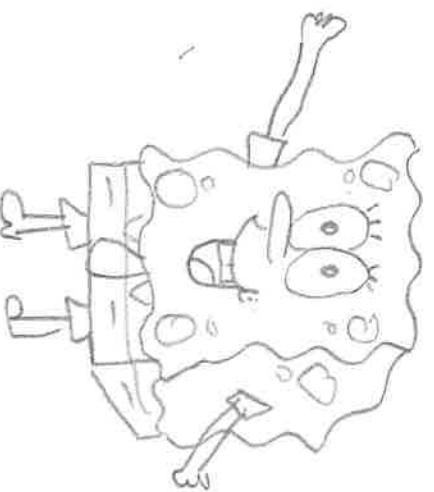
```
powerset [1,2,3] = [[], [1], [2], [3], [1,2], [1,3], [2,3], [1,2,3]]
```

$1 : []$

$x : xs$

This question was on my cmsc330 final and I still have no clue how to do it lol

```
powerset (x:xs) = x:xs: powerset xs
powerset [] = []
```



I hope you like
Krabby Pattys idk

